Proiect Statistica

Maria-Alexandra GHIȚĂ (Grupa 311)

Luis-Antonio HUSDUP (Grupa 311)

George-Eduard NEDELESCU (Grupa 311)

George-Robert MARCU (Grupa 311)

Capitoul 1 – Inegalitatea lui Kantorovich-prezentare generală

Exercițiul 1

1.1

```
m <- 2
M <- 7
set.seed(123)
n <- 6
vector_lambda <- sort(runif(n,m,M))
vector_lambda
```

```
## [1] 2.227782 3.437888 4.044885 5.941526 6.415087 6.702336
```

1.2

```
#definim laplace
my_rlaplace <- function(n, mu = 0, b = 1) {
  # Diferența a două exponențiale generează Laplace standard, apoi ajustăm cu mu și b
  u1 <- rexp(n)
  u2 <- rexp(n)
  return(mu + b * (u1 - u2))
}

selector <- 1
rate <- 1
y <- switch(
  selector,
  rnorm(n,0,1),
  rt(n,3),
  rt(n,4),
  rt(n,5),
  my_rlaplace(n),
  rexp(n,rate)
)
p <- (y^2)/sum(y^2)

p
```

```
## [1] 0.0009473502 0.0031852504 0.5605186647 0.0404830173 0.3049664137
## [6] 0.0898993038
```

1.3

```
T_pmf<- data.frame(
  Lambda_i = vector_lambda,
  Probabilitate  = p
)
T_pmf
```

```
##   Lambda_i Probabilitate
## 1 2.227782  0.0009473502
## 2 3.437888  0.0031852504
## 3 4.044885  0.5605186647
## 4 5.941526  0.0404830173
## 5 6.415087  0.3049664137
## 6 6.702336  0.0898993038
```

```
# Cautam in data.frame
probabilitatea <- T_pmf$Probabilitate[T_pmf$Lambda_i == vector_lambda[1]]
probabilitatea
```

```
## [1] 0.0009473502
```

1.4

```
#reordonam descrescator T_pmf dupa p_i(optimizare)
T_pmf_nou <- T_pmf %>% arrange(desc(Probabilitate))
T_pmf_nou
```

```
##   Lambda_i Probabilitate
## 1 4.044885  0.5605186647
## 2 6.415087  0.3049664137
## 3 6.702336  0.0898993038
## 4 5.941526  0.0404830173
## 5 3.437888  0.0031852504
## 6 2.227782  0.0009473502
```

```
F_inversa <- function(u,T_pmf_nou){
  suma <- 0
  for(i in 1:nrow(T_pmf_nou)){
    if(u>suma & u<=T_pmf_nou$Probabilitate[i]+suma){
      return(T_pmf_nou$Lambda_i[i])
    }
    suma=suma+T_pmf_nou$Probabilitate[i]
  }
}

N <- 10^6
T_observat <- replicate(N,{
  u <- runif(1)
  t <- F_inversa(u,T_pmf_nou)
  t
})

hist(T_observat)

T_empiric <- mean(T_observat)
T_invers_empiric <- mean(1/T_observat)
R_N_empiric <-T_empiric*T_invers_empiric

T_empiric
```

```
## [1] 5.078632
```

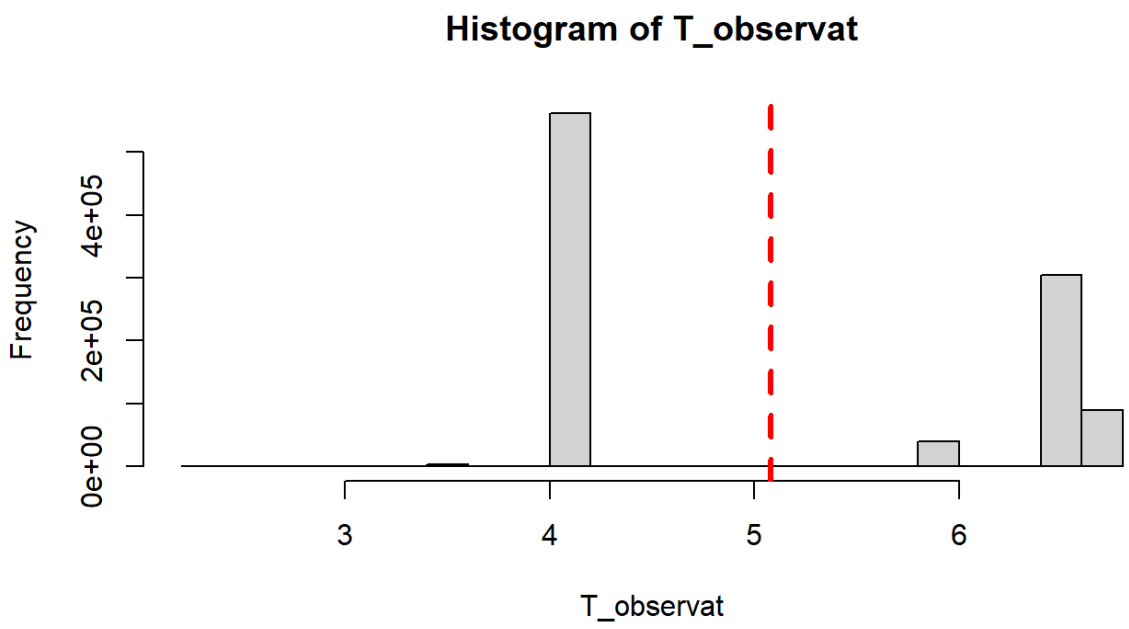
```
T_invers_empiric
```

```
## [1] 0.2077394
```

```
R_N_empiric
```

```
## [1] 1.055032
```

```
abline(v=T_empiric, col = "red", lwd = 3, lty = 2)
abline(v=T_invers_empiric, col = "red", lwd = 3, lty = 2)
```



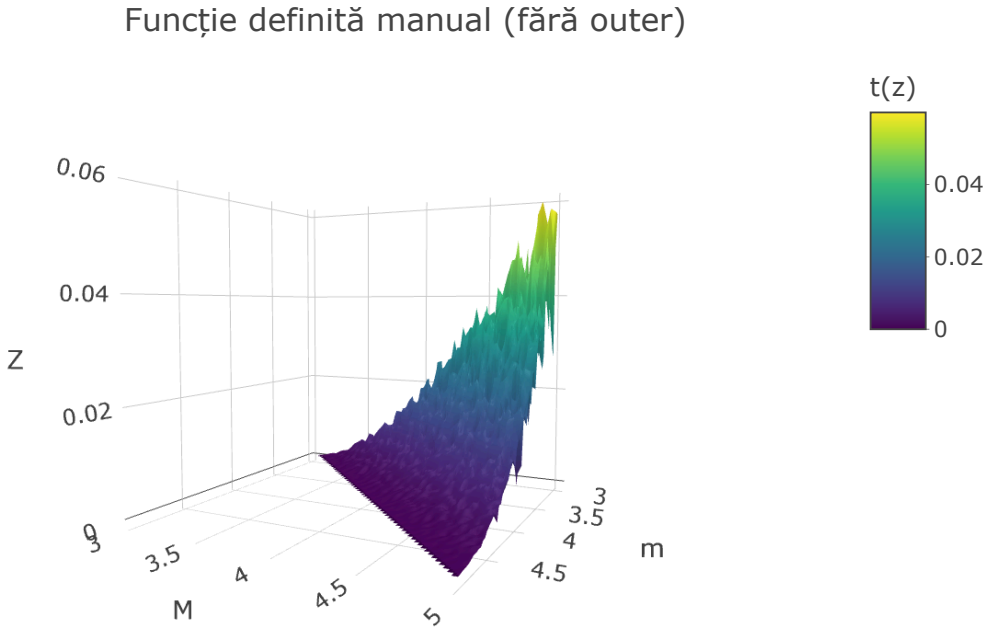
1.5

```
B <- (M+m)^2 / (4*M*m) #media teoretica a lui R_n
abs(R_N_empiric-B)
```

```
## [1] 0.3913969
```

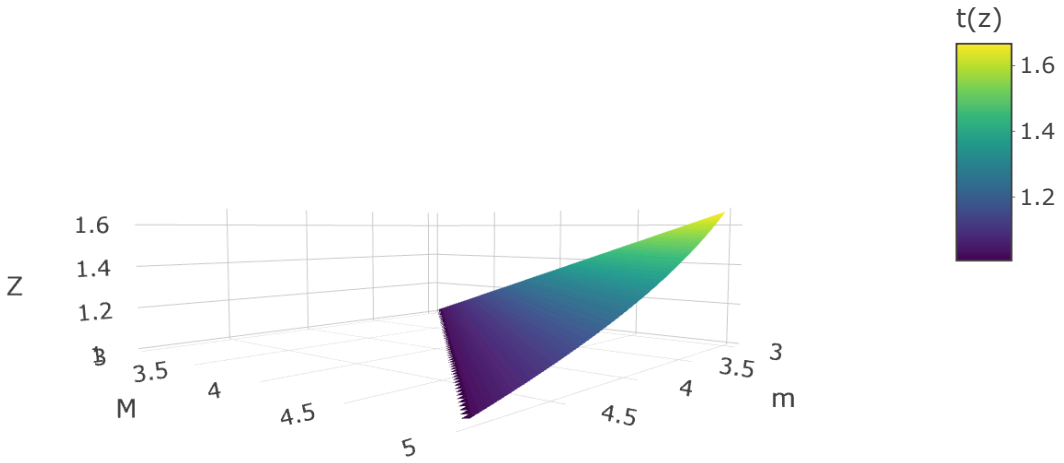
```r
m_vals <- seq(3, 5, length.out = 60) # Axa m
M_vals <- seq(3, 5, length.out = 60) # Axa M

f_mea <- function(m, M) {
  n <- 6
  vector_lambda <- sort(runif(n,m,M))
  selector <- 5
  rate <- 1
  y <- switch(
    selector,
    rnorm(n,0,1),
    rt(n,3),
    rt(n,4),
    rt(n,5),
    my_rlaplace(n),
    rexp(n,rate)
  )
  p <- (y^2)/sum(y^2)
  T_pmf<- data.frame(
    Lambda_i = vector_lambda,
    Probabilitate  = p
  )
  T_pmf_nou <- T_pmf %>% arrange(desc(Probabilitate))
  N <- 10^2
  T_observat <- replicate(N,{
    u <- runif(1)
    t <- F_inversa(u,T_pmf_nou)
    t
  })
  T_empiric <- mean(T_observat)
  T_invers_empiric <- mean(1/T_observat)
  R_N_empiric <-T_empiric*T_invers_empiric
  B <- (M+m)^2 / (4*M*m) #media teoretica a lui R_n
  return(abs(R_N_empiric-B))
}

z <- matrix(NA, nrow = length(m_vals), ncol = length(M_vals))

for(i in 1:length(m_vals)) {
  for(j in 1:length(M_vals)) {
    val_m <- m_vals[i]
    val_M <- M_vals[j]

    if(val_m < val_M) {
      z[i, j] <- f_mea(val_m, val_M)
    } else {
      z[i, j] <- NA
    }
  }
}
```

```
plot_ly(x = ~m_vals, y = ~M_vals, z = ~t(z)) %>%
  add_surface() %>%
  layout(
    title = "Funcție definită manual (fără outer)",
    scene = list(
      xaxis = list(title = "m"),
      yaxis = list(title = "M"),
      zaxis = list(title = "Z")
    )
  )
```




```
##################
z <- matrix(NA, nrow = length(m_vals), ncol = length(M_vals))

for(i in 1:length(m_vals)) {
  for(j in 1:length(M_vals)) {
    val_m <- m_vals[i]
    val_M <- M_vals[j]

    if(val_m < val_M) {
      z[i, j] <- val_M/val_m
    } else {
      z[i, j] <- NA
    }
  }
}
```

```
plot_ly(x = ~m_vals, y = ~M_vals, z = ~t(z)) %>%
  add_surface() %>%
  layout(
    title = "Funcție definită manual (fără outer)",
    scene = list(
      xaxis = list(title = "m"),
      yaxis = list(title = "M"),
      zaxis = list(title = "Z")
    )
  )
```



Exercițiul 2

2.1

```
N_valori <- c(10^2,10^3,10^4)
tabel_final <- data.frame()

T_media_teoretica <- vector_lambda %*% p #produs scalar
T_invers_media_teoretica <- (1/vector_lambda) %*% p

for(N in N_valori){
  T_observat <- replicate(N,{
    u <- runif(1)
    t <- F_inversa(u,T_pmf_nou)
    t
  })
  T_empiric <- mean(T_observat)
  T_invers_empiric <- mean(1/T_observat)

  rand_nou <- data.frame(
    N = N,
    T_Empiric = T_empiric,
    T_Teoretic = T_media_teoretica,

    T_Invers_Empiric = T_invers_empiric,
    T_Invers_Teoretic = T_invers_media_teoretica,

    Eroare_Abs = abs(T_empiric - T_media_teoretica),
    Eroare_Abs_Inversa = abs(T_invers_empiric-T_invers_media_teoretica)
  )

  tabel_final <- rbind(tabel_final, rand_nou)
}

tabel_final
```

```
##       N T_Empiric T_Teoretic T_Invers_Empiric T_Invers_Teoretic Eroare_Abs
## 1   100  5.209312   5.079747        0.2024395         0.2076921 0.12956522
## 2  1000  5.116944   5.079747        0.2065077         0.2076921 0.03719727
## 3 10000  5.093268   5.079747        0.2071893         0.2076921 0.01352095
##   Eroare_Abs_Inversa
## 1       0.0052525924
## 2       0.0011844382
## 3       0.0005028221
```

2.2

Pentru a justifica teoretic convergența estimatorului $\hat{R}_N$, ne bazăm pe *Legea Slabă a Numerelor Mari (LSNM) în formă vectorială* și pe *Teorema Aplicațiilor Continue (TAC)*.

Fie vectorii aleatori bidimensionali $X_1, X_2, \ldots, X_N$ independenți și identic distribuiți (i.i.d.), definiți astfel:

$$\mathbf{X}_i = \begin{pmatrix} T_i \\ \frac{1}{T_i} \end{pmatrix}, \quad i = 1, \ldots, N$$

unde T_i sunt variabile aleatoare cu suportul în intervalul finit $[m, M]$ ($m > 0$). Deoarece variabila este mărginită, momentele de ordinul 1 sunt finite.

Notăm vectorul mediilor teoretice cu $\boldsymbol{\mu}$:

$$\boldsymbol{\mu} = \mathbb{E}[\mathbf{X}_1] = \begin{pmatrix} \mathbb{E}[T] \\ \mathbb{E}[1/T] \end{pmatrix}$$

Definim estimatorul de selecție vectorial (media empirică vectorială):

$$\overline{\mathbf{X}}N = \frac{1}{N} \sum i = 1^N \mathbf{X}_i = \begin{pmatrix} \overline{T}_N \\ \overline{\left(\frac{1}{T}\right)}_N \end{pmatrix}$$

Estimatorul nostru pentru raportul Kantorovich este produsul componentelor acestui vector:

$$\hat{R}_N = \overline{T}_N \cdot \overline{\left(\frac{1}{T}\right)}_N$$

1. Convergența în probabilitate ($\overset{\mathbb{P}}{\to}$)

Argumentul se desfășoară în doi pași riguroși:

Pasul 1: Aplicarea Legii Slabe a Numerelor Mari (LSNM) Conform WLLN aplicată vectorial, media de selecție converge în probabilitate către media teoretică:

$$\overline{\mathbf{X}}_N \overset{\mathbb{P}}{\to} \boldsymbol{\mu}$$

Pasul 2: Aplicarea Teoremei Aplicațiilor Continue (TAC) Considerăm funcția $g : \mathbb{R}^2 \to \mathbb{R}$, definită prin produsul componentelor, $g(x, y) = x \cdot y$. Această funcție este continuă pe tot domeniul său (și implicit în punctul $\boldsymbol{\mu}$).

Conform Teoremei Aplicațiilor Continue (TAC), dacă $\mathbf{Z}_N \overset{\mathbb{P}}{\to} \mathbf{c}$, atunci $g(\mathbf{Z}_N) \overset{\mathbb{P}}{\to} g(\mathbf{c})$. Aplicând acest rezultat pentru $\mathbf{Z}_N = \overline{\mathbf{X}}_N$:

$$g(\overline{\mathbf{X}}_N) \overset{\mathbb{P}}{\to} g(\boldsymbol{\mu})$$

Înlocuind cu expresiile specifice problemei noastre, obținem convergența estimatorului către valoarea teoretică R:

$$\underbrace{\overline{T}N \cdot \overline{\left(\frac{1}{T}\right)}_N}\hat{R}N \xrightarrow{\mathbb{P}} \underbrace{\mathbb{E}[T] \cdot \mathbb{E}\left[\frac{1}{T}\right]}R$$

2. Convergența în distribuție ($\xrightarrow{d}$)

Deoarece am demonstrat că $\hat{R}_N \xrightarrow{\mathbb{P}} R$ (unde R este o constantă), rezultă imediat, conform proprietăților standard de convergență, că are loc și convergența în distribuție:

$$\hat{R}_N \xrightarrow{d} R$$

Aceasta explică rezultatele obținute prin simulare: pe măsură ce N crește, distribuția valorilor empirice $\hat{R}_N$ se concentrează (degenerează) în punctul R.

```
N <- 10^6
T_observat <- replicate(N,{
  u <- runif(1)
  t <- F_inversa(u,T_pmf_nou)
  t
})
T_empiric <- mean(T_observat)
T_invers_empiric <- mean(1/T_observat)

R_N_empiric <-T_empiric*T_invers_empiric
R_N_teoretic <- T_media_teoretica * T_invers_media_teoretica

rezultat <- data.frame(
  N = N,
  R_N_empiric = R_N_empiric,
  R_N_teoretic =R_N_teoretic,
  Eroare_absoluta=abs(R_N_empiric - R_N_teoretic)
)
rezultat
```

```
##       N R_N_empiric R_N_teoretic Eroare_absoluta
## 1 1e+06    1.055041     1.055023    1.795419e-05
```

2.3

Explicatie 2.3: Deși inegalitatea Kantorovich ($E[T]E[1/T] \leq \frac{(m+M)^2}{4mM}$) este o proprietate algebrică și teoretic ar trebui să fie respectată de orice eșantion cu valori în $[m, M]$, pentru N mic pot apărea "încălcări" sau anomalii aparente din două motive:

1. **Volatilitatea Estimatorului (Zgomot statistic):** Pentru N mic (ex: $N = 100$), estimatorii mediei $\hat{\mathbb{E}}_N(T)$ au o varianță foarte mare. Legea numerelor mari nu a stabilizat încă rezultatul. Deși produsul mediilor de eșantionare nu va depăși teoretic limita B (dacă calculăm B cu m, M globale), el poate fi foarte departe de valoarea reală a lui R (a populației), creând o imagine distorsionată a convergenței.
2. **Erori Numerice (Floating Point):** Calculatoarele au o precizie finită. Când lucrăm cu numere foarte mici sau foarte mari și facem multe operații (sume, împărțiri), erorile de rotunjire se pot acumula, ducând la situații rare unde o inegalitate strictă matematic (ex: $A/H \leq B$) poate fi încălcată la a 15-a zecimală.

Exercițiul 3

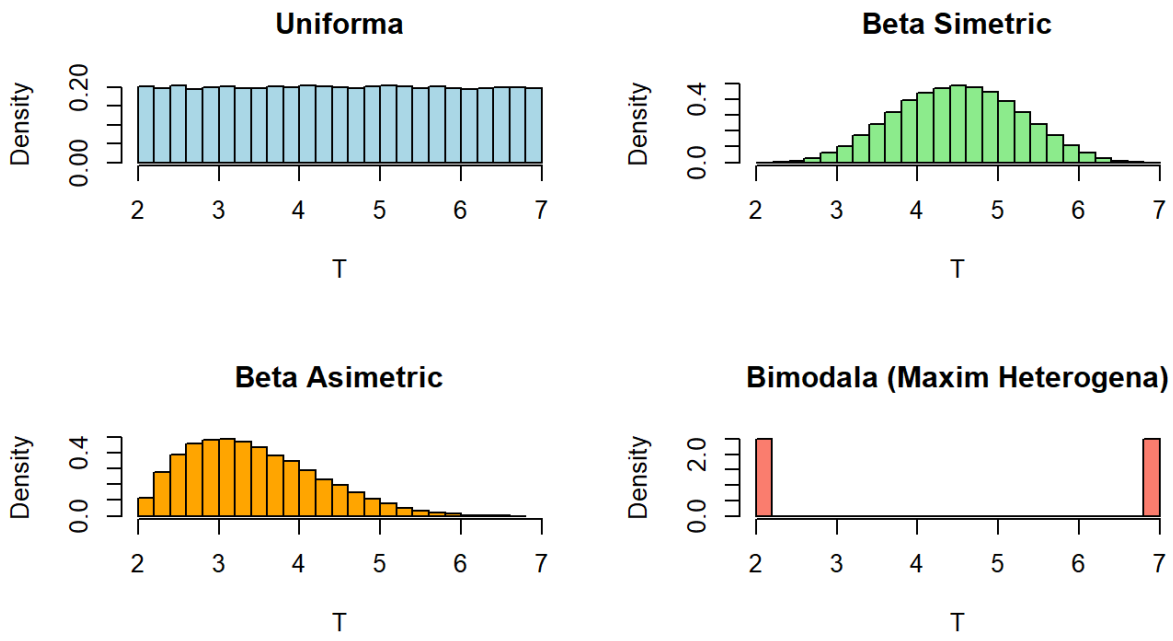
```
# 0. Setări inițiale (Contextul problemei)
m <- 2
M <- 7
B <- (m + M)^2 / (4 * m * M)  # Limita Kantorovich teoretică
cat("Limita Teoretică Kantorovich (B) este:", B, "\n\n")
```

```
## Limita Teoretică Kantorovich (B) este: 1.446429
```

```r
# Număr mare de simulări pentru precizie
N_sim <- 10^5

# Funcție pentru calculul estimatorilor și afișare
calculeaza_si_afiseaza <- function(nume_distributie, esantion_T) {
  # 3.2 Estimare A, H și Raportul A/H
  A_est <- mean(esantion_T)               # Media Aritmetică
  H_est <- 1 / mean(1 / esantion_T)       # Media Armonică
  Raport <- A_est / H_est

  # Verificare cu limita teoretică
  Diferenta <- B - Raport

  # Returnăm un rând pentru tabelul final
  return(data.frame(
    Distributie = nume_distributie,
    Media_Aritmetica = A_est,
    Media_Armonica = H_est,
    Raport_AH = Raport,
    Limita_Teoretica = B,
    Respecta_Inegalitatea = (Raport <= B + 0.001) # Toleranță mică pt erori numerice
  ))
}

# --- 3.1 Construcția celor 4 distribuții pe [m, M] ---

# 1. Distribuția Uniformă (Heterogenitate medie)
T_uniform <- runif(N_sim, m, M)

# 2. Distribuția Beta Simetrică (Rescalată pe [m, M])
# Parametrii alpha=beta=5 generează un "clopot" la mijloc.
# Valorile sunt concentrate în centru -> A și H vor fi apropiate.
T_beta_simetric <- m + (M - m) * rbeta(N_sim, 5, 5)

# 3. Distribuția Beta Asimetrică (Rescalată)
# Concentrare spre unul din capete.
T_beta_skew <- m + (M - m) * rbeta(N_sim, 2, 5)

# 4. Distribuția Bimodală (Amestec de două Uniforme la capete)
# Acesta este cazul extrem care testează limita.
# 50% din valori sunt foarte aproape de m, 50% foarte aproape de M.
u <- runif(N_sim)
epsilon <- 0.01 # interval foarte mic la capete
T_bimodal <- ifelse(u < 0.5,
                    runif(N_sim, m, m + epsilon),
                    runif(N_sim, M - epsilon, M))

# --- Centralizarea Rezultatelor ---

rezultate <- data.frame()
rezultate <- rbind(rezultate, calculeaza_si_afiseaza("Uniforma", T_uniform))
rezultate <- rbind(rezultate, calculeaza_si_afiseaza("Beta Simetric", T_beta_simetric))
rezultate <- rbind(rezultate, calculeaza_si_afiseaza("Beta Asimetric", T_beta_skew))
rezultate <- rbind(rezultate, calculeaza_si_afiseaza("Bimodala (Extreme)", T_bimodal))

# Afișarea tabelului final pentru 3.3 (Verificare numerică)
print(rezultate)
```

```
##          Distributie Media_Aritmetica Media_Armonica Raport_AH Limita_Teoretica
## 1           Uniforma         4.496018       3.987723  1.127465         1.446429
## 2      Beta Simetric         4.501961       4.369999  1.030197         1.446429
## 3     Beta Asimetric         3.430631       3.255869  1.053676         1.446429
## 4 Bimodala (Extreme)         4.491211       3.110570  1.443855         1.446429
##   Respecta_Inegalitatea
## 1                  TRUE
## 2                  TRUE
## 3                  TRUE
## 4                  TRUE
```

```
# Ilustrare grafică a diferențelor de distribuție (Opțional dar recomandat)
par(mfrow=c(2,2)) #afiseaza histogramele in acelasi plot
hist(T_uniform, main="Uniforma", xlim=c(m,M), col="lightblue", probability = T, xlab="T")
hist(T_beta_simetric, main="Beta Simetric", xlim=c(m,M), col="lightgreen", probability = T, xlab="T")
hist(T_beta_skew, main="Beta Asimetric", xlim=c(m,M), col="orange", probability = T, xlab="T")
hist(T_bimodal, main="Bimodala (Maxim Heterogena)", xlim=c(m,M), col="salmon", probability = T, xlab
="T")
```




```
par(mfrow=c(1,1))
```

3.3

Din rezultatele numerice observăm că, deși toate distribuțiile respectă limita superioară B, raportul A/H diferă semnificativ

Cea mai mică valoare a raportului (cea mai apropiată de 1) apare la Beta Simetric. Motivul: Valorile sunt concentrate în jurul mediei intervalului. Când varianța este mică, media aritmetică este aproape egală cu cea armonică. Heterogenitatea este minimă.

Cea mai mare valoare a raportului (cea mai apropiată de limita B) apare la Distribuția Bimodală. Motivul: Aceasta reprezintă “heterogenitate maximă”. Masa probabilistă este divizată între valorile minime posibile (m) și maxime posibile (M). Acest caz maximizează diferența dintre E[T] și 1/E[1/T], fiind scenariul pentru care inegalitatea Kantorovich atinge egalitatea.

3.4

Inegalitatea Kantorovich măsoară distanța maximă dintre curba funcției 1/t și coarda care unește punctele (m,1/m) și (M,1/M).

Deoarece funcția 1/t este strict convexă pe [m,M], această coardă (care corespunde mediei aritmetice a extremelor) se află întotdeauna deasupra curbei (care corespunde mediei armonice).

Punctul maxim al acestei distanțe este determinat strict de capetele intervalului m și M. Indiferent cum distribuim probabilitățile în interiorul intervalului [m,M], nu putem "împinge" raportul mai sus decât o facem punând toată greutatea în capete.

Astfel (B+b)^/4mM este o proprietate a intervalului, nu a modului în care variabila aleatoare se plimbă în interiorul acestuia.

Capitolul 2 – Familia exponențială, estimare și acuratețea aproximării normale (Berry-Esseen)

Exercițiul 4 (a. și b.)

Distribuția Binomială

$X \sim \mathrm{Bin}(m, p), \quad m \in \mathbb{N}, \quad p \in (0,1).$

$$P(X = k) = C_m^k p^k (1-p)^{m-k}, \quad \forall k \in \mathbb{N}.$$

$$\theta = p$$

Forma densității:

$$f_X(x;p) = C_m^x p^x (1-p)^{m-x}, \quad x \in \{0, \dots, m\}$$
$$\xrightarrow{\text{VREAU}} \quad \underbrace{h(x)}_{>0} \exp[\eta_1(p) T_1(x) - A(p)]$$

Analiza componentelor: 1. $h(x) = C_m^x > 0, \quad \forall x \in \{0, \dots, m\}$ 2. Prelucrarea exponențială:

$$p^x \xrightarrow{p>0} e^{\ln(p^x)} = e^{x \ln p} = \exp(x \ln p)$$

$$(1-p)^{m-x} \xrightarrow{p<1} e^{\ln((1-p)^{m-x})} = e^{(m-x)\ln(1-p)} = \exp((m-x)\ln(1-p))$$

Combinarea termenilor în exponent:

$$\begin{aligned}\exp(x \ln p + (m-x)\ln(1-p)) &= \exp(x(\ln p - \ln(1-p)) + m \ln(1-p)) \\ &= \exp\left[x \ln \frac{p}{1-p} + m \ln(1-p)\right]\end{aligned}$$

Identificarea parametrilor familiei exponențiale:

$$\eta_1(\theta) = \ln \frac{p}{1-p}, \quad T_1(x) = x, \quad A(\theta) = -m \ln(1-p)$$

Demonstrație 1: $T_1(x) = x$ statistică suficientă (prin definiție)

Calculăm probabilitatea condiționată:

$$P(X = x | T_1 = t) = \frac{P(\{X = x\} \cap \{T_1 = t\})}{P(T_1 = t)}$$

Cazuri:

$$= \begin{cases} \frac{P(X=t)}{P(T_1=t)} = \frac{C_m^t p^t (1-p)^{m-t}}{C_m^t p^t (1-p)^{m-t}} = 1, & \text{dacă } x = t \quad (\text{nu depinde de } p) \\ 0, & \text{dacă } x \neq t \quad (\text{nu depinde de } p) \end{cases}$$

Demonstrație 2: Factorizare Fisher-Neyman

Rescriem funcția finală:

$$f_X(x; p) = h(x) \cdot g(\underbrace{T(x)}_{x}, \theta) = C_m^x \exp\left[\ln \frac{p}{1-p} \cdot x - (-m \ln(1-p))\right]$$

Unde componenta dependentă de parametru este:

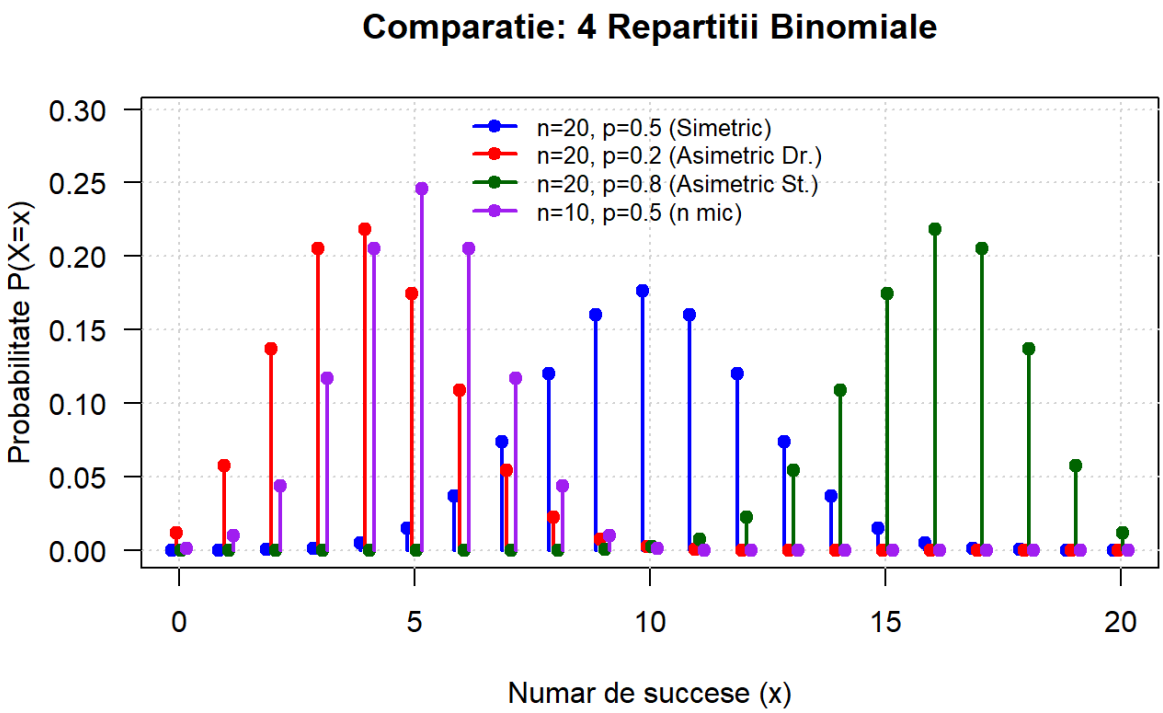
$$g(x, p) = e^{x \ln \frac{p}{1-p} - m \ln(1-p)}$$

$\leftarrow$ Factorizare Fisher-Neyman.

Deci $T_1(x) = x$ este statistică suficientă.

```r
# 1. Definirea domeniului de valori
# Luăm x până la maximul n-ului pe care îl vom folosi (aici 20)
x <- 0:20

# 2. Definirea celor 4 seturi de parametri
# Set 1: Simetric
n1 <- 20; p1 <- 0.5
# Set 2: Asimetric dreapta
n2 <- 20; p2 <- 0.2
# Set 3: Asimetric stanga
n3 <- 20; p3 <- 0.8
# Set 4: n mai mic
n4 <- 10; p4 <- 0.5

# Calculăm probabilitățile pentru fiecare
y1 <- dbinom(x, size = n1, prob = p1)
y2 <- dbinom(x, size = n2, prob = p2)
y3 <- dbinom(x, size = n3, prob = p3)
y4 <- dbinom(x, size = n4, prob = p4)

# 3. Pregătirea graficului
y_max <- max(y1, y2, y3, y4)

plot(x, y1, type = "n",
     ylim = c(0, y_max + 0.05),
     xlim = c(0, 20),
     main = "Comparatie: 4 Repartitii Binomiale",
     xlab = "Numar de succese (x)",
     ylab = "Probabilitate P(X=x)",
     las = 1)

grid(nx = NULL, ny = NULL, col = "lightgray", lty = "dotted")

# 4. Serii de date (offset-uri mici ca sa nu se suprapuna)
off1 <- -0.15
segments(x + off1, 0, x + off1, y1, col = "blue", lwd = 2)
points(x + off1, y1, pch = 19, col = "blue", cex = 1)

off2 <- -0.05
segments(x + off2, 0, x + off2, y2, col = "red", lwd = 2)
points(x + off2, y2, pch = 19, col = "red", cex = 1)

off3 <- 0.05
segments(x + off3, 0, x + off3, y3, col = "darkgreen", lwd = 2)
points(x + off3, y3, pch = 19, col = "darkgreen", cex = 1)

off4 <- 0.15
segments(x + off4, 0, x + off4, y4, col = "purple", lwd = 2)
points(x + off4, y4, pch = 19, col = "purple", cex = 1)

# 5. Legenda
legend("top",
       legend = c(
         paste0("n=", n1, ", p=", p1, " (Simetric)"),
         paste0("n=", n2, ", p=", p2, " (Asimetric Dr.)"),
         paste0("n=", n3, ", p=", p3, " (Asimetric St.)"),
         paste0("n=", n4, ", p=", p4, " (n mic)")
       ),
       col = c("blue", "red", "darkgreen", "purple"),
       pch = 19,
```

```
        lwd = 2,
        bty = "n",
        cex = 0.8)
```



Distribuția Geometrică

Fie $X \sim \text{Geom}(p), \quad p \in (0,1)$

$$P(X=k) = p(1-p)^{k-1}, \quad k \in \{1,\ldots\}, \quad \theta = p$$

Forma densității și obiectivul:

$$f_X(x;p) = p(1-p)^{x-1} \quad \xrightarrow{\text{VREAU}} \quad \underbrace{h(x)}_{\equiv 1 > 0} \cdot \exp[\eta_1(p)T_1(x) - A(p)]$$

Transformarea exponențială:

$$p(1-p)^{x-1} = \exp\left[\underbrace{\ln p}_{-A(p)} + \underbrace{(x-1)}_{T_1(x)}\underbrace{\ln(1-p)}_{\eta_1(p)}\right]$$

Întrebare: E $T_1(x) = x - 1$ statistică suficientă?

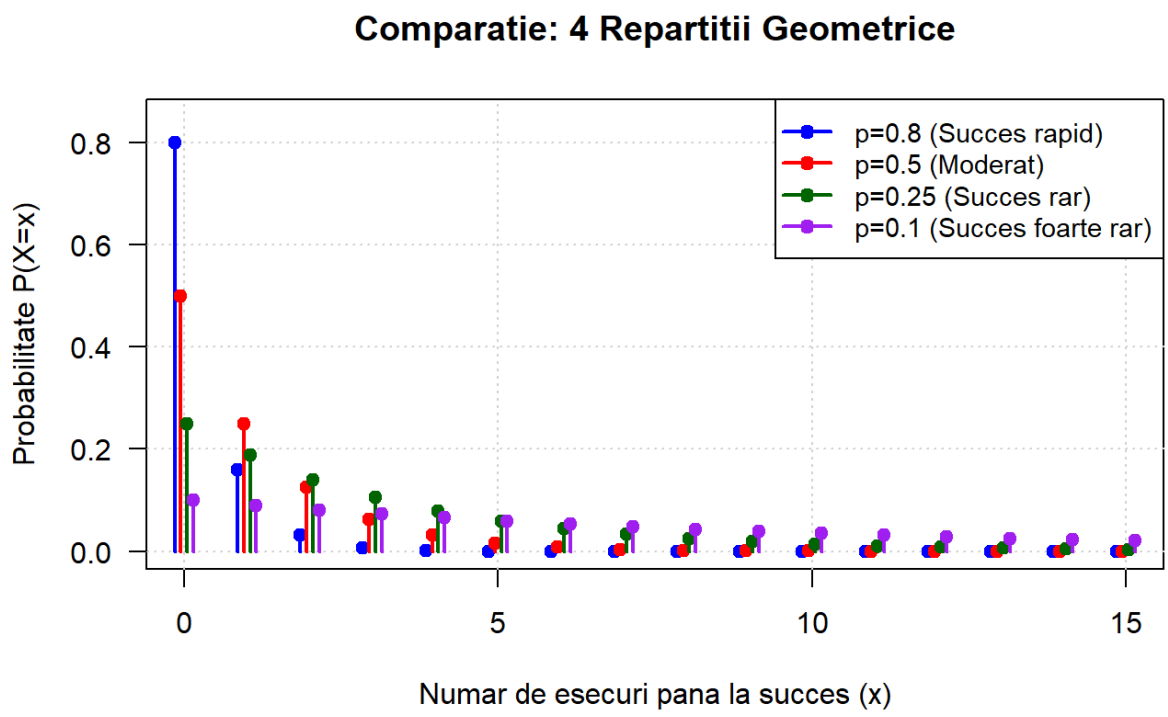
Verificare prin factorizare:

$$T_1(x) = x - 1 \text{ statistică suficientă}$$

$$f_X(x;p) = \underbrace{h(x)}_{\equiv 1} \cdot g(T(x),p) = 1 \cdot e^{\underbrace{(x-1)}_{T(x)}\ln(1-p) + \ln p}$$

Deci T_1 st. suf. (Statistică suficientă). Deci X e in clasa exp. (Familia Exponențială).

```r
# 1. Domeniu (geometrica are coada infinita, ne oprim la 15)
x <- 0:15

# 2. Parametri (p)
p1 <- 0.8
p2 <- 0.5
p3 <- 0.25
p4 <- 0.1

# PMF
y1 <- dgeom(x, prob = p1)
y2 <- dgeom(x, prob = p2)
y3 <- dgeom(x, prob = p3)
y4 <- dgeom(x, prob = p4)

# 3. Pregatirea graficului
y_max <- max(y1, y2, y3, y4)

plot(x, y1, type = "n",
     ylim = c(0, y_max + 0.05),
     xlim = c(0, 15),
     main = "Comparatie: 4 Repartitii Geometrice",
     xlab = "Numar de esecuri pana la succes (x)",
     ylab = "Probabilitate P(X=x)",
     las = 1)

grid(nx = NULL, ny = NULL, col = "lightgray", lty = "dotted")

# 4. Serii (cu offset)
off1 <- -0.15
segments(x + off1, 0, x + off1, y1, col = "blue", lwd = 2)
points(x + off1, y1, pch = 19, col = "blue", cex = 1)

off2 <- -0.05
segments(x + off2, 0, x + off2, y2, col = "red", lwd = 2)
points(x + off2, y2, pch = 19, col = "red", cex = 1)

off3 <- 0.05
segments(x + off3, 0, x + off3, y3, col = "darkgreen", lwd = 2)
points(x + off3, y3, pch = 19, col = "darkgreen", cex = 1)

off4 <- 0.15
segments(x + off4, 0, x + off4, y4, col = "purple", lwd = 2)
points(x + off4, y4, pch = 19, col = "purple", cex = 1)

# 5. Legenda
legend("topright",
       legend = c(
         paste0("p=", p1, " (Succes rapid)"),
         paste0("p=", p2, " (Moderat)"),
         paste0("p=", p3, " (Succes rar)"),
         paste0("p=", p4, " (Succes foarte rar)")
       ),
       col = c("blue", "red", "darkgreen", "purple"),
       pch = 19,
       lwd = 2,
       bg = "white",
       cex = 0.9)
```



Distribuția Poisson

$X \sim \text{Poisson}(\lambda), \quad \lambda \in (0, \infty)$

$$P(X = k) = \frac{\lambda^k e^{-\lambda}}{k!}, \quad k \in \mathbb{N}$$

Scrierea sub formă de familie exponențială:

$$f_X(x, \lambda) = \frac{1}{x!} \cdot \lambda^x \cdot e^{-\lambda} \quad \xrightarrow{\text{VREM}} \quad \underbrace{h(x)}_{\substack{\| \\ \frac{1}{x!} > 0}} \cdot \exp[\eta_1(\lambda) \cdot T_1(x) - A(\lambda)]$$

Transformarea termenilor dependenți de parametru:

$$\lambda^x \cdot e^{-\lambda} = e^{\ln(\lambda^x e^{-\lambda})} = e^{\overbrace{T_1(x) \cdot \eta_1(\lambda)}^{x \ln \lambda} - \overbrace{A(\lambda)}^{-\lambda}}$$

Identificarea componentelor (din zonele evidențiate):

$$\eta_1(\lambda) = \ln \lambda, \quad T_1(x) = x, \quad A(\lambda) = \lambda$$

Reconstituirea funcției:

$$\text{Deci } f_X(x, \lambda) = h(x) \cdot g(T_1(x), \lambda) = \frac{1}{x!} \cdot e^{x \ln \lambda - \lambda}$$

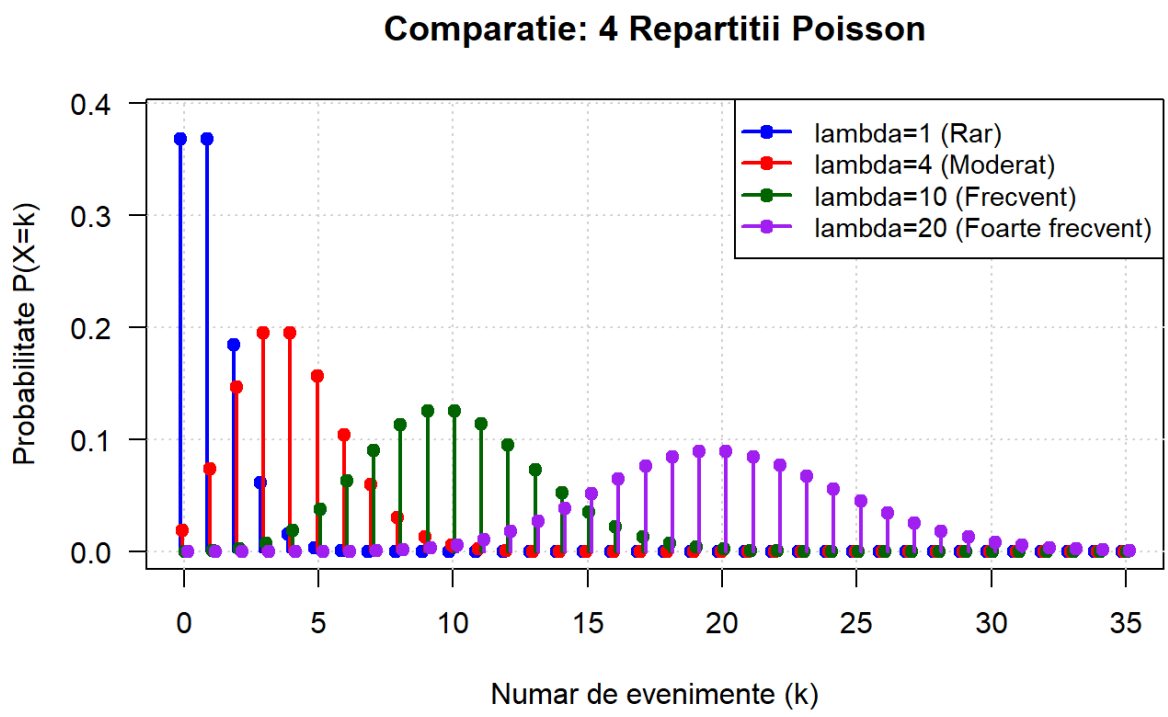
Concluzii:

$$\text{Deci } T_1 \text{ st. suf. (Statistică suficientă).}$$

$$\text{Deci } X \text{ în cls. exp. (Clasa Exponențială).}$$

—

```
x <- 0:35

lam1 <- 1
lam2 <- 4
lam3 <- 10
lam4 <- 20

y1 <- dpois(x, lambda = lam1)
y2 <- dpois(x, lambda = lam2)
y3 <- dpois(x, lambda = lam3)
y4 <- dpois(x, lambda = lam4)

y_max <- max(y1, y2, y3, y4)

plot(x, y1, type = "n",
     ylim = c(0, y_max + 0.02),
     xlim = c(0, 35),
     main = "Comparatie: 4 Repartitii Poisson",
     xlab = "Numar de evenimente (k)",
     ylab = "Probabilitate P(X=k)",
     las = 1)

grid(nx = NULL, ny = NULL, col = "lightgray", lty = "dotted")

off1 <- -0.15
segments(x + off1, 0, x + off1, y1, col = "blue", lwd = 2)
points(x + off1, y1, pch = 19, col = "blue", cex = 1)

off2 <- -0.05
segments(x + off2, 0, x + off2, y2, col = "red", lwd = 2)
points(x + off2, y2, pch = 19, col = "red", cex = 1)

off3 <- 0.05
segments(x + off3, 0, x + off3, y3, col = "darkgreen", lwd = 2)
points(x + off3, y3, pch = 19, col = "darkgreen", cex = 1)

off4 <- 0.15
segments(x + off4, 0, x + off4, y4, col = "purple", lwd = 2)
points(x + off4, y4, pch = 19, col = "purple", cex = 1)

legend("topright",
       legend = c(
         paste0("lambda=", lam1, " (Rar)"),
         paste0("lambda=", lam2, " (Moderat)"),
         paste0("lambda=", lam3, " (Frecvent)"),
         paste0("lambda=", lam4, " (Foarte frecvent)")
       ),
       col = c("blue", "red", "darkgreen", "purple"),
       pch = 19,
       lwd = 2,
       bg = "white",
       cex = 0.9)
```



Distribuția Exponențială

$X \sim \text{Exp}(\lambda), \quad \lambda \in (0, \infty)$

$$P(X = x) = \lambda e^{-\lambda x}, \quad x \in [0, \infty)$$

$$f_X(x, \lambda) = \lambda e^{-\lambda x} \quad \xrightarrow{\text{VREAU}} \quad \underbrace{h(x)}_{=1} \cdot \exp[\eta_1(\lambda) T_1(x) - A(\lambda)]$$

Transformarea parametrilor:

$$\lambda = e^{\ln \lambda} \implies \lambda e^{-\lambda x} = e^{-\lambda x + \ln \lambda}$$

Identificarea termenilor:

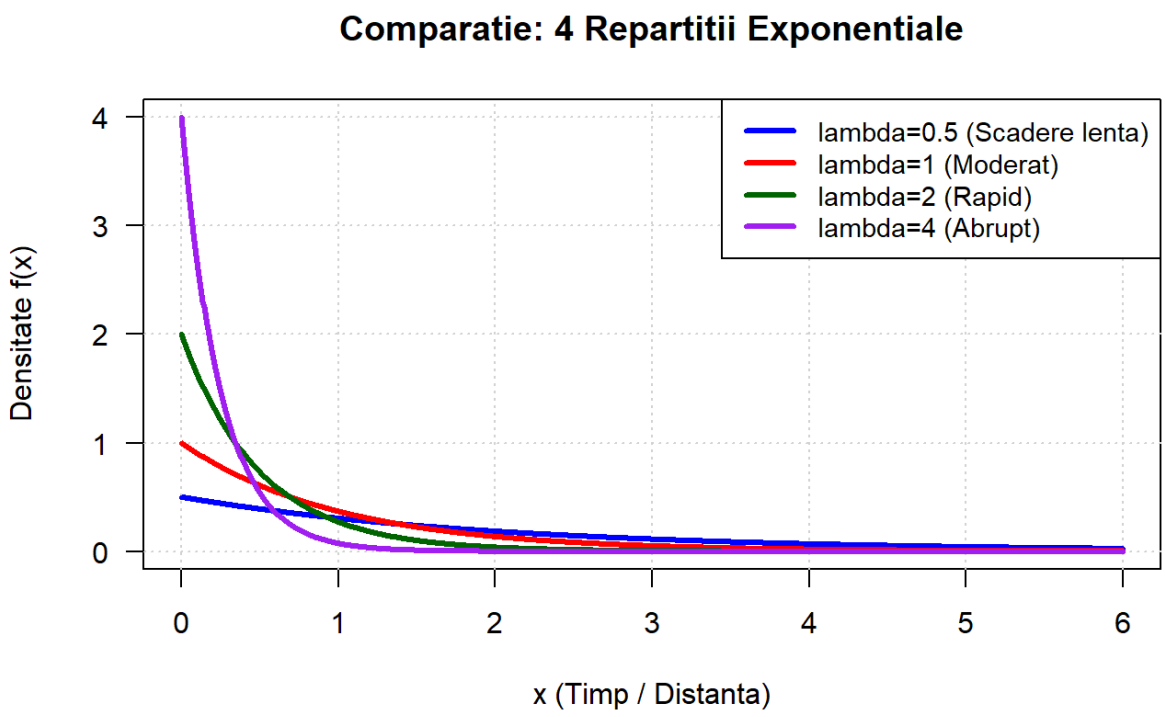
$$\eta_1(\lambda) = -\lambda, \quad T_1(x) = x, \quad A(\lambda) \approx \ln \lambda$$

Deci, rescrierea finală:

$$f_X(x, \lambda) = h(x) \cdot g(T(x), \lambda) = 1 \cdot e^{-\lambda \underbrace{x}_{T_1(x)} + \ln \lambda} \implies T_1 \text{ st. suf.}$$

Deci X in cls. exp. (Clasa Exponențială).

```r
x <- seq(0, 6, length.out = 1000)

lam1 <- 0.5
lam2 <- 1.0
lam3 <- 2.0
lam4 <- 4.0

y1 <- dexp(x, rate = lam1)
y2 <- dexp(x, rate = lam2)
y3 <- dexp(x, rate = lam3)
y4 <- dexp(x, rate = lam4)

y_max <- max(lam1, lam2, lam3, lam4)

plot(x, y1, type = "n",
     ylim = c(0, y_max),
     xlim = c(0, 6),
     main = "Comparatie: 4 Repartitii Exponentiale",
     xlab = "x (Timp / Distanta)",
     ylab = "Densitate f(x)",
     las = 1)

grid(nx = NULL, ny = NULL, col = "lightgray", lty = "dotted")

lines(x, y1, col = "blue", lwd = 3)
lines(x, y2, col = "red", lwd = 3)
lines(x, y3, col = "darkgreen", lwd = 3)
lines(x, y4, col = "purple", lwd = 3)

legend("topright",
       legend = c(
         paste0("lambda=", lam1, " (Scadere lenta)"),
         paste0("lambda=", lam2, " (Moderat)"),
         paste0("lambda=", lam3, " (Rapid)"),
         paste0("lambda=", lam4, " (Abrupt)")
       ),
       col = c("blue", "red", "darkgreen", "purple"),
       lty = 1,
       lwd = 3,
       bg = "white",
       cex = 0.9)
```



Distribuția Gamma

$X \sim \text{Gamma}(\alpha, \beta), \quad \alpha, \beta \in (0, \infty)$

$$f_X(x; \alpha, \beta) = \frac{\beta^\alpha}{\Gamma(\alpha)} \cdot x^{\alpha-1} \cdot e^{-\beta x}, \quad x \in (0, \infty), \quad \theta = (\alpha, \beta).$$

(Notă: $\Gamma(\alpha) = \int_0^\infty t^{\alpha-1} e^{-t} dt$)

Obiectivul ("VREM"):

$$f_X(x; \alpha, \beta) = \underbrace{h(x)}_{\equiv 1} \cdot \exp\left[\sum_{i=1}^{2} (\eta_i(\theta) T_i(x) - A(\theta))\right], \quad \forall x \in (0, \infty).$$

Pentru $\alpha > 0, \Gamma(\alpha) > 0$, pot lua $\ln$. Transformarea în exponent:

$$e^{\ln\left(\frac{\beta^\alpha}{\Gamma(\alpha)} \cdot x^{\alpha-1}\right)} e^{-\beta x} = e^{\underbrace{\alpha \ln \beta - \ln \Gamma(\alpha)}_{-A(\theta)} + \underbrace{(\alpha-1)}_{\eta_1(\theta)} \underbrace{\ln x}_{T_1(x)} \underbrace{-\beta}_{\eta_2(\theta)} \underbrace{x}_{T_2(x)}}$$

Identificare Statistici Suficiente:

$$T_1(x) = \ln x \quad \text{și} \quad T_2(x) = x \quad \text{sunt st. suf.?}$$

Verificare prin factorizare:

$$f_X(x; \alpha, \beta) = \underbrace{h(x)}_{\equiv 1} \cdot g(T_1(x), T_2(x), \theta)$$

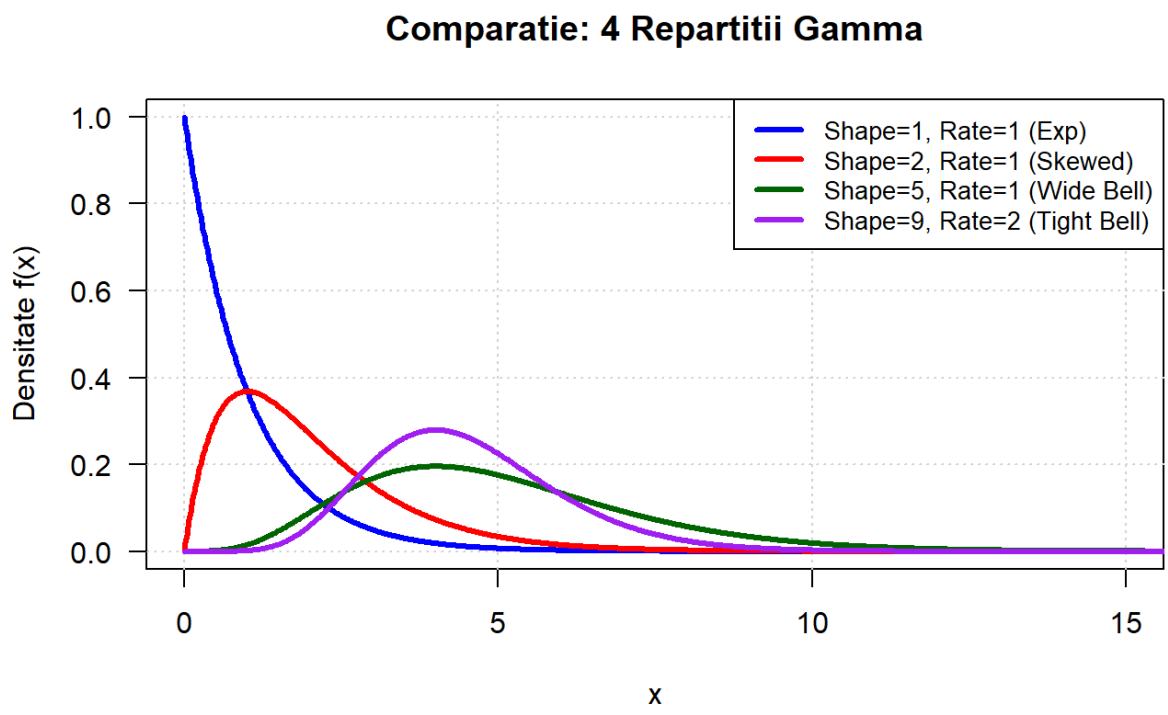
$$= \exp\left((\alpha - 1)\underbrace{\ln x}_{T_1(x)} - \beta \underbrace{x}_{T_2(x)} + \alpha \ln \beta - \ln \Gamma(\alpha)\right) \implies T_1, T_2 \text{ st. suf.}$$

Sau revenind la forma inițială pentru verificare:

$$= \underbrace{e^{-\beta x + \alpha \ln \beta - \ln \Gamma(\alpha)}}_{g_2(T_2,\theta)} \cdot \underbrace{e^{(\alpha-1)\ln x}}_{(x^{\alpha-1}) = g_1(T_1,\theta)}$$

Deci Gamma în cls. exp. (Clasa Exponențială).

```
x <- seq(0, 20, length.out = 1000)

shape1 <- 1; rate1 <- 1
shape2 <- 2; rate2 <- 1
shape3 <- 5; rate3 <- 1
shape4 <- 9; rate4 <- 2

y1 <- dgamma(x, shape = shape1, rate = rate1)
y2 <- dgamma(x, shape = shape2, rate = rate2)
y3 <- dgamma(x, shape = shape3, rate = rate3)
y4 <- dgamma(x, shape = shape4, rate = rate4)

y_max <- max(y1, y2, y3, y4)

plot(x, y1, type = "n",
     ylim = c(0, y_max),
     xlim = c(0, 15),
     main = "Comparatie: 4 Repartitii Gamma",
     xlab = "x",
     ylab = "Densitate f(x)",
     las = 1)

grid(nx = NULL, ny = NULL, col = "lightgray", lty = "dotted")

lines(x, y1, col = "blue", lwd = 3)
lines(x, y2, col = "red", lwd = 3)
lines(x, y3, col = "darkgreen", lwd = 3)
lines(x, y4, col = "purple", lwd = 3)

legend("topright",
       legend = c(
         paste0("Shape=", shape1, ", Rate=", rate1, " (Exp)"),
         paste0("Shape=", shape2, ", Rate=", rate2, " (Skewed)"),
         paste0("Shape=", shape3, ", Rate=", rate3, " (Wide Bell)"),
         paste0("Shape=", shape4, ", Rate=", rate4, " (Tight Bell)")
       ),
       col = c("blue", "red", "darkgreen", "purple"),
       lty = 1,
       lwd = 3,
       bg = "white",
       cex = 0.85)
```



Distribuția Beta

$X \sim \text{Beta}(\alpha, \beta), \quad \alpha, \beta \in (0, \infty)$

$$P(X = x) = \frac{1}{B(\alpha, \beta)} \cdot x^{\alpha-1} \cdot (1-x)^{\beta-1}, \quad x \in (0,1), \quad \theta = (\alpha, \beta)$$

Notație:

$$B(\alpha, \beta) = \frac{\Gamma(\alpha)\Gamma(\beta)}{\Gamma(\alpha+\beta)} > 0$$

Obiectivul ("VREAU"):

$$f_X(x; \alpha, \beta) \xrightarrow{\text{VREAU}} \underbrace{h(x)}_{\equiv 1} \cdot \exp\left(\sum_{i=1}^{2} \eta_i(\theta) T_i(x) - A(\theta)\right)$$

Transformarea exponențială:

$$e^{\ln\left(\frac{1}{B(\alpha,\beta)} \cdot x^{\alpha-1} \cdot (1-x)^{\beta-1}\right)} = e^{\underbrace{-\ln B(\alpha,\beta)}_{A(\theta)} + \underbrace{(\alpha-1)}_{\eta_1(\theta)} \underbrace{\ln x}_{T_1(x)} + \underbrace{(\beta-1)}_{\eta_2(\theta)} \underbrace{\ln(1-x)}_{T_2(x)}}$$

Încercare de verificare pentru T_1:

$$T_1(x) = \ln x \text{ st. suf.?}$$

$$f_X(x, \alpha, \beta) = \underbrace{h(x)}_{\equiv 1} \cdot g(T_1(x), \theta) = e^{A(\theta) + (\alpha-1) \underbrace{\ln x}_{T_1(x)} + (\beta-1) \ln(1 - e^{\overbrace{T_1(x)}^{\ln x}})}$$

Observație pentru T_2:

$$T_2(x) = \ln(1-x)$$

$$f_X(x,(\alpha,\beta)) = h(T_2(x),\theta) = e^{A(\theta)+(\beta-1)\underbrace{\ln(1-x)}_{T_2(x)}+(\alpha-1)\ln x}$$

Formalizare: Densitatea se scrie sub forma canonică a familiei exponențiale de ordin 2:

$$f_X(x) = h(x)\exp\big(\eta^T \cdot \mathbf{T}(x) - A(\eta)\big)$$

Unde vectorul statisticilor suficiente este $\mathbf{T}(x) = (\ln x, \ln(1-x))$.

Verificare prin Factorizare Fisher-Neyman: Funcția poate fi descompusă în:

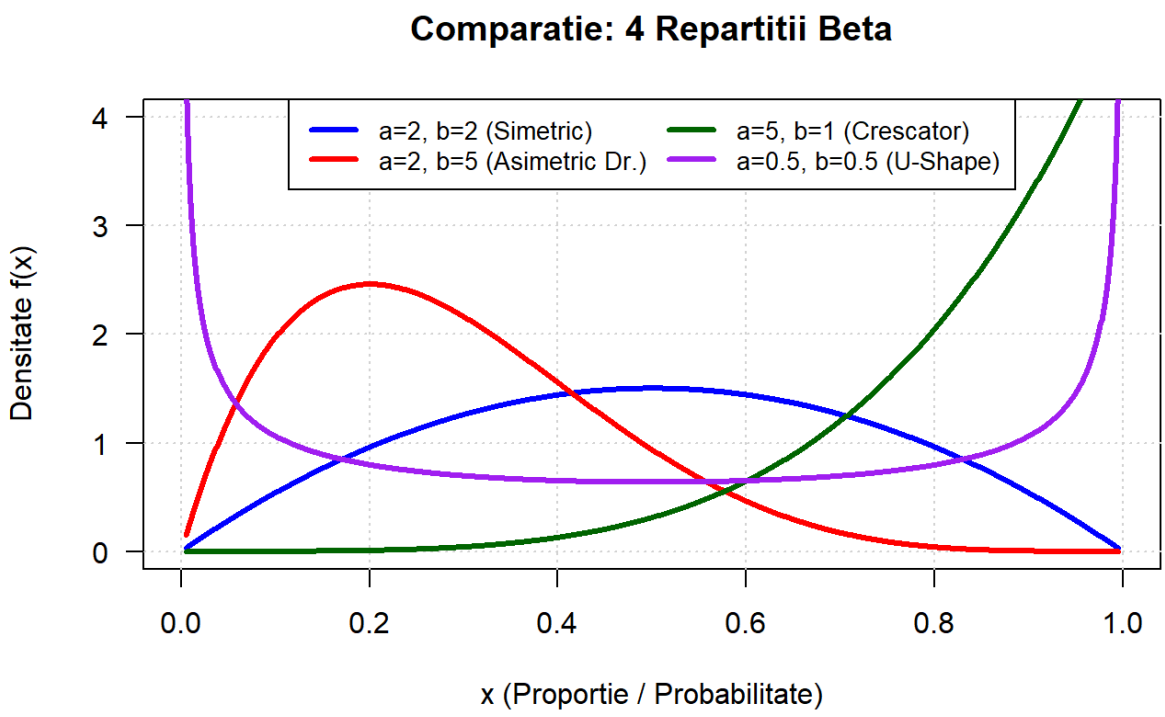
$$h(x) = 1$$

$$g(T(x),\theta) = \frac{1}{B(\alpha,\beta)} e^{(\alpha-1)\ln x+(\beta-1)\ln(1-x)}$$

Deoarece g depinde de date (x) doar prin intermediul vectorului $(\ln x, \ln(1-x))$, rezultă că $T(x) = (\ln x, \ln(1-x))$ sunt **statistici suficiente comune** pentru parametrii (α,β).

Concluzie: Distribuția Beta aparține Familiei Exponențiale.

```r
x <- seq(0.005, 0.995, length.out = 1000)

a1 <- 2; b1 <- 2
a2 <- 2; b2 <- 5
a3 <- 5; b3 <- 1
a4 <- 0.5; b4 <- 0.5

y1 <- dbeta(x, shape1 = a1, shape2 = b1)
y2 <- dbeta(x, shape1 = a2, shape2 = b2)
y3 <- dbeta(x, shape1 = a3, shape2 = b3)
y4 <- dbeta(x, shape1 = a4, shape2 = b4)

plot(x, y1, type = "n",
     ylim = c(0, 4),
     xlim = c(0, 1),
     main = "Comparatie: 4 Repartitii Beta",
     xlab = "x (Proportie / Probabilitate)",
     ylab = "Densitate f(x)",
     las = 1)

grid(nx = NULL, ny = NULL, col = "lightgray", lty = "dotted")

lines(x, y1, col = "blue", lwd = 3)
lines(x, y2, col = "red", lwd = 3)
lines(x, y3, col = "darkgreen", lwd = 3)
lines(x, y4, col = "purple", lwd = 3)

legend("top",
       legend = c(
         paste0("a=", a1, ", b=", b1, " (Simetric)"),
         paste0("a=", a2, ", b=", b2, " (Asimetric Dr.)"),
         paste0("a=", a3, ", b=", b3, " (Crescator)"),
         paste0("a=", a4, ", b=", b4, " (U-Shape)")
       ),
       col = c("blue", "red", "darkgreen", "purple"),
       lty = 1,
       lwd = 3,
       bg = "white",
       ncol = 2,
       cex = 0.85)
```



Distribuția Uniformă

$X \sim \text{Unif}(a, b), \quad -\infty < a < b < +\infty$

$$P(X = x) = \frac{1}{b-a} \cdot 1_{(a,b)}(x), \quad \forall x \in \mathbb{R}.$$

Parametrii:

$$\theta = (a, b)$$

Încercare de scriere în forma exponențială:

$$f_X(x; (a, b)) \overset{\text{VREAU}}{=} \underbrace{h(x)}_{=\frac{1}{b-a}} \cdot \exp\left[\sum_{i=1}^{2} \eta_i(\theta) T_i(x) - A(\theta)\right]$$

Problema (zona încercuită):

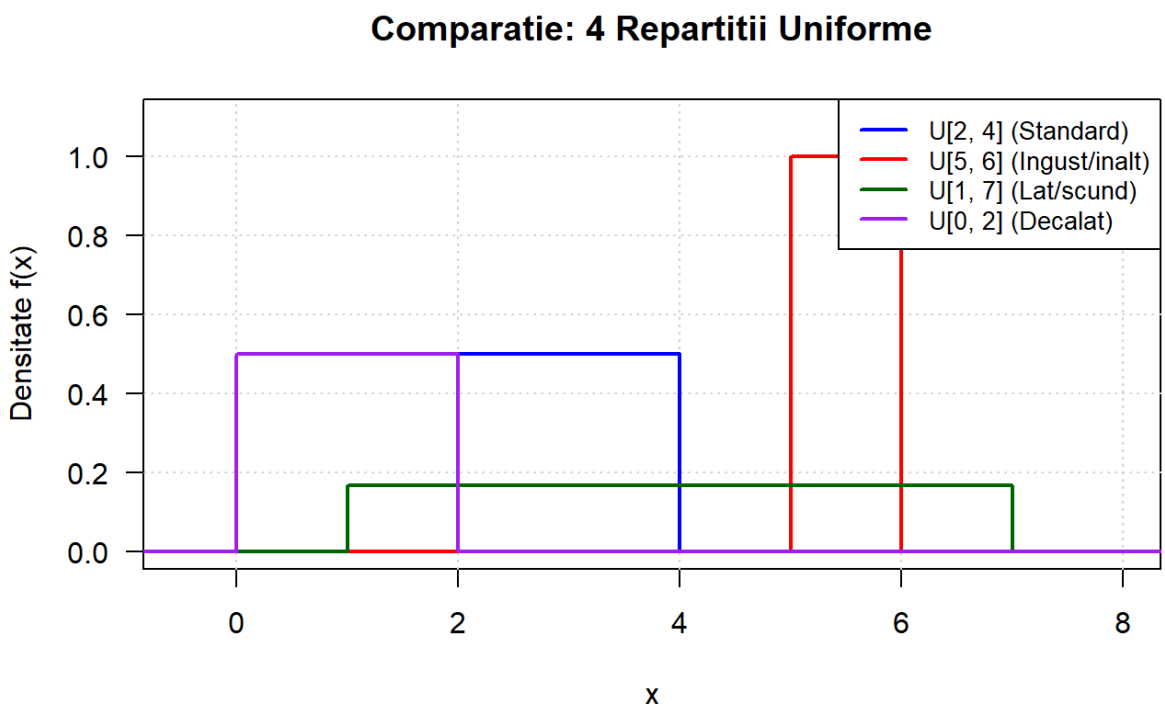
$$1_{(a,b)}(x) = \exp\left[\sum_{i=1}^{2} \eta_i(\theta) T_i(x) - A(\theta)\right]$$

Analiza suportului (diagrama de jos):

$$\text{supp} = [a, b] \quad (\text{depinde de parametri})$$

$$\text{fct. continuă} \quad \text{vs} \quad \text{supp} = \mathbb{R} \cup \{-\infty, +\infty\}$$

```r
x <- seq(-1, 9, length.out = 5000)

min1 <- 2; max1 <- 4
min2 <- 5; max2 <- 6
min3 <- 1; max3 <- 7
min4 <- 0; max4 <- 2

y1 <- dunif(x, min = min1, max = max1)
y2 <- dunif(x, min = min2, max = max2)
y3 <- dunif(x, min = min3, max = max3)
y4 <- dunif(x, min = min4, max = max4)

y_max <- max(y1, y2, y3, y4)

plot(x, y1, type = "n",
     ylim = c(0, y_max + 0.1),
     xlim = c(-0.5, 8),
     main = "Comparatie: 4 Repartitii Uniforme",
     xlab = "x",
     ylab = "Densitate f(x)",
     las = 1)

grid(nx = NULL, ny = NULL, col = "lightgray", lty = "dotted")

lines(x, y1, col = "blue", lwd = 2)
lines(x, y2, col = "red", lwd = 2)
lines(x, y3, col = "darkgreen", lwd = 2)
lines(x, y4, col = "purple", lwd = 2)

legend("topright",
       legend = c(
         paste0("U[", min1, ", ", max1, "] (Standard)"),
         paste0("U[", min2, ", ", max2, "] (Ingust/inalt)"),
         paste0("U[", min3, ", ", max3, "] (Lat/scund)"),
         paste0("U[", min4, ", ", max4, "] (Decalat)")
       ),
       col = c("blue", "red", "darkgreen", "purple"),
       lty = 1,
       lwd = 2,
       bg = "white",
       cex = 0.85)
```



Exercițiul 5

5.a) Derivarea în raport cu parametrii θ și Demonstrarea Concavității

Pentru a oferi o demonstrație riguroasă a concavității, vom considera **forma canonică** a familiei exponențiale. Presupunem că θ este vectorul parametrilor naturali (canonici), astfel încât densitatea se scrie:

$$f_X(x;\theta) = h(x)\exp\big(\theta^\top T(x) - A(\theta)\big)$$

Log-verosimilitatea pentru un eșantion i.i.d. $x_1, \ldots, x_m$ este:

$$l(\theta; \underline{x}) = \sum_{i=1}^{m} \ln h(x_i) + \theta^\top \left(\sum_{i=1}^{m} T(x_i)\right) - mA(\theta)$$

Pentru a studia punctele de maxim, calculăm gradientul (vectorul scor) și matricea Hessiană în raport cu θ.

Pasul 1: Calculul Gradientului (Funcția Scor)

Derivăm $l(\theta)$ în raport cu vectorul θ:

$$\nabla_\theta l(\theta) = \sum_{i=1}^{m} T(x_i) - m\nabla_\theta A(\theta)$$

Pentru a evalua $\nabla A(\theta)$, folosim proprietatea că densitatea de probabilitate integrează la 1:

$$\int f(x;\theta)dx = 1 \iff \int h(x)e^{\theta^\top T(x) - A(\theta)}dx = 1$$

$$\iff e^{-A(\theta)} \int h(x)e^{\theta^\top T(x)}dx = 1 \iff A(\theta) = \ln\left(\int h(x)e^{\theta^\top T(x)}dx\right)$$

Derivând identitatea $\int f(x;\theta)dx = 1$ în raport cu θ (sub semnul integralei, presupunând condiții de regularitate):

$$\nabla_\theta \int h(x) e^{\theta^\top T(x) - A(\theta)} dx = 0$$

$$\int h(x) e^{\theta^\top T(x) - A(\theta)} \cdot (T(x) - \nabla A(\theta)) dx = 0$$

$$\int (T(x) - \nabla A(\theta)) f(x;\theta) dx = 0$$

$$\mathbb{E}[T(X)] - \nabla A(\theta) = 0 \implies \boxed{\nabla A(\theta) = \mathbb{E}[T(X)]}$$

Astfel, gradientul log-verosimilității devine:

$$\nabla_\theta l(\theta) = \sum_{i=1}^{m} T(x_i) - m\mathbb{E}[T(X)]$$

Egalând cu 0 pentru a găsi MLE, obținem intuiția Metoadei Momentelor: media empirică a statisticii suficiente trebuie să fie egală cu media teoretică.

Pasul 2: Calculul Matricei Hessiene

Derivăm din nou gradientul pentru a obține Hessianul $H(\theta)$:

$$H(\theta) = \nabla_\theta^2 l(\theta) = \frac{\partial}{\partial \theta^\top} \left(\sum_{i=1}^{m} T(x_i) - m\nabla A(\theta) \right)$$

Deoarece primul termen nu depinde de θ:

$$H(\theta) = -m\nabla^2 A(\theta)$$

Trebuie să calculăm $\nabla^2 A(\theta)$. Derivăm din nou relația $\mathbb{E}[T(X)] = \nabla A(\theta)$, sau mai exact, identitatea integrală anterioară:

$$\nabla_\theta \int (T(x) - \nabla A(\theta)) f(x;\theta) dx = 0$$

Aplicăm regula produsului:

$$\int \left[-\nabla^2 A(\theta) f(x;\theta) + (T(x) - \nabla A(\theta)) \cdot \underbrace{\nabla f(x;\theta)}_{(T(x) - \nabla A(\theta)) f(x;\theta)} \right] dx = 0$$

$$-\nabla^2 A(\theta) \underbrace{\int f(x;\theta) dx}_{1} + \int (T(x) - \nabla A(\theta))(T(x) - \nabla A(\theta))^\top f(x;\theta) dx = 0$$

Termenul din dreapta este definiția matricei de covarianță a vectorului aleator $T(X)$, deoarece $\mathbb{E}[T(X)] = \nabla A(\theta)$.

$$\nabla^2 A(\theta) = \mathrm{VarCov}[T(X)]$$

Pasul 3: Concluzia (Negativ Definirea)

Am demonstrat că Hessianul log-verosimilității este:

$$H(\theta) = -m \cdot \mathrm{VarCov}[T(X)]$$

Din algebră liniară, știm că **orice matrice de covarianță este pozitiv semi-definită** (adică $z^\top \Sigma z \geq 0, \forall z \neq 0$).

Prin urmare, matricea $-m \cdot \mathrm{VarCov}[T(X)]$ este **negativ semi-definită**.

$$H(\theta) \preceq 0$$

Consecință: Funcția de log-verosimilitate $l(\theta)$ este o funcție **concavă** pe tot domeniul parametrului natural. Aceasta garantează că: 1. Dacă există un punct critic (unde gradientul este 0), acesta este un **maxim global**. 2. Algoritmii de optimizare numerică (ex. Newton-Raphson) vor converge către soluția corectă (MLE unic, dacă Hessianul este strict negativ definit).

5.b) Formele log-verosimilitate pentru distribuții specifice

1) Binomială
$X \sim \mathrm{Bin}(m, p), \quad m \in \mathbb{N},\ p \in (0,1)$

$$l(\theta; \underline{x}) = \sum_{i=1}^{m} \left[\ln C_m^{x_i} + \ln \frac{p}{1-p} \cdot x_i + m \cdot \ln(1-p) \right]$$

2) Geometrică
$X \sim \mathrm{Geom}(p), \quad p \in (0,1)$

$$l(\theta; \underline{x}) = \sum_{i=1}^{m} [\ln(1-p) \cdot (x_i - 1) + \ln p]$$

3) Poisson
$X \sim \mathrm{Poisson}(\lambda), \quad \lambda \in (0, \infty)$

$$l(\theta; \underline{x}) = \sum_{i=1}^{m} \left[\ln \frac{1}{x_i!} + \ln \lambda \cdot x_i - \lambda \right]$$

4) Exponențială
$X \sim \mathrm{Exp}(\lambda), \quad \lambda \in (0, +\infty)$

$$l(\theta; \underline{x}) = \sum_{i=1}^{m} (-\lambda \cdot x_i + \ln \lambda)$$

5) Gamma
$X \sim \mathrm{Gamma}(\alpha, \beta), \quad \alpha, \beta \in (0, +\infty)$

$$l(\theta; \underline{x}) = \sum_{i=1}^{m} [(\alpha - 1) \ln x_i - \beta x_i + \alpha \ln \beta - \ln \Gamma(\alpha)]$$

6) Beta
$X \sim \mathrm{Beta}(\alpha, \beta), \quad \alpha, \beta \in (0, +\infty)$

$$l(\theta; \underline{x}) = \sum_{i=1}^{m} [(\alpha - 1) \ln x_i + (\beta - 1) \ln(1 - x_i) - \ln B(\alpha, \beta)]$$

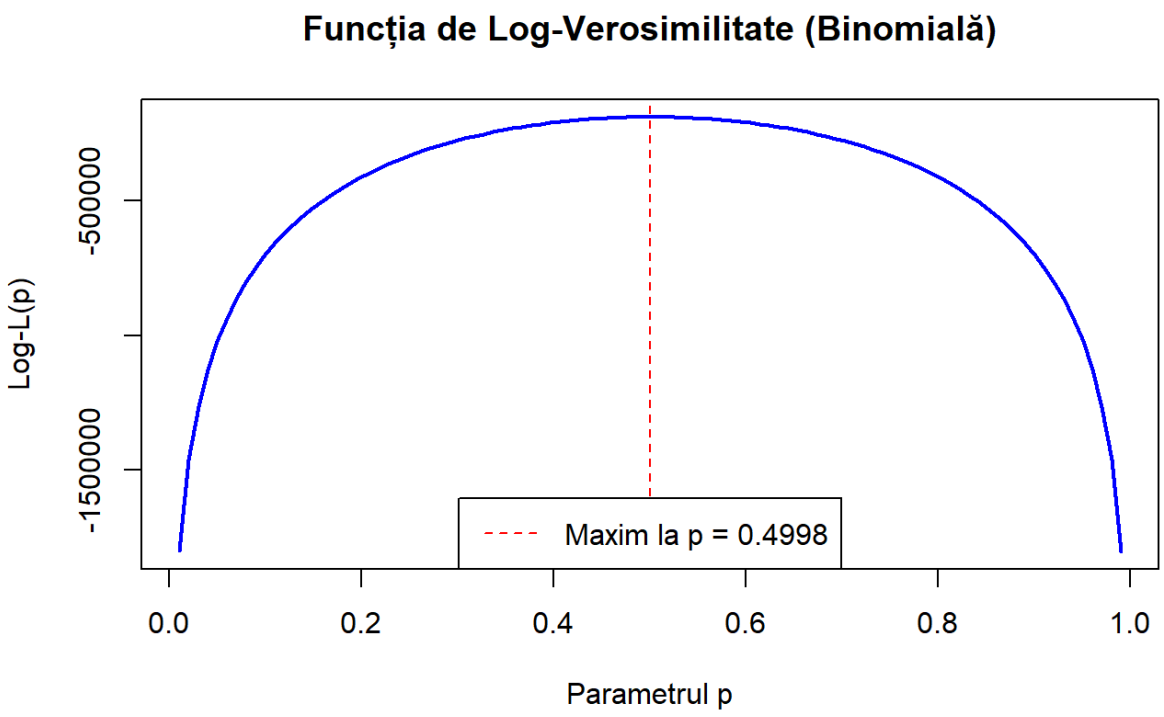
5.c) Reprezentări grafice ale log-verosimilitate + punctul de maxim (MLE)

5.c.1 Binomială

```r
set.seed(123)

#binomiala
N <- 10^5
n <- 10
p <- 0.5
X_n <- rbinom(N, n, p)

log_verosim <- function(n, x, p){
  termen_combinari <- lchoose(n, x) # ln(combinari)
  termen_2 <- x * log(p/(1-p))
  termen_3 <- n * log(1-p)
  suma <- sum(termen_combinari + termen_2 + termen_3)
  return(suma)
}

p_valori <- seq(0.01, 0.99, length.out = 100)
log_valori <- sapply(p_valori, log_verosim, n = n, x = X_n)

plot(p_valori, log_valori, type = "l", col = "blue", lwd = 2,
     main = "Funcția de Log-Verosimilitate (Binomială)",
     xlab = "Parametrul p", ylab = "Log-L(p)")

rezultat_optimizare <- optimize(f = log_verosim,
                                interval = c(0, 1),
                                n = n,
                                x = X_n,
                                maximum = TRUE)

cat("Punctul de maxim (MLE) este p =", rezultat_optimizare$maximum, "\n")
```

```
## Punctul de maxim (MLE) este p = 0.4997677
```

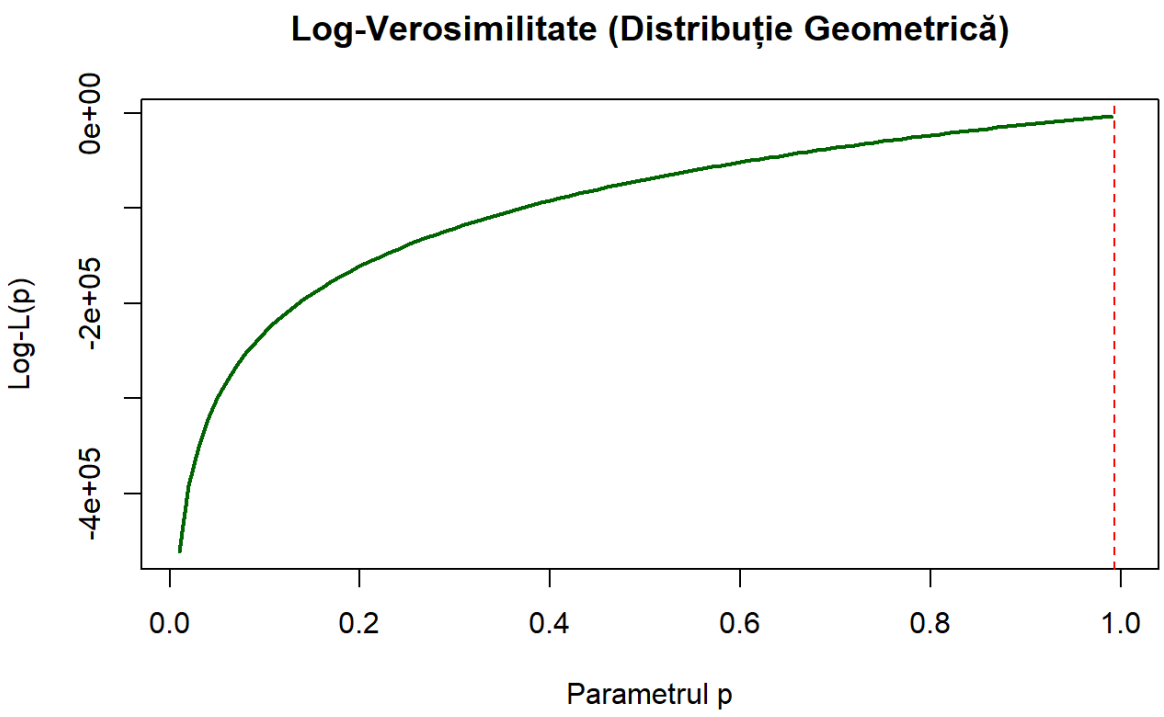
```r
abline(v = rezultat_optimizare$maximum, col = "red", lty = 2)
legend("bottom", legend = paste("Maxim la p =", round(rezultat_optimizare$maximum, 4)),
       col = "red", lty = 2)
```




```
set.seed(123)

N <- 10^5
p <- 0.5
X_n <- rgeom(N, p)

log_verosim <- function(x, p){
  termen1 <- log(1-p) * (x-1)
  termen2 <- log(p)
  suma <- sum(termen1 + termen2)
  return(suma)
}

p_valori <- seq(0.01, 1, length.out = 100)
log_valori <- sapply(p_valori, log_verosim, x = X_n)

plot(p_valori, log_valori, type = "l", col = "darkgreen", lwd = 2,
     main = "Log-Verosimilitate (Distribuție Geometrică)",
     xlab = "Parametrul p", ylab = "Log-L(p)")

rezultat_optimizare <- optimize(f = log_verosim,
                                interval = c(0, 1),
                                x = X_n,
                                maximum = TRUE)

cat("Punctul de maxim (MLE) calculat este p =", rezultat_optimizare$maximum, "\n")
```

```
## Punctul de maxim (MLE) calculat este p = 0.9932642
```

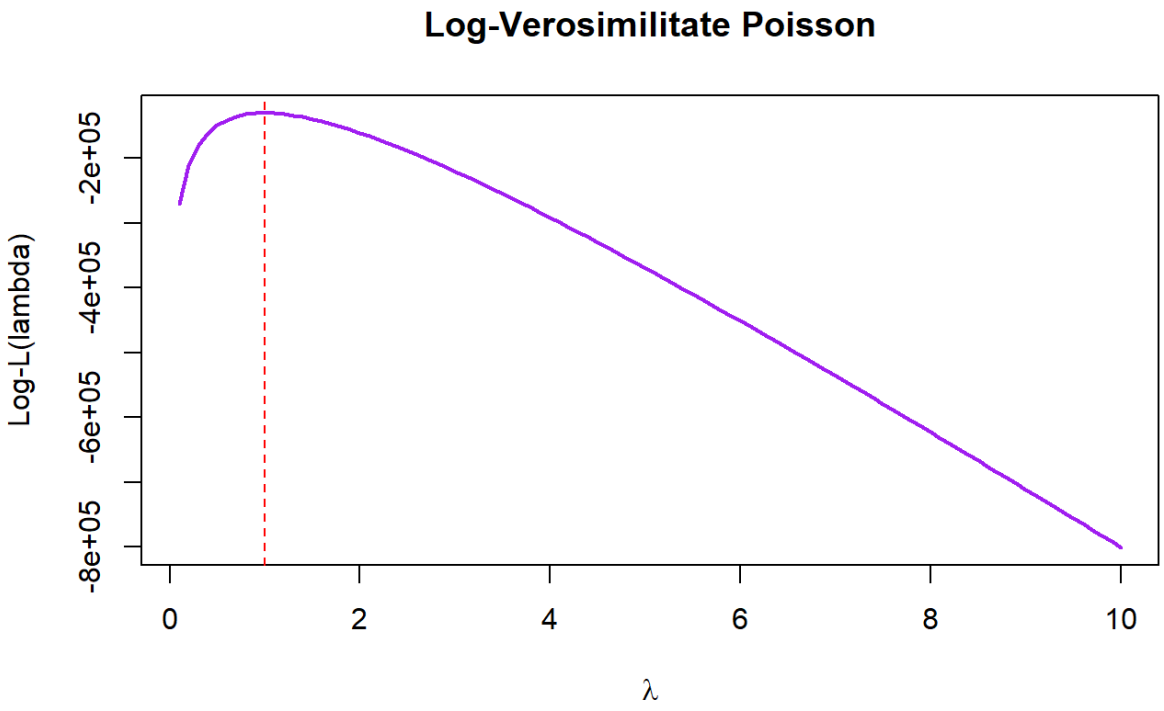
```
abline(v = rezultat_optimizare$maximum, col = "red", lty = 2)
```




```
set.seed(123)

N <- 10^5
lambda <- 1
X_n <- rpois(N, lambda)

log_verosim <- function(x, lambda){
  termen1 <- -lfactorial(x)
  termen2 <- log(lambda) * x
  return(sum(termen1 + termen2 - lambda))
}

lambda_valori <- seq(0.1, 10, length.out = 100)
log_valori <- sapply(lambda_valori, log_verosim, x = X_n)

plot(lambda_valori, log_valori, type = "l", col = "purple", lwd = 2,
     main = "Log-Verosimilitate Poisson",
     xlab = expression(lambda), ylab = "Log-L(lambda)")

rezultat_optimizare <- optimize(f = log_verosim,
                                interval = c(0.01, 5),
                                x = X_n,
                                maximum = TRUE)

cat("Punctul de maxim (MLE) calculat este lambda =", rezultat_optimizare$maximum, "\n")
```

```
## Punctul de maxim (MLE) calculat este lambda = 0.998297
```

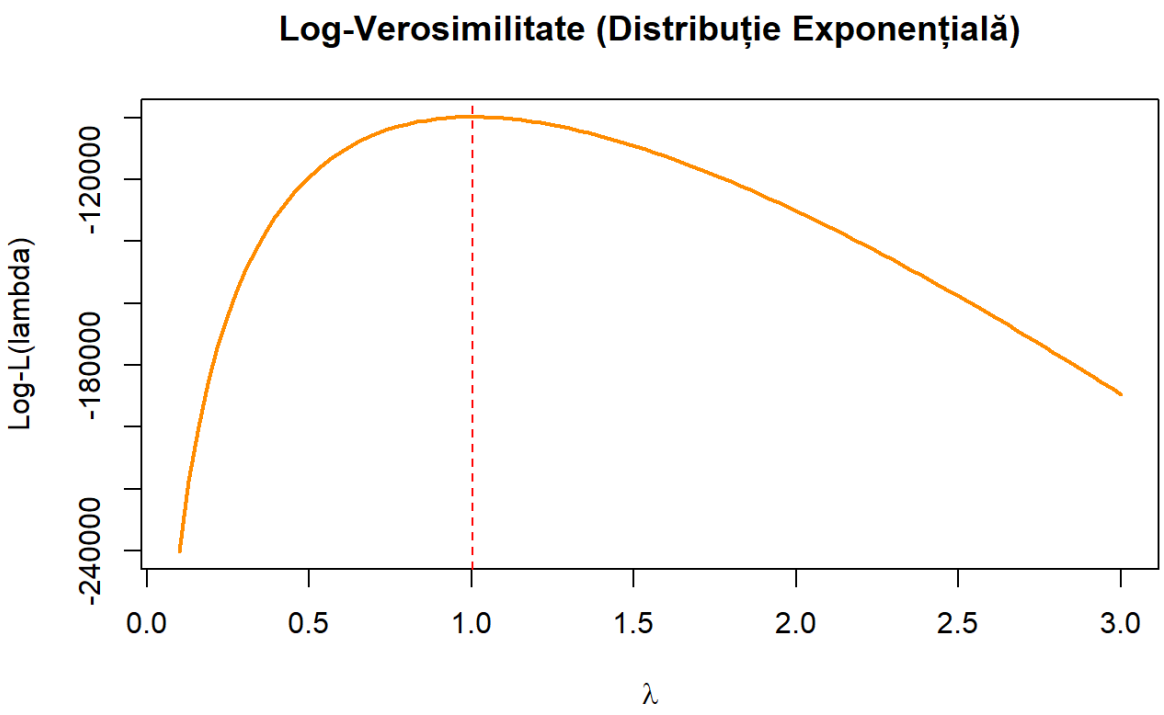
```
abline(v = rezultat_optimizare$maximum, col = "red", lty = 2)
```



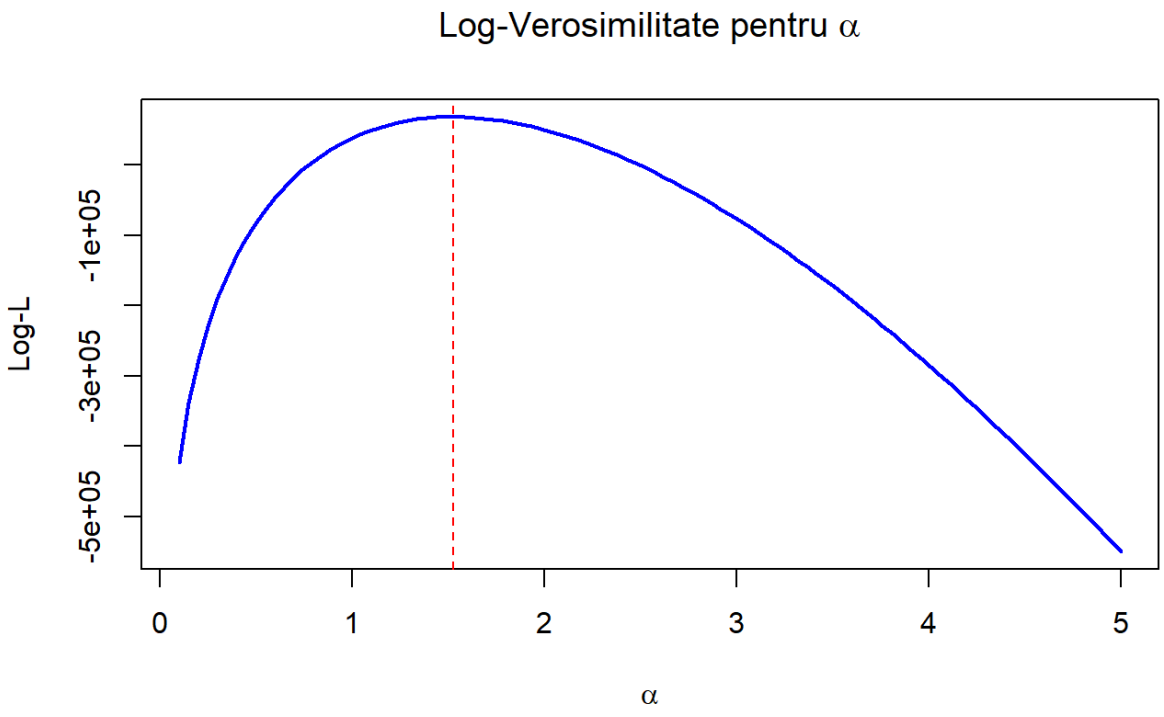

```
set.seed(123)

N <- 10^5
lambda <- 1
X_n <- rexp(N, rate = lambda)

log_verosim <- function(lambda, x){
  termen1 <- -lambda * x
  termen2 <- log(lambda)
  return(sum(termen1 + termen2))
}

lambda_valori <- seq(0.1, 3, length.out = 100)
log_valori <- sapply(lambda_valori, log_verosim, x = X_n)

plot(lambda_valori, log_valori, type = "l", col = "darkorange", lwd = 2,
     main = "Log-Verosimilitate (Distribuție Exponențială)",
     xlab = expression(lambda), ylab = "Log-L(lambda)")

rezultat_optimizare <- optimize(f = log_verosim,
                                interval = c(0.01, 10),
                                x = X_n,
                                maximum = TRUE)

cat("Punctul de maxim (MLE) este lambda =", rezultat_optimizare$maximum, "\n")
```

```
## Punctul de maxim (MLE) este lambda = 1.002556
```

```
abline(v = rezultat_optimizare$maximum, col = "red", lty = 2)
```



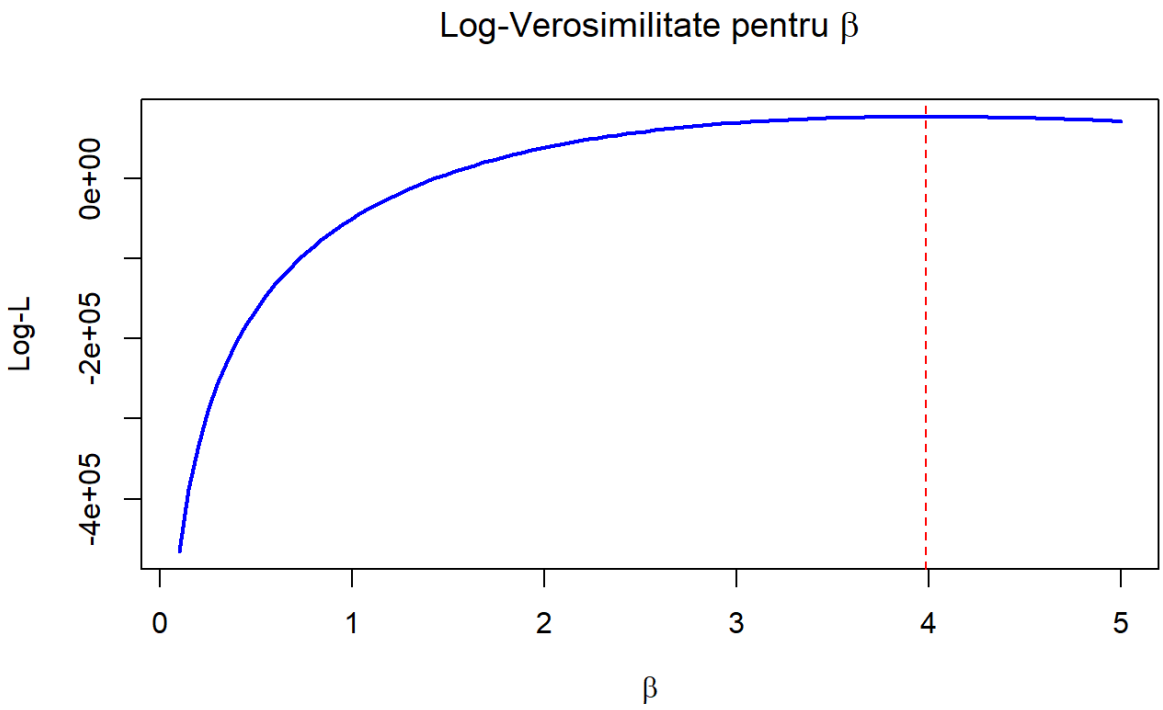

```
set.seed(123)

N <- 10^5
alfa_real <- 1
beta_real <- 2
X_n <- rgamma(N, shape = alfa_real, rate = beta_real)

log_verosim <- function(alfa, beta, x) {
  if (alfa <= 0) return(-Inf)

  termen1 <- (alfa - 1) * log(x)
  termen2 <- -beta * x
  termen3 <- 2 * alfa * log(beta)
  termen4 <- -2 * lgamma(alfa)

  return(sum(termen1 + termen2 + termen3 + termen4))
}

alfa_valori <- seq(0.1, 5, length.out = 100)
log_valori <- sapply(alfa_valori, log_verosim, beta = beta_real, x = X_n)

plot(alfa_valori, log_valori, type = "l", col = "blue", lwd = 2,
     main = expression(paste("Log-Verosimilitate pentru ", alpha)),
     xlab = expression(alpha), ylab = "Log-L")

rezultat_optimizare <- optimize(f = log_verosim,
                                interval = c(0.1, 10),
                                beta = beta_real,
                                x = X_n,
                                maximum = TRUE)

abline(v = rezultat_optimizare$maximum, col = "red", lty = 2)
```




```
cat("Punctul de maxim calculat pentru alfa este:", rezultat_optimizare$maximum, "\n")
```

```
## Punctul de maxim calculat pentru alfa este: 1.524488
```

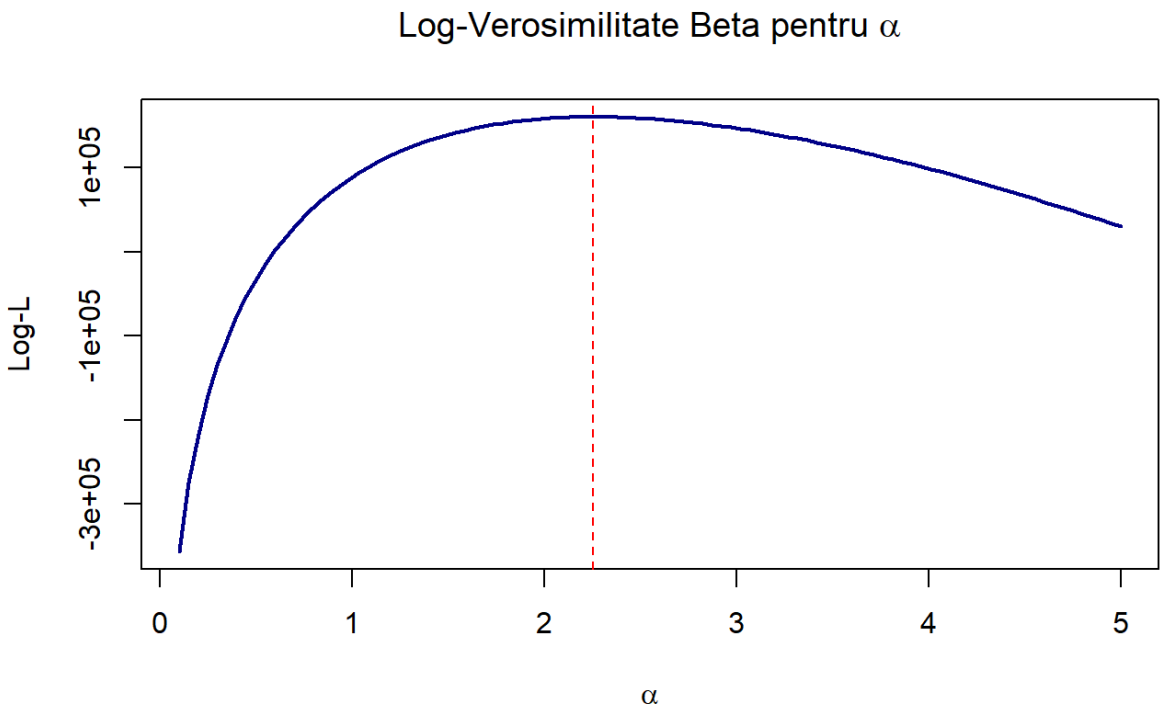
```
set.seed(123)

N <- 10^5
alfa_real <- 1
beta_real <- 2
X_n <- rgamma(N, shape = alfa_real, rate = beta_real)

log_verosim <- function(alfa, beta, x) {
  if (alfa <= 0) return(-Inf)

  termen1 <- (alfa - 1) * log(x)
  termen2 <- -beta * x
  termen3 <- 2 * alfa * log(beta)
  termen4 <- -2 * lgamma(alfa)

  return(sum(termen1 + termen2 + termen3 + termen4))
}

beta_valori <- seq(0.1, 5, length.out = 100)
log_valori <- sapply(beta_valori, function(b) log_verosim(alfa_real, b, X_n))

plot(beta_valori, log_valori, type = "l", col = "blue", lwd = 2,
     main = expression(paste("Log-Verosimilitate pentru ", beta)),
     xlab = expression(beta), ylab = "Log-L")

rezultat_optimizare <- optimize(f = function(b) log_verosim(alfa_real, b, X_n),
                                interval = c(0.1, 10),
                                maximum = TRUE)

abline(v = rezultat_optimizare$maximum, col = "red", lty = 2)
```




```
cat("Punctul de maxim calculat pentru beta este:", rezultat_optimizare$maximum, "\n")
```

```
## Punctul de maxim calculat pentru beta este: 3.988111
```

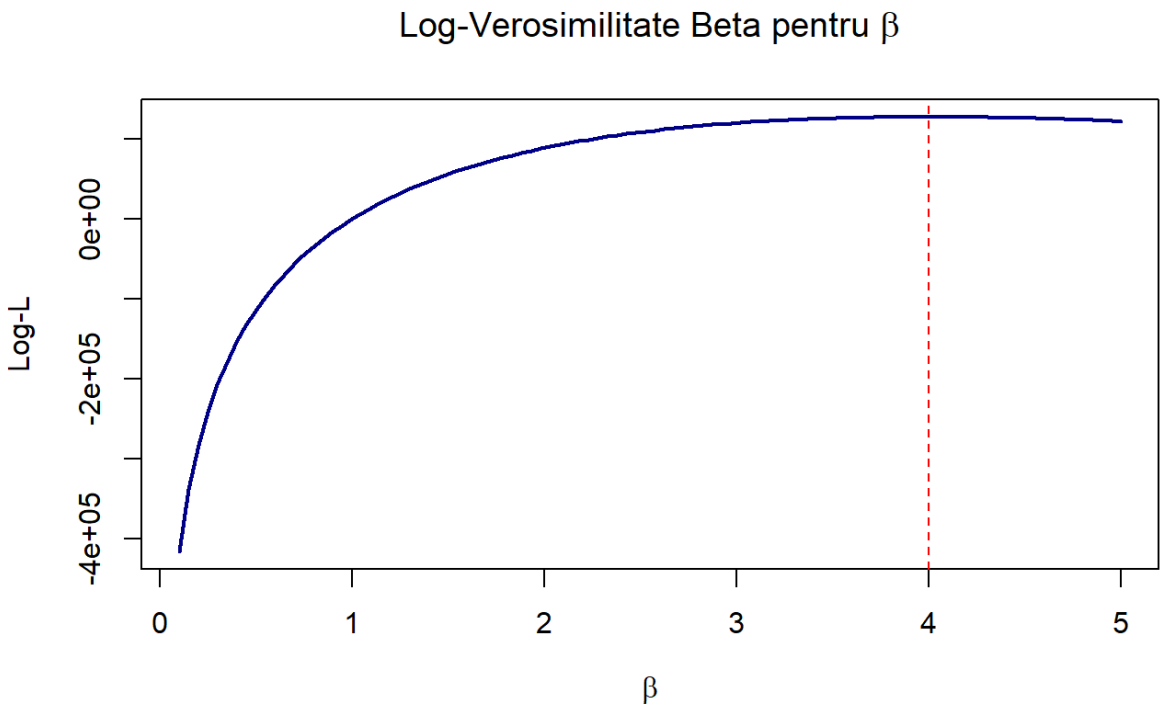
```
set.seed(123)

N <- 10^5
alfa_real <- 1
beta_real <- 2
X_n <- rbeta(N, shape1 = alfa_real, shape2 = beta_real)

log_verosim <- function(alfa_real, beta_real, x){
  termen1 <- (alfa_real - 1) * log(x)
  termen2 <- (beta_real - 1) * log(1 - x)
  termen3 <- -2 * log(beta(alfa_real, beta_real))
  return(sum(termen1 + termen2 + termen3))
}

alfa_valori <- seq(0.1, 5, length.out = 100)
log_valori <- sapply(alfa_valori, log_verosim, beta_real = beta_real, x = X_n)

plot(alfa_valori, log_valori, type = "l", col = "darkblue", lwd = 2,
     main = expression(paste("Log-Verosimilitate Beta pentru ", alpha)),
     xlab = expression(alpha), ylab = "Log-L")

rezultat_optimizare <- optimize(f = function(a) log_verosim(a, beta_real, X_n),
                                interval = c(0.1, 10),
                                maximum = TRUE)

abline(v = rezultat_optimizare$maximum, col = "red", lty = 2)
```




```
cat("Punctul de maxim calculat pentru alfa este:", rezultat_optimizare$maximum, "\n")
```

```
## Punctul de maxim calculat pentru alfa este: 2.255245
```

```
set.seed(123)

N <- 10^5
alfa_real <- 1
beta_real <- 2
X_n <- rbeta(N, shape1 = alfa_real, shape2 = beta_real)

log_verosim <- function(alfa_real, beta_real, x){
  termen1 <- (alfa_real - 1) * log(x)
  termen2 <- (beta_real - 1) * log(1 - x)
  termen3 <- -2 * log(beta(alfa_real, beta_real))
  return(sum(termen1 + termen2 + termen3))
}

beta_valori <- seq(0.1, 5, length.out = 100)
log_valori <- sapply(beta_valori, function(b) log_verosim(alfa_real, b, X_n))

plot(beta_valori, log_valori, type = "l", col = "darkblue", lwd = 2,
     main = expression(paste("Log-Verosimilitate Beta pentru ", beta)),
     xlab = expression(beta), ylab = "Log-L")

rezultat_optimizare <- optimize(f = function(b) log_verosim(alfa_real, b, X_n),
                                interval = c(0.1, 10),
                                maximum = TRUE)

abline(v = rezultat_optimizare$maximum, col = "red", lty = 2)
```




```
cat("Punctul de maxim calculat pentru beta este:", rezultat_optimizare$maximum, "\n")
```

```
## Punctul de maxim calculat pentru beta este: 4.001826
```

Exercițiul 6

6.a)

a. $X \sim \text{Bin}(n, p), \quad p \in (0,1), \quad m \in \mathbb{N}.$

$$P(X = k) = C_n^k p^k (1-p)^{n-k}, \quad k \in \mathbb{N}$$

$$E[X] = n \cdot p, \quad \text{Var}(X) = np(1-p)$$

Fie $X_1, \ldots, X_m$ i.i.d $\sim \text{Bin}(n, p)$

$$E[X] = \overline{X}_m = \frac{1}{m}\sum_{i=1}^{m} X_i \iff n \cdot p = \overline{X}_m \iff \underbrace{\hat{p}_m = \frac{1}{n}\overline{X}_m}_{\text{estimatorul obținut prin MM}}, \quad n \in \mathbb{N}^*.$$

$$\ell(p; x_1, \ldots, x_m) = \sum_{i=1}^{m} \ln C_n^{x_i} + \ln \frac{p}{1-p} \cdot x_i + n \ln(1-p) \quad \Big| \frac{\partial}{\partial p} \implies$$

Calculul derivatei:

$$\implies \frac{\partial \ell}{\partial p}(p; \underline{x}) = \sum_{i=1}^{m} \frac{1-p}{p} \cdot \left(\frac{p}{1-p}\right)'_p \cdot x_i - \frac{n}{1-p}$$

$$= \sum_{i=1}^{m} \left(\frac{x_i}{p(1-p)} - \frac{n}{1-p}\right) = -\frac{m \cdot n}{1-p} + \frac{1}{p(1-p)} m\overline{x}_m$$

Vrem p a.î. $\frac{\partial l}{\partial p}(p, x_i) = 0$.

$$\Leftrightarrow -\frac{m\cdot n}{1-p}+\frac{1}{p(1-p)}m\overline{x}_m=0\Leftrightarrow \frac{1}{p(1-p)}m\overline{x}_m=\frac{m\cdot n}{1-p}$$

$$\Rightarrow \frac{1}{p}=\frac{n}{\overline{x}_m}\Rightarrow p_m=\frac{\overline{x}_m}{n}\quad \text{(estimare)}$$

Observație (verificare):

$$m\overline{x}_m=\frac{m\cdot n}{(1-p)}\cdot p(1-p)=m\cdot n\cdot p\quad \text{(Nu estimator)}$$

p_m punct de maxim pentru l

$$\frac{\partial^2 l}{\partial p^2}(p,x_i)=\frac{\partial}{\partial p}\left(\frac{\partial l}{\partial p}(p,x_i)\right)=\frac{\partial}{\partial p}\left(-\frac{m\cdot n}{1-p}+\frac{1}{p(1-p)}m\overline{x}_m\right)$$

$$=-\frac{m\cdot n}{(1-p)^2}-\frac{m\overline{x}_m}{p^2(1-p)^2}(1-2p)$$

Substituind $m\overline{x}_m=m\cdot n\cdot p$:

$$=\frac{-m\cdot n\cdot p^2-m\cdot n\cdot p\cdot(1-2p)}{p^2(1-p)^2}=\frac{-m\cdot n\cdot p(p^2+1)(1-2p)}{p^2(1-p)^2}$$

Analiza semnului pentru termenul $(1-2p)$:

$$\begin{aligned}0<p<1\mid\cdot 2&\Rightarrow 0<2p<2\\&\Rightarrow -2<-2p<0\\&\Rightarrow -1<1-2p<1\quad \text{(nu are semn constant)}\end{aligned}$$

Calcul derivată fracție (caseta de jos):

$$\left(\frac{1}{p(1-p)}\right)'=\left(\frac{1}{p}\cdot\frac{1}{1-p}\right)'=-\frac{1}{p^2}\cdot\frac{1}{1-p}-\frac{1}{p}\frac{1}{(1-p)^2}(-1)$$

Evaluare finală (scrierea cu roșu):

$$=\frac{1}{p(1-p)}\left(-\frac{1}{p}-\frac{1}{1-p}\right)\cdots=\frac{m\cdot n}{1-p}\left[-\frac{1}{1-p}+\frac{1}{p}+\frac{1}{1-p}\right]>0$$

---

b. $X_1,\dots,X_n\sim \text{Geom}(p), p\in(0,1)$, i.i.d

$$\text{MVM}\quad l(p,\underline{x})=\sum_{i=1}^{n}[\ln(1-p)\cdot(x_i-1)+\ln p]=n\ln p+\ln(1-p)\sum_{i=1}^{n}(x_i-1)$$

$$=n\ln p+\ln(1-p)\sum_{i=1}^{n}x_i-n\ln(1-p)=n\left[\ln p+\ln(1-p)\cdot\overline{x}_m-\ln(1-p)\right]$$

$$\frac{\partial l}{\partial p}(p,\underline{x})=n\left[\frac{1}{p}-\frac{\overline{x}_m}{1-p}+\frac{1}{1-p}\right]\overset{\text{VREAU}}{=}0\Rightarrow\frac{1}{p}=-\frac{1-\overline{x}_m}{1-p}$$

$$\Leftrightarrow 1-p=-p(1-\overline{x}_m)\Leftrightarrow\frac{1-p}{p}=\overline{x}_m-1\Rightarrow\frac{1}{p}=\overline{x}_m\Rightarrow p=\frac{1}{\overline{x}_m}$$

$$\frac{\partial^2 l}{\partial p^2}=\frac{\partial}{\partial p}\left(n\left[\frac{1}{p}-\frac{\overline{x}_m}{1-p}+\frac{1}{1-p}\right]\right)=n\left[-\frac{1}{p^2}-\frac{\overline{x}_m}{(1-p)^2}+\frac{1}{(1-p)^2}\right]$$

Substituind $\overline{x}_m=\frac{1}{p}$ (din punctul critic):

$$= n\left[-\frac{1}{p^2} - \frac{1/p}{(1-p)^2} + \frac{1}{(1-p)^2}\right] = n\frac{-(1-p)^2 - p + p^2}{p^2(1-p)^2}$$

$$= \frac{n}{p^2(1-p)^2}(p^2 - p - 1 + 2p - p^2) = -\frac{n(1-p)}{p^2(1-p)^2} = -\frac{n}{p^2(1-p)} < 0$$

$$\Rightarrow \hat{p} = \frac{1}{\overline{x}_m}$$

$$\text{MM} \quad E[X] = \overline{X}_m \Leftrightarrow \frac{1}{p} = \overline{X}_m \Rightarrow \hat{p} = \frac{1}{\overline{x}_m}$$

---

c. $X_1, \ldots, X_n \sim \text{Poisson}(\lambda), \lambda > 0$ i.i.d

$$\text{MVM} \quad l(\lambda, \underline{x}) = \sum_{i=1}^{n}\left[\ln\frac{1}{x_i!} + \ln\lambda \cdot x_i - \lambda\right]$$

$$\frac{\partial l}{\partial \lambda}(\lambda, \underline{x}) = \sum_{i=1}^{n}\left(\frac{x_i}{\lambda} - 1\right) = -n + \frac{1}{\lambda}\cdot\sum_{i=1}^{n} x_i = \frac{n}{\lambda}\overline{x}_n - n = n\left(\frac{\overline{x}_n}{\lambda} - 1\right)$$

Vreau $\frac{\partial l}{\partial \lambda} = 0 \Rightarrow \frac{\overline{x}_n}{\lambda} - 1 = 0 \Leftrightarrow \lambda = \overline{x}_n$.

$$\frac{\partial^2 l}{\partial \lambda^2}(\lambda, \underline{x}) = \frac{\partial}{\partial \lambda}\left(\sum_{i=1}^{n}\left(\frac{x_i}{\lambda} - 1\right)\right) = \sum_{i=1}^{n}\left(-\frac{x_i}{\lambda^2}\right) = -\frac{1}{\lambda^2}\sum_{i=1}^{n} x_i = -\frac{n}{\lambda^2}\overline{x}_n$$

$$= -\frac{n}{\lambda} < 0$$

$$\Rightarrow \hat{\lambda}_n = \overline{x}_n$$

$$\text{MM} \quad E[X] = \overline{X}_n \Leftrightarrow \hat{\lambda} = \overline{X}_n$$

---

d. $X_1, \ldots, X_n \sim \text{Exp}(\lambda), \lambda > 0$, i.i.d

$$l(\lambda, \underline{x}) = \sum_{i=1}^{n}(-\lambda x_i + \ln\lambda)$$

$$\frac{\partial l}{\partial \lambda}(\lambda, \underline{x}) = \sum_{i=1}^{n}\left(-x_i + \frac{1}{\lambda}\right) = \frac{n}{\lambda} - n\overline{x}_n = n\left(\frac{1}{\lambda} - \overline{x}_n\right)$$

Vrem $\frac{\partial l}{\partial \lambda} = 0 \Rightarrow \lambda = \frac{1}{\overline{x}_n}$.

$$\frac{\partial^2 l}{\partial \lambda^2} = \frac{\partial}{\partial \lambda}\left(n\left(\frac{1}{\lambda} - \overline{x}_n\right)\right) = -\frac{n}{\lambda^2} < 0 \Rightarrow \hat{\lambda}_n = \frac{1}{\overline{X}_n}$$

$$\text{MM: trivial (vezi A.)}$$

---

e. $X_1, \ldots, X_n \sim \text{Gamma}(\alpha, \beta); \alpha, \beta > 0$ i.i.d.

$$\text{MM}: \begin{cases} E[X] = \overline{X}_n \iff \frac{\alpha}{\beta} = \overline{X}_n \\ E[X^2] = \frac{1}{n}\sum_{i=1}^{n} x_i^2 = \overline{x_n^2} \end{cases}$$

$$\frac{\alpha}{\beta^2} = \mathrm{Var}(X) = E[X^2] - (E[X])^2 = E[X^2] - \frac{\alpha^2}{\beta^2} \Rightarrow E[X^2] = \frac{\alpha(\alpha+1)}{\beta^2}$$

$$\text{Deci } \begin{cases} \frac{\alpha}{\beta} = \overline{X}_n \Rightarrow \alpha = \beta\overline{X}_n \\ \frac{\alpha(\alpha+1)}{\beta^2} = \overline{x_n^2} \end{cases} \implies \frac{\beta\overline{X}_n(\beta\overline{X}_n+1)}{\beta^2} = \overline{x_n^2} \iff \beta(\overline{X}_n)^2 + \overline{X}_n = \beta\overline{x_n^2}$$

$$\Rightarrow \hat{\beta} = \frac{-\overline{X}_n}{(\overline{X}_n)^2 - \overline{x_n^2}} \Rightarrow \hat{\alpha} = \frac{-(\overline{X}_n)^2}{(\overline{X}_n)^2 - \overline{x_n^2}}$$

$$\text{Deci } \hat{\gamma}_n = (\hat{\alpha}, \hat{\beta}) \text{ este estimatorul 2-d}$$

$$\text{MVM} \quad l((\alpha,\beta);\underline{x}) = \sum_{i=1}^{n} [(\alpha-1)\ln x_i - \beta x_i + \alpha\ln\beta - \ln\Gamma(\alpha)]$$

$$\nabla l = \left(\frac{\partial l}{\partial\alpha}, \frac{\partial l}{\partial\beta}\right), \ \Delta l = 0, \ H_l(\alpha_0,\beta_0) \text{ negativ (semi) definita}$$

$$\frac{\partial l}{\partial\alpha}((\alpha,\beta),\underline{x}) = \sum_{i=1}^{n}\left[\ln x_i + \ln\beta - \frac{\Gamma'(\alpha)}{\Gamma(\alpha)}\right]$$

$$\left(\text{obs: } \frac{\Gamma'(\alpha)}{\Gamma(\alpha)} = \psi(\alpha); \quad \Gamma(\alpha) = \int_0^\infty t^{\alpha-1}e^{-t}dt \Rightarrow \Gamma'(\alpha) = \int_0^\infty t^{\alpha-1}\ln t \cdot e^{-t}dt\right)$$

$$= \sum_{i=1}^{n}(\ln x_i) + n\ln\beta - n\psi(\alpha) \overset{\text{VREAU}}{=} 0.$$

$$\frac{\partial l}{\partial\beta}((\alpha,\beta),\underline{x}) = -\sum_{i=1}^{n} x_i + n\frac{\alpha}{\beta} \overset{\text{VREAU}}{=} 0 \Rightarrow \beta = \ldots \text{ succes } \Rightarrow \beta = \frac{\alpha}{\overline{x}_n}$$

---

f. $X_1, \ldots, X_n \sim \mathrm{Beta}(\alpha,\beta), \alpha,\beta > 0$ i.i.d.

$$l((\alpha,\beta),\underline{x}) = \sum_{i=1}^{n}[(\alpha-1)\ln x_i + (\beta-1)\ln(1-x_i) - \ln B(\alpha,\beta)]$$

$$\text{MM} \quad \begin{cases} E[X] = \overline{X}_n \iff \frac{\alpha}{\alpha+\beta} = \overline{X}_n \\ E[X^2] = \overline{X_n^2} \Leftrightarrow \frac{\alpha\beta+\alpha^2(\alpha+\beta+1)}{(\alpha+\beta)^2(\alpha+\beta+1)} = \overline{X_n^2} \end{cases}$$

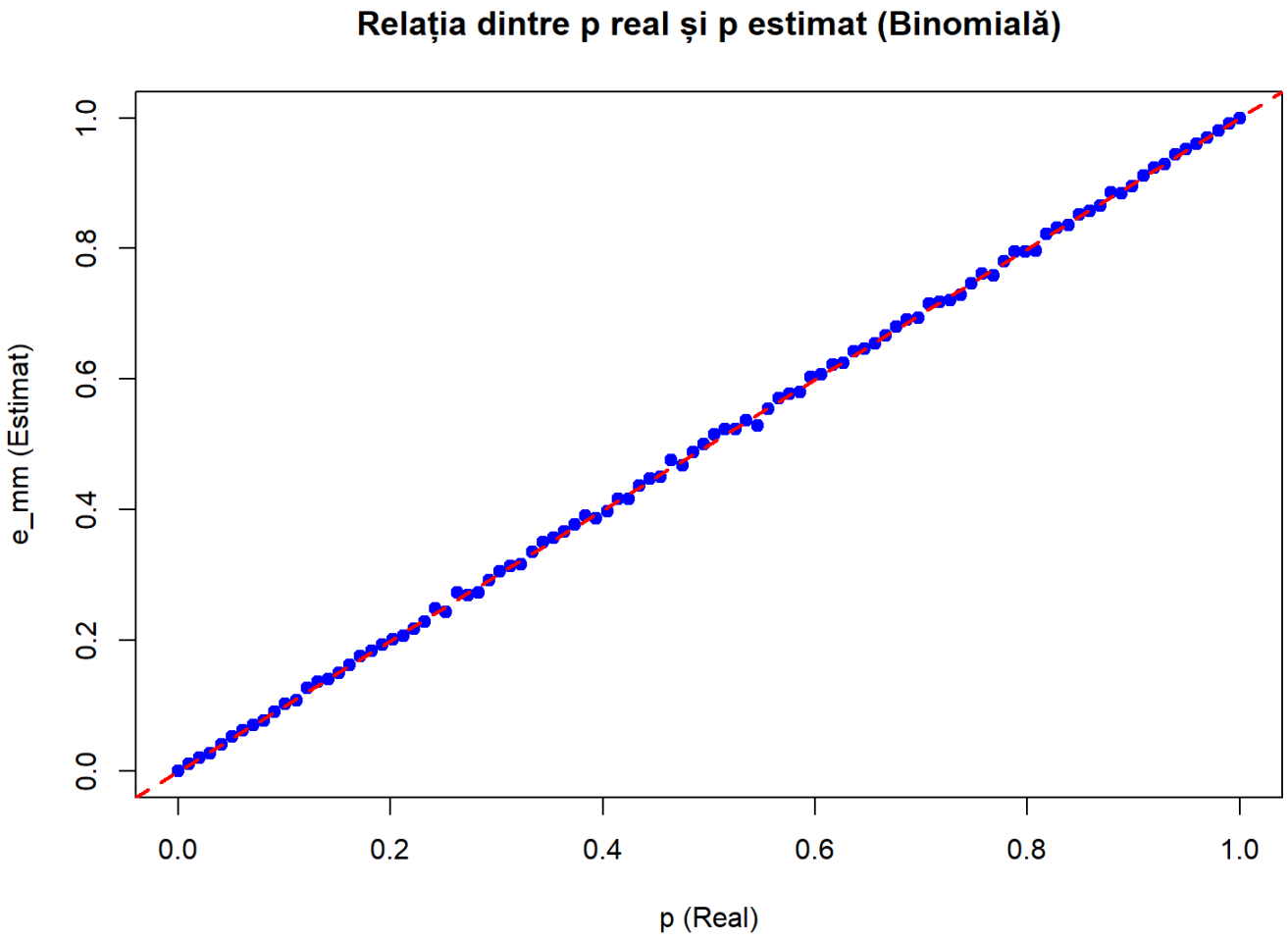
$$\frac{\alpha\beta}{(\alpha+\beta)^2(\alpha+\beta+1)} = \mathrm{Var}(X) = E[X^2] - (E[X])^2 \Rightarrow$$

$$\Rightarrow E[X^2] = \frac{\alpha\beta}{(\alpha+\beta)^2(\alpha+\beta+1)} + \frac{\alpha^2}{(\alpha+\beta)^2} = \frac{\alpha\beta+\alpha^2(\alpha+\beta+1)}{(\alpha+\beta)^2(\alpha+\beta+1)}$$

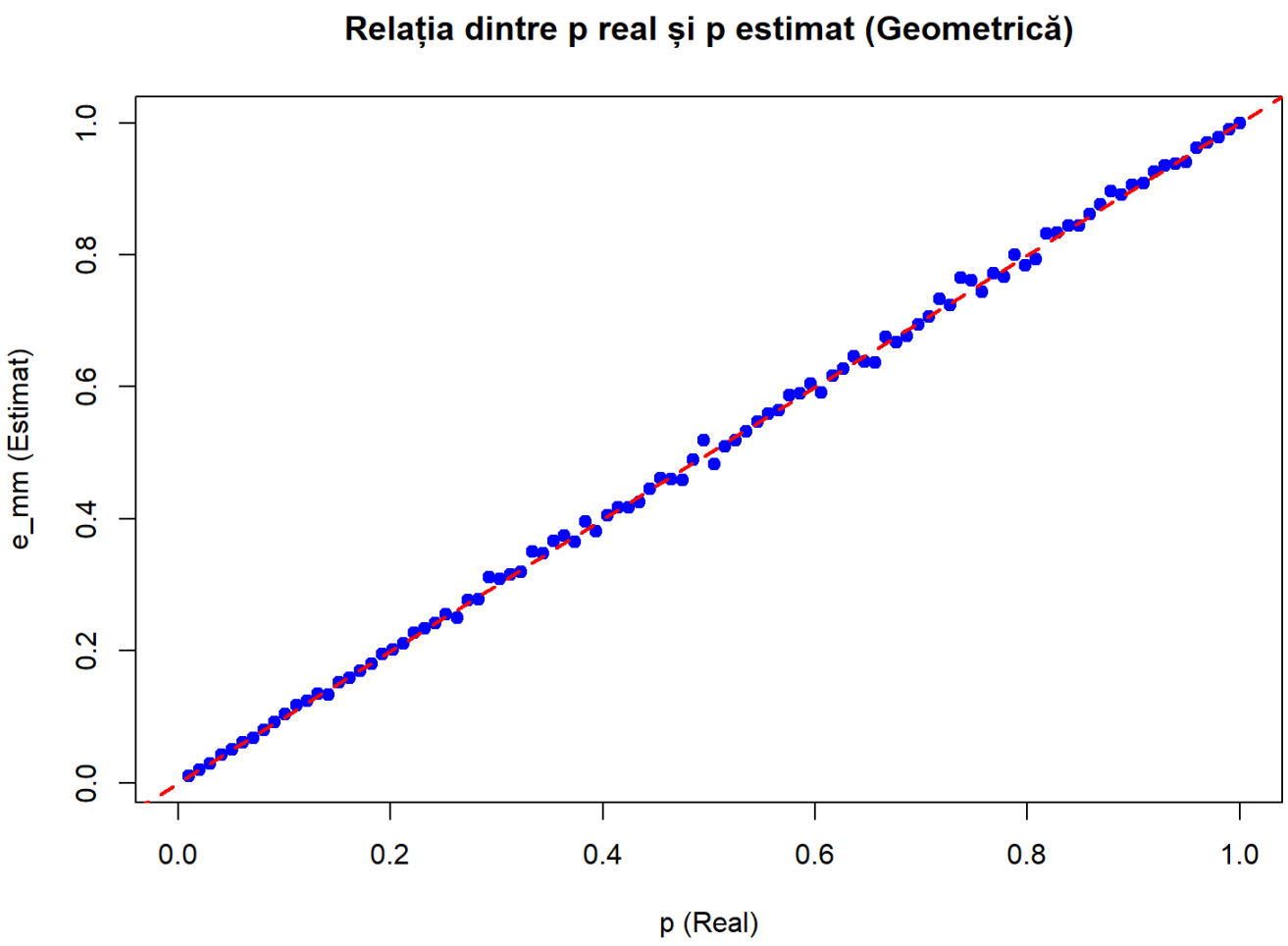
6.b)

În această secțiune verificăm numeric, prin simulări Monte Carlo, calitatea estimatorilor obținuți prin Metoda Momentelor (MM) și Metoda Verosimilității Maxime (MLE).

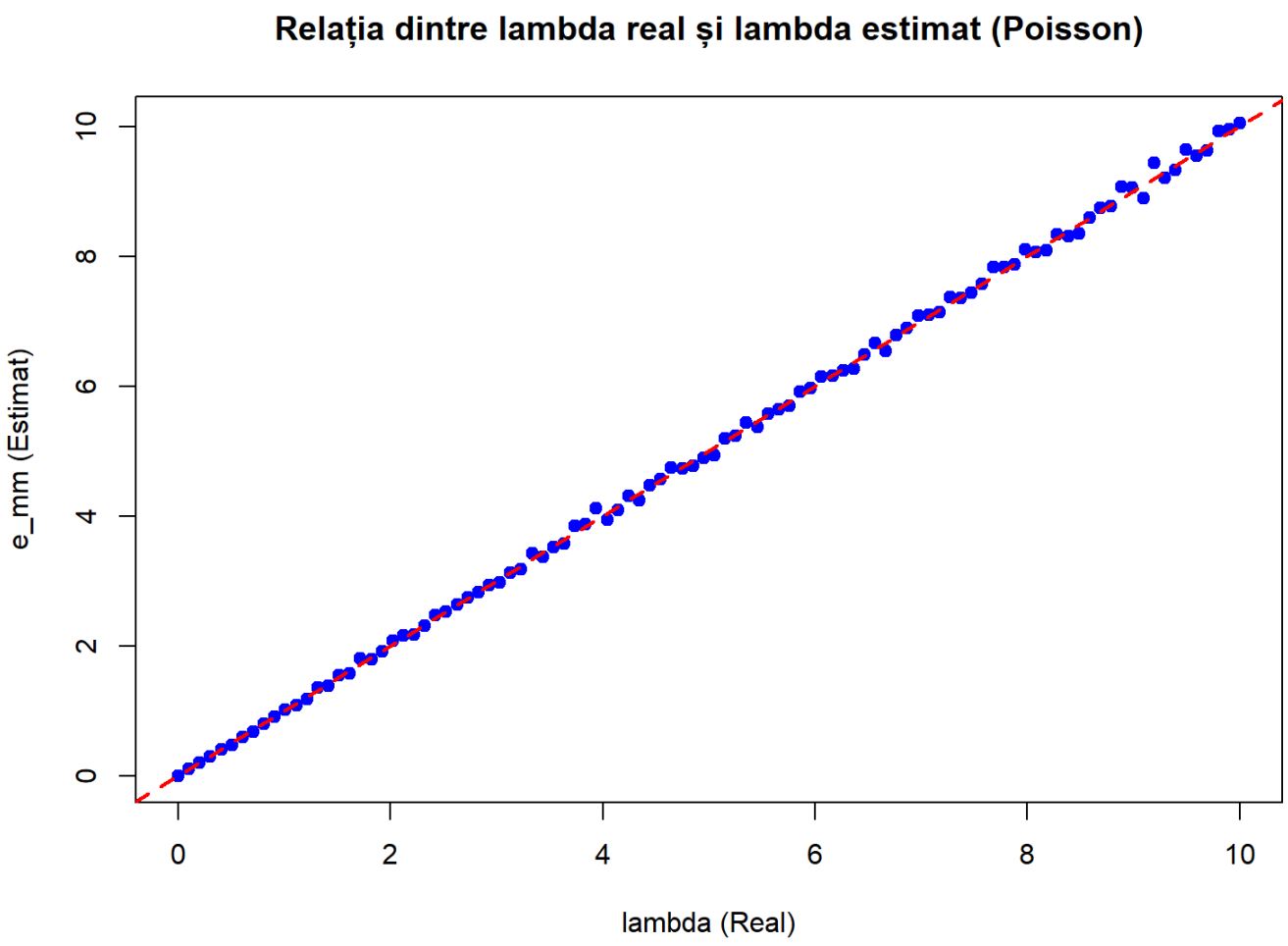
```r
# ==========================================================
# 1. Distribuția Binomială (MM)
# ==========================================================
N <- 1000
n <- 7
p_real <- seq(0, 1, length.out = 100)

# Calculul estimatorilor
e_mm_values <- sapply(p_real, function(p) {
  x <- rbinom(N, n, p)
  (1/n) * mean(x)
})

# Graficul
plot(p_real, e_mm_values,
     main = "Relația dintre p real și p estimat (Binomială)",
     xlab = "p (Real)",
     ylab = "e_mm (Estimat)",
     col = "blue",
     pch = 19)

# Adăugăm linia roșie de referință y = x
abline(a = 0, b = 1, col = "red", lwd = 2, lty = 2)
```



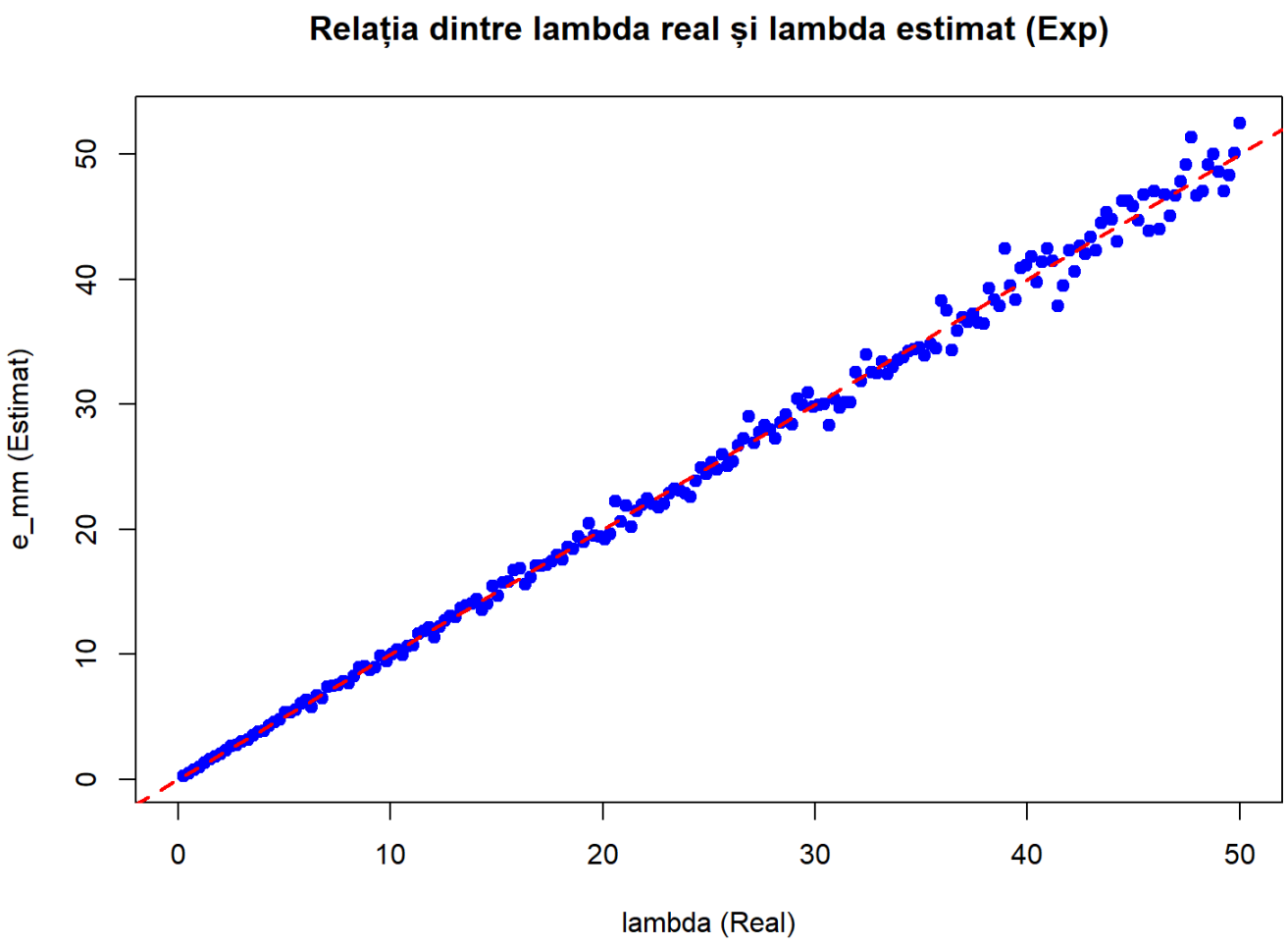
```r
# ==========================================================
# 2. Distribuția Geometrică (MM)
# ==========================================================
N <- 1000
p_real <- seq(0, 1, length.out = 100)

# Calculul estimatorilor
e_mm_values <- sapply(p_real, function(p) {
  x <- rgeom(N,p)
  1/(mean(x)+1)
})

# Graficul
plot(p_real, e_mm_values,
     main = "Relația dintre p real și p estimat (Geometrică)",
     xlab = "p (Real)",
     ylab = "e_mm (Estimat)",
     col = "blue",
     pch = 19)

# Adăugăm linia roșie de referință y = x
abline(a = 0, b = 1, col = "red", lwd = 2, lty = 2)
```



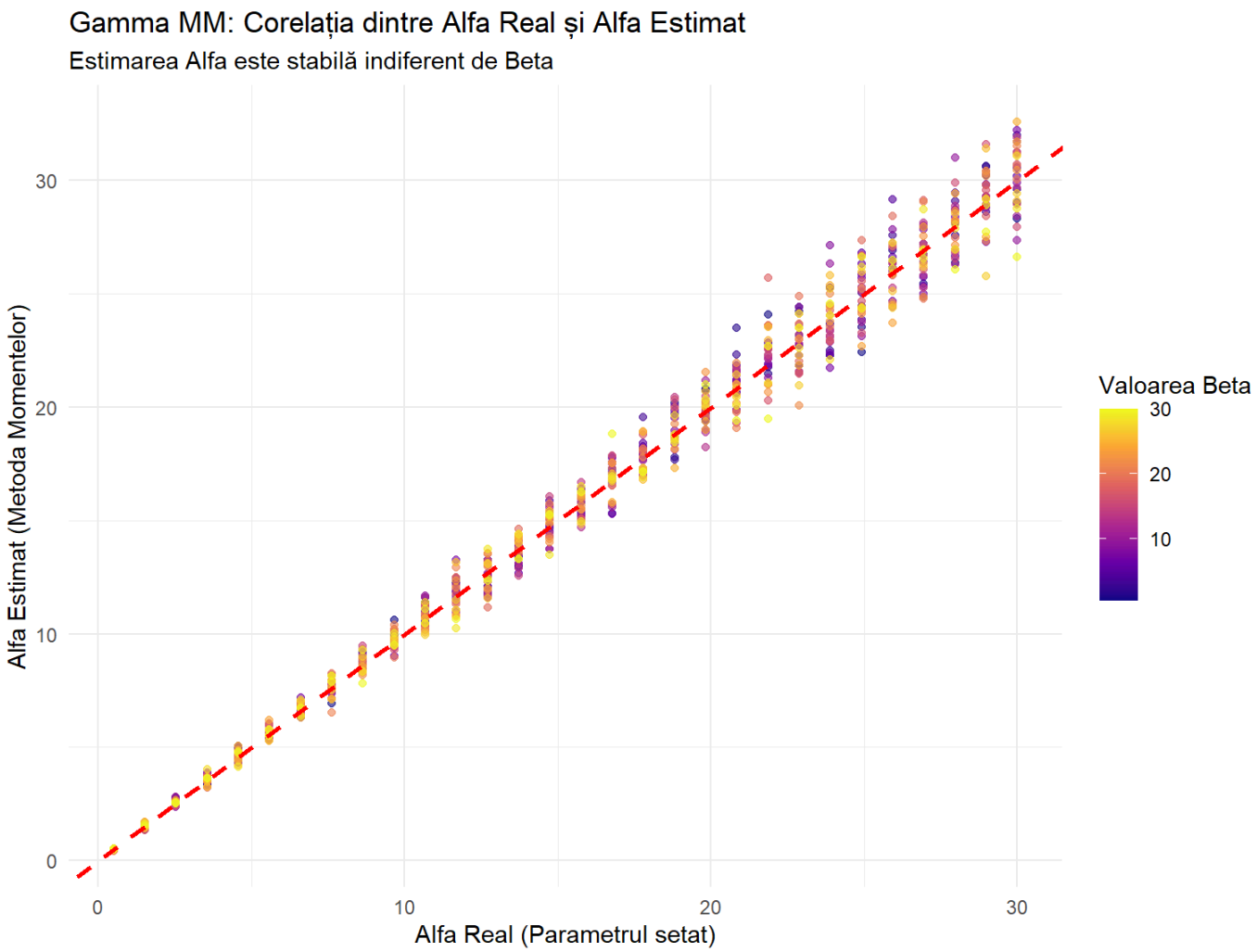
```r
# ==========================================================
# 3. Distribuția Poisson (MM)
# ==========================================================
N <- 1000
lambda_real <- seq(0, 10, length.out = 100)

# Calculul estimatorilor
e_mm_values <- sapply(lambda_real, function(p) {
  x <- rpois(N,p)
  mean(x)
})

# Graficul
plot(lambda_real, e_mm_values,
     main = "Relația dintre lambda real și lambda estimat (Poisson)",
     xlab = "lambda (Real)",
     ylab = "e_mm (Estimat)",
     col = "blue",
     pch = 19)

# Adăugăm linia roșie de referință y = x
abline(a = 0, b = 1, col = "red", lwd = 2, lty = 2)
```



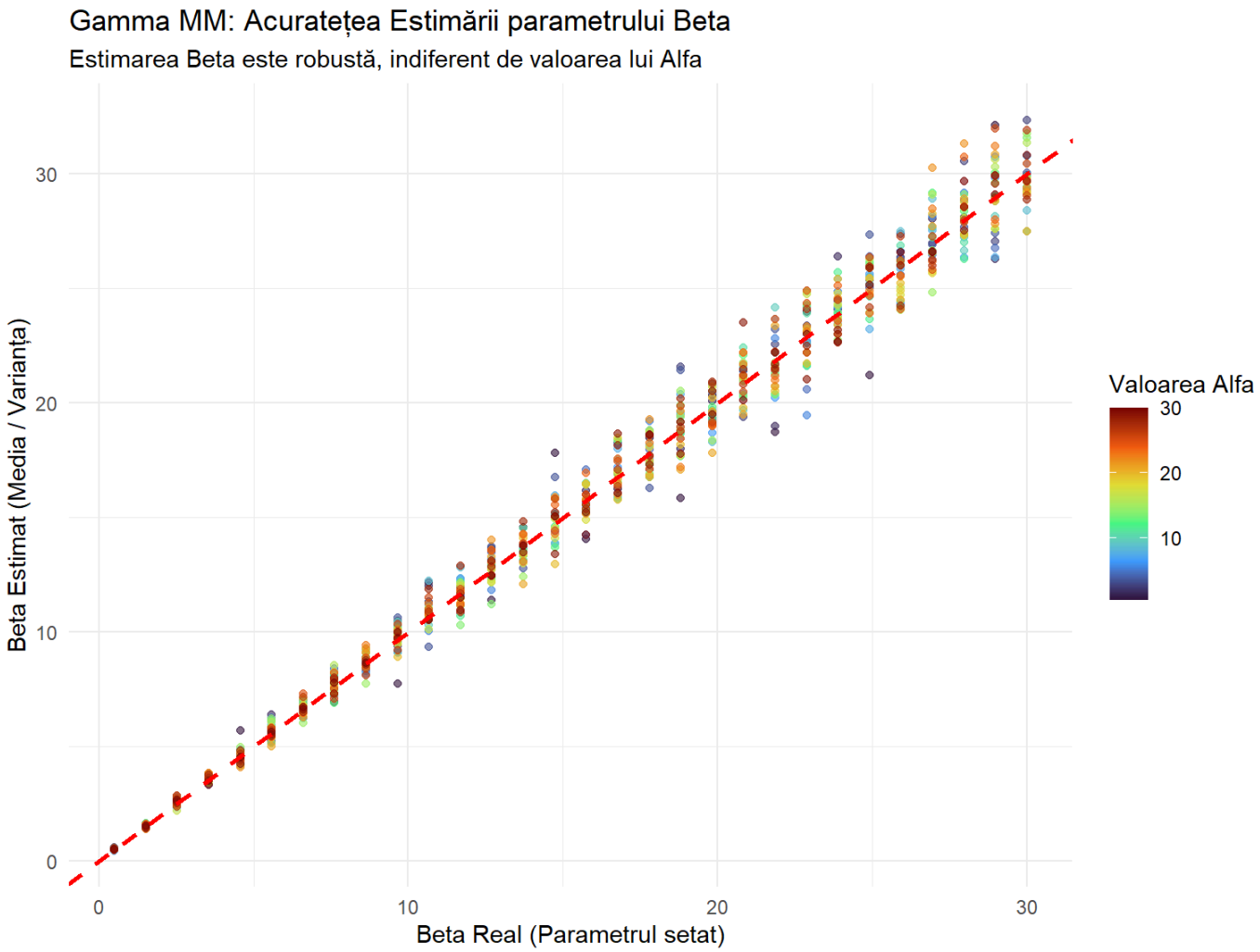
```r
# ==========================================================
# 4. Distribuția Exponențială (MM)
# ==========================================================
N <- 1000
lambda_real <- seq(0, 50, length.out = 200)

# Calculul estimatorilor
e_mm_values <- sapply(lambda_real, function(p) {
  x <- rexp(N,p)
  1/mean(x)
})

# Graficul
plot(lambda_real, e_mm_values,
     main = "Relația dintre lambda real și lambda estimat (Exp)",
     xlab = "lambda (Real)",
     ylab = "e_mm (Estimat)",
     col = "blue",
     pch = 19)

# Adăugăm linia roșie de referință y = x
abline(a = 0, b = 1, col = "red", lwd = 2, lty = 2)
```



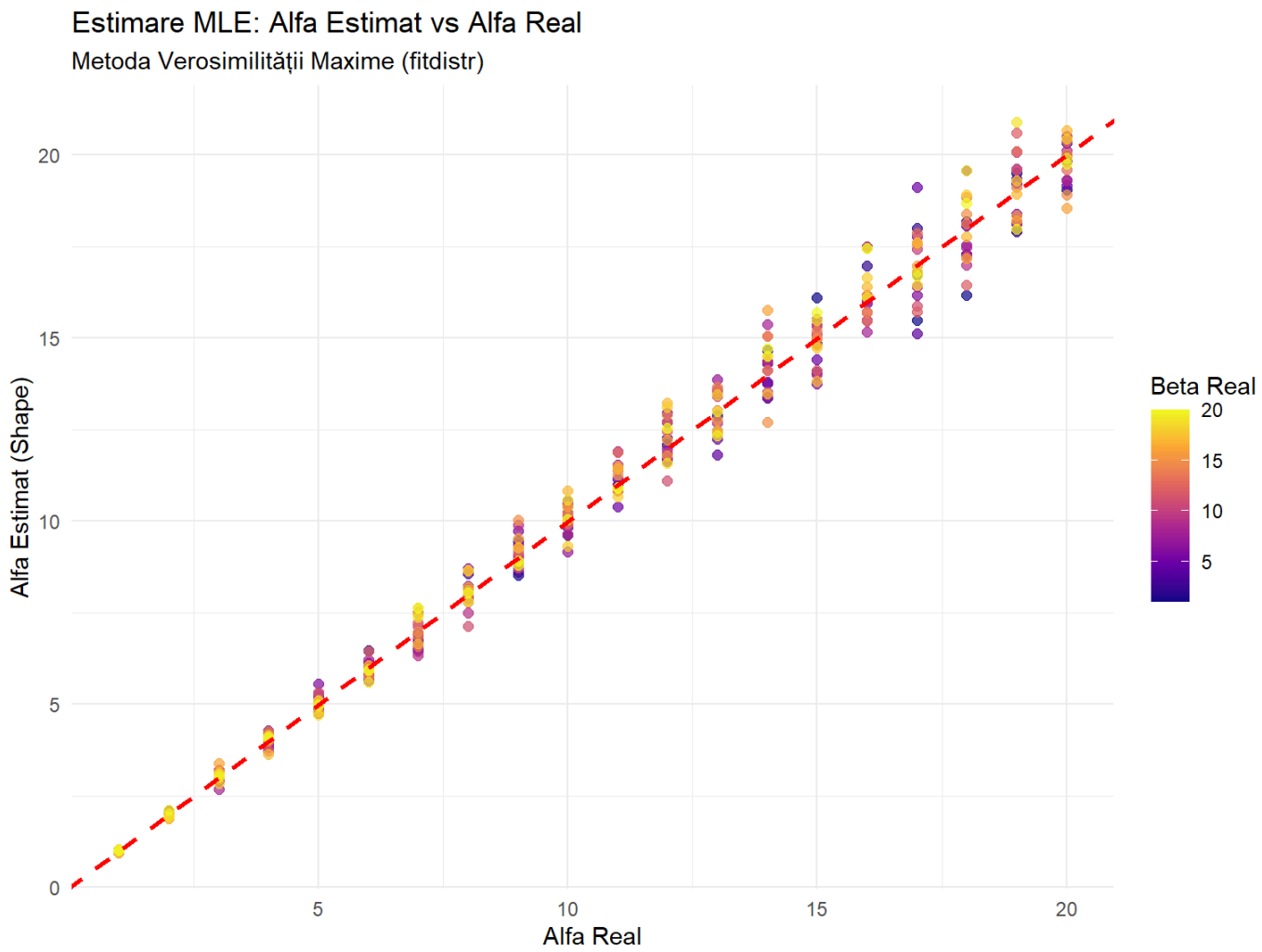
```r
# ==========================================================
# 5. Distribuția Gamma
# Metoda Momentelor (MM) - Formule explicite
# ==========================================================

library(ggplot2)

# 1. Setări
N <- 1000

# Definim secvențele scurte
vals_alfa <- seq(0.5, 30, length.out = 30)
vals_beta <- seq(0.5, 30, length.out = 30)

# 2. Generarea Gridului
param_grid <- expand.grid(Alfa_Real = vals_alfa, Beta_Real = vals_beta)

# 3. Calculul Estimatorilor pe Grid (Alfa)
param_grid$Alfa_Estimat <- mapply(function(a, b) {
 x <- rgamma(N, shape = a, rate = b)
 return((mean(x)^2) / var(x))
}, param_grid$Alfa_Real, param_grid$Beta_Real)

# 4. Graficul pentru Alfa (MM)
p1 <- ggplot(param_grid, aes(x = Alfa_Real, y = Alfa_Estimat)) +
  geom_point(aes(color = Beta_Real), alpha = 0.6, size = 1.5) +
  geom_abline(slope = 1, intercept = 0, color = "red", linetype = "dashed", size = 1) +
  scale_color_viridis_c(option = "plasma", name = "Valoarea Beta") +
  labs(
   title = "Gamma MM: Corelația dintre Alfa Real și Alfa Estimat",
   subtitle = "Estimarea Alfa este stabilă indiferent de Beta",
   x = "Alfa Real (Parametrul setat)",
   y = "Alfa Estimat (Metoda Momentelor)"
  ) +
  theme_minimal()

print(p1)
```



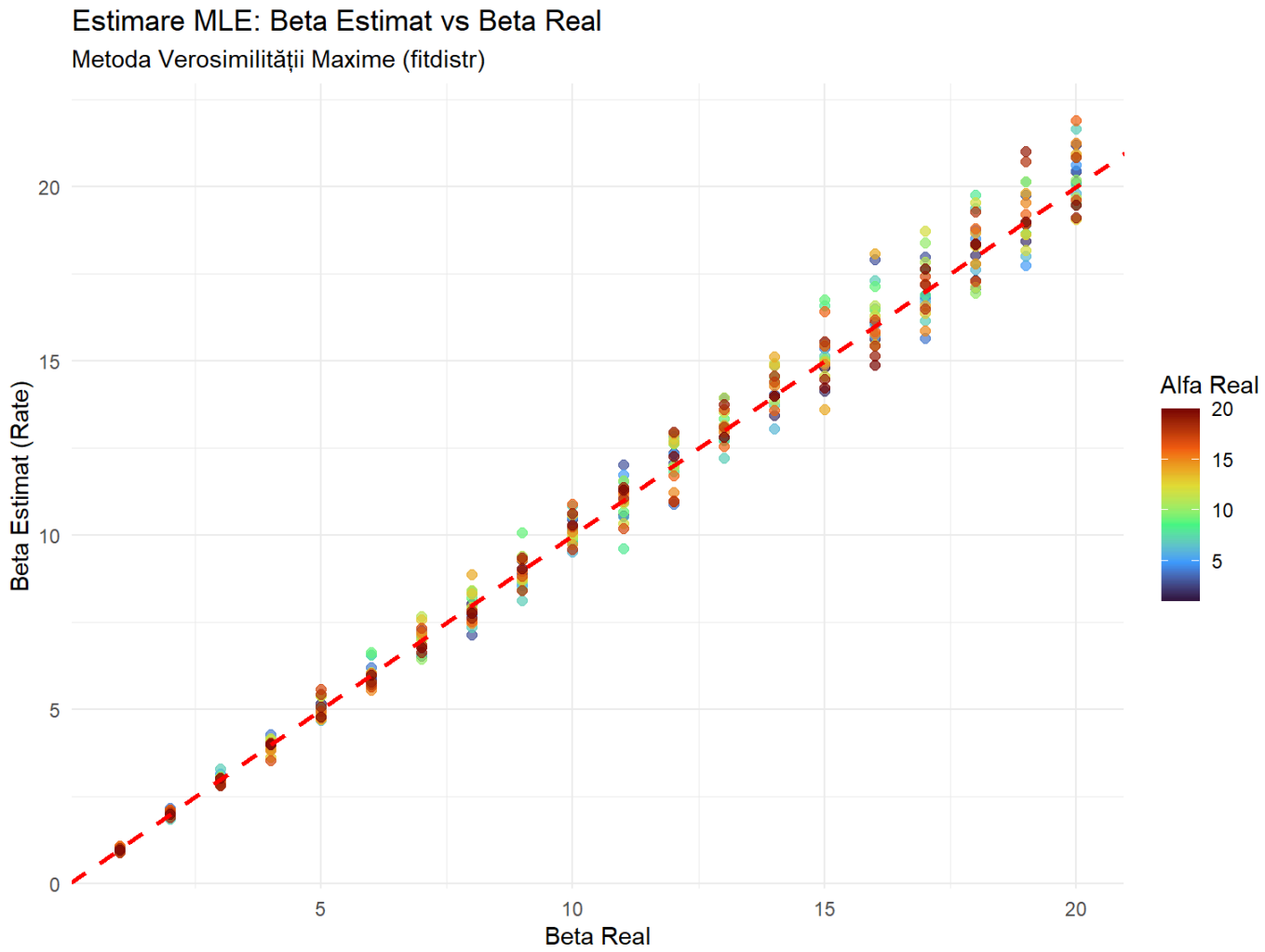

```
# 5. Calculul Estimatorilor pe Grid (Beta)
param_grid$Beta_Estimat <- mapply(function(a, b) {
 x <- rgamma(N, shape = a, rate = b)
 return(mean(x) / var(x))
}, param_grid$Alfa_Real, param_grid$Beta_Real)

# 6. Graficul pentru Beta (MM)
p2 <- ggplot(param_grid, aes(x = Beta_Real, y = Beta_Estimat)) +
  geom_point(aes(color = Alfa_Real), alpha = 0.6, size = 1.5) +
  geom_abline(slope = 1, intercept = 0, color = "red", linetype = "dashed", size = 1) +
  scale_color_viridis_c(option = "turbo", name = "Valoarea Alfa") +
  labs(
   title = "Gamma MM: Acuratețea Estimării parametrului Beta",
   subtitle = "Estimarea Beta este robustă, indiferent de valoarea lui Alfa",
   x = "Beta Real (Parametrul setat)",
   y = "Beta Estimat (Media / Varianța)"
  ) +
  theme_minimal()

print(p2)
```


Gamma MM: Acuratețea Estimării parametrului Beta
Estimarea Beta este robustă, indiferent de valoarea lui Alfa
Beta Estimat (Media / Varianța)
Beta Real (Parametrul setat)
0
10
20
30
Valoarea Alfa
30
20
10

```r
# ==========================================================
# Metoda Verosimilității Maxime (MLE) - Numerică (Gamma)
# ==========================================================
library(MASS) # Pentru fitdistr

# 1. Setări Simulare
N <- 1000
vals_alfa <- seq(1, 20, length.out = 20)
vals_beta <- seq(1, 20, length.out = 20)
param_grid <- expand.grid(Alfa_Real = vals_alfa, Beta_Real = vals_beta)

# 2. Calculul Estimatorilor MLE
# Această parte poate dura câteva secunde!
rezultate_mle <- mapply(function(a, b) {
  x <- rgamma(N, shape = a, rate = b)
  fit <- fitdistr(x, "gamma")
  return(c(fit$estimate["shape"], fit$estimate["rate"]))
}, param_grid$Alfa_Real, param_grid$Beta_Real)

param_grid$Alfa_Estimat <- rezultate_mle[1, ]
param_grid$Beta_Estimat <- rezultate_mle[2, ]

# 3. Graficul Alfa (MLE)
plot_alfa <- ggplot(param_grid, aes(x = Alfa_Real, y = Alfa_Estimat)) +
  geom_point(aes(color = Beta_Real), alpha = 0.7, size = 2) +
  geom_abline(slope = 1, intercept = 0, color = "red", linetype = "dashed", size = 1) +
  scale_color_viridis_c(option = "plasma", name = "Beta Real") +
  labs(
    title = "Estimare MLE: Alfa Estimat vs Alfa Real",
    subtitle = "Metoda Verosimilității Maxime (fitdistr)",
    x = "Alfa Real",
    y = "Alfa Estimat (Shape)"
  ) +
  theme_minimal()

print(plot_alfa)
```




```
# 4. Graficul Beta (MLE)
plot_beta <- ggplot(param_grid, aes(x = Beta_Real, y = Beta_Estimat)) +
  geom_point(aes(color = Alfa_Real), alpha = 0.7, size = 2) +
  geom_abline(slope = 1, intercept = 0, color = "red", linetype = "dashed", size = 1) +
  scale_color_viridis_c(option = "turbo", name = "Alfa Real") +
  labs(
    title = "Estimare MLE: Beta Estimat vs Beta Real",
    subtitle = "Metoda Verosimilității Maxime (fitdistr)",
    x = "Beta Real",
    y = "Beta Estimat (Rate)"
  ) +
  theme_minimal()

print(plot_beta)
```


Estimare MLE: Beta Estimat vs Beta Real
Metoda Verosimilității Maxime (fitdistr)
Beta Estimat (Rate)
Beta Real
0
5
10
15
20
Alfa Real
20
15
10
5

Exercițiul 7 și Exercițiul 8

1.Distribuția Binomială

```
# Problema 7

# Binomială de parametri N și p

# Funcție ce calculează rho = E[|X-mu|^3]
rho_binom <- function(N, p) {
  mu <- N * p
  k <- 0:N

  sum(abs(k - mu)^3 * dbinom(k, size = N, prob = p))
}

# Funcție ce calculează marginea Berry-Esseen, dată de inegalitate
margine_binom <- function(n, N, p, C = 0.4748) {
  mu <- N * p
  sigma <- sqrt(N * p * (1 - p))
  rho <- rho_binom(N, p)

  C * rho / (sigma^3 * sqrt(n))
}

# Funcție ce aproximează convenabil supremumul din inegalitatea Berry-Esseen
sup_binom <- function(n, N, p,
                              M = 1000000,
                              grid_size = 2000,
                              q_low = 1e-4,
                              q_high = 1 - 1e-4) {
  mu <- N * p
  sigma <- sqrt(N * p * (1 - p))

  X <- matrix(
    rbinom(M * n, size = N, prob = p),
    nrow = M,
    ncol = n
  )
  X_bar <- rowMeans(X)

  Z <- sqrt(n) * (X_bar - mu) / sigma

  probs <- seq(q_low, q_high, length.out = grid_size)
  x_grid <- qnorm(probs)

  cdf_empirica <- sapply(
    x_grid,
    function(x) mean(Z <= x)
  )
  cdf_normala <- pnorm(x_grid)

  max(abs(cdf_empirica - cdf_normala))
}

# Funcție ce construiește dataframe-ul cu marginile Berry-Esseen vs. volumul eșantionului
dataframe_binom <- function(n_valori, N, p, C = 0.4748) {
  margini <- sapply(
    n_valori,
    function(n) margine_binom(n, N, p, C)
```

```r
  )

  data.frame(
    n = n_valori,
    margine_BE = margini
  )
}

# Funcție ce reprezintă dependența x vs.
plot_cdf_diferenta_binom <- function(n, N, p,
                                     M = 1000000,
                                     grid_size = 2000,
                                     q_low = 1e-4,
                                     q_high = 1 - 1e-4) {
  mu <- N * p
  sigma <- sqrt(N * p * (1 - p))

  X <- matrix(
    rbinom(M * n, size = N, prob = p),
    nrow = M,
    ncol = n
  )
  X_bar <- rowMeans(X)

  Z <- sqrt(n) * (X_bar - mu) / sigma

  probs <- seq(q_low, q_high, length.out = grid_size)
  x_grid <- qnorm(probs)

  cdf_empirica <- sapply(
    x_grid,
    function(x) mean(Z <= x)
  )
  cdf_normala <- pnorm(x_grid)

  diff_abs <- abs(cdf_empirica - cdf_normala)

  plot(
    x_grid, diff_abs,
    type = "l",
    lwd = 2,
    col = "blue",
    xlab = expression(x),
    ylab = bquote(abs(F[.(n)](x) - Phi(x))),
    main = bquote("|" * F[.(n)](x) - Phi(x) * "|" ~~ "pentru Bin("~.(N)~","~.(p)~")")
  )

  max_idx <- which.max(diff_abs)
  points(x_grid[max_idx], diff_abs[max_idx], col = "red", pch = 19)

  invisible(
    list(
      x = x_grid,
      diff = diff_abs,
      sup_numeric = max(diff_abs)
    )
  )
}

# Parametri
n <- 300   #volumul eșantionului
```

```r
N <- 15      #Bin(N, p)
p <- 0.25    #Bin(N, p)
n_valori <- c(10, 20, 50, 100, 200, 500, 1000, 5000, 10000, 50000, 100000, 500000, 1000000)

# Linii de cod pentru execuție
rho_binom(N, p)
```

```
## [1] 7.456
```

```r
margine_binom(n, N, p)
```

```
## [1] 0.04333284
```

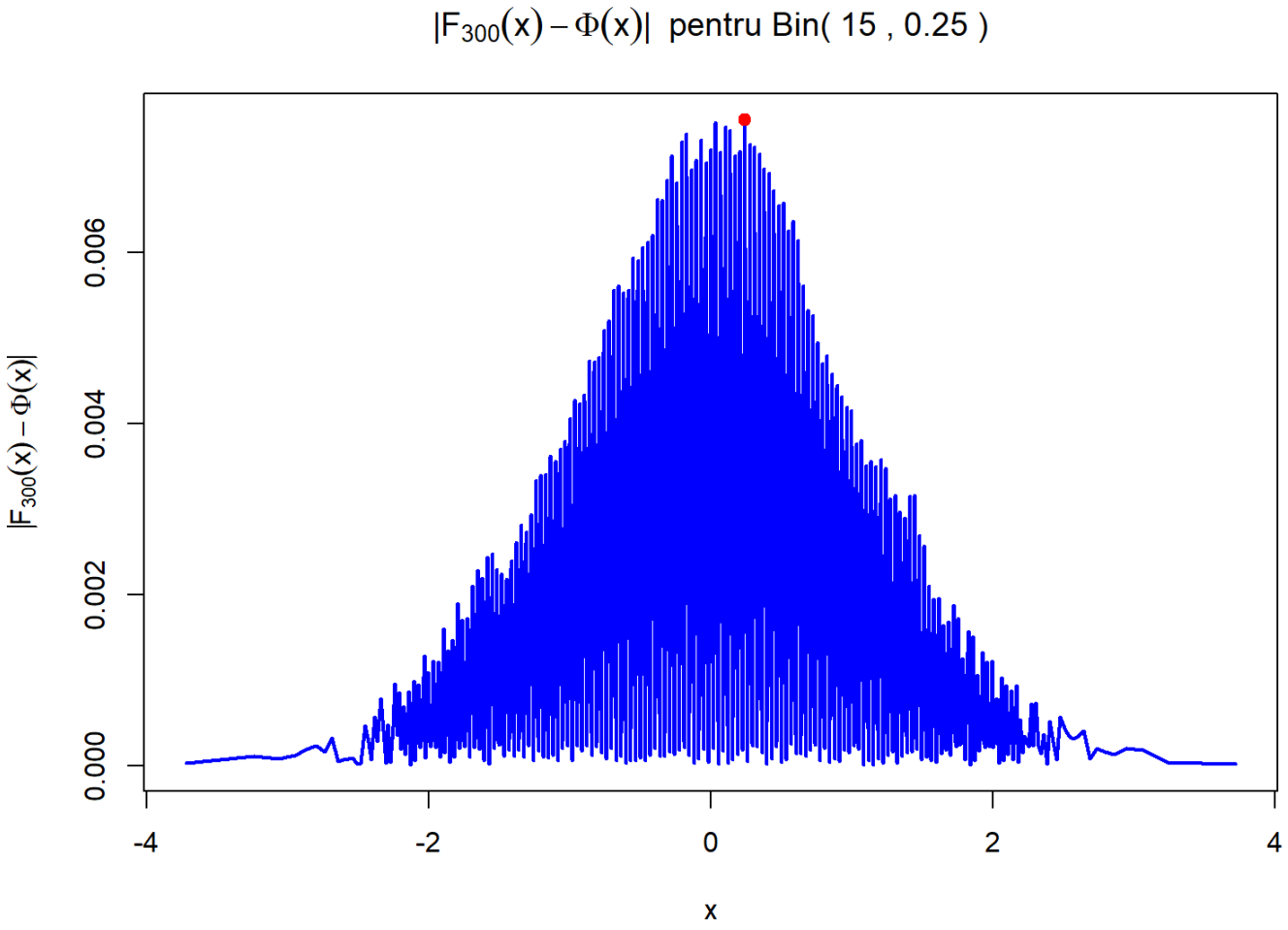
```r
sup_binom(n, N, p)
```

```
## [1] 0.007871179
```

```r
dataframe_binom(n_valori, N, p)
```

```
##        n   margine_BE
## 1  1e+01 0.2373437623
## 2  2e+01 0.1678273838
## 3  5e+01 0.1061433573
## 4  1e+02 0.0750546877
## 5  2e+02 0.0530716787
## 6  5e+02 0.0335654768
## 7  1e+03 0.0237343762
## 8  5e+03 0.0106143357
## 9  1e+04 0.0075054688
## 10 5e+04 0.0033565477
## 11 1e+05 0.0023734376
## 12 5e+05 0.0010614336
## 13 1e+06 0.0007505469
```

```r
plot_cdf_diferenta_binom(n, N, p)
```




```
# Exercițiul 8
#a) definim C_n
C_n <- sup_binom(n, N, p) / margine_binom(n, N, p, C = 1)

#b) folosim deja metoda Monte-Carlo în funcția sup_binom
n_valori <- c(30, 50, 100, 200, 500)

calculeaza_Cn <- function(n, N, p, M = 5000, grid_size = 2000, q_low = 1e-4, q_high = 1 - 1e-4, C = 1)
{
  sup_binom(n, N, p, M, grid_size, q_low, q_high) / margine_binom(n, N, p, C)
}
valori_Cn_n <- sapply(n_valori, function(n) calculeaza_Cn(n, N, p))
data.frame(
  n = n_valori,
  C_n = valori_Cn_n
)
```

```
##     n        C_n
## 1  30 0.09218548
## 2  50 0.11373038
## 3 100 0.11626110
## 4 200 0.11805969
## 5 500 0.34612569
```

```r
#c) variem, pe rând, n (păstrând p constant), apoi p (păstrând n constant)
n_valori <- c(30, 50, 100, 200, 500, 1000, 5000, 10000, 15000, 20000)
valori_Cn_n <- sapply(n_valori, function(n) calculeaza_Cn(n, N, p))
data.frame(
  n = n_valori,
  C_n = valori_Cn_n
)
```

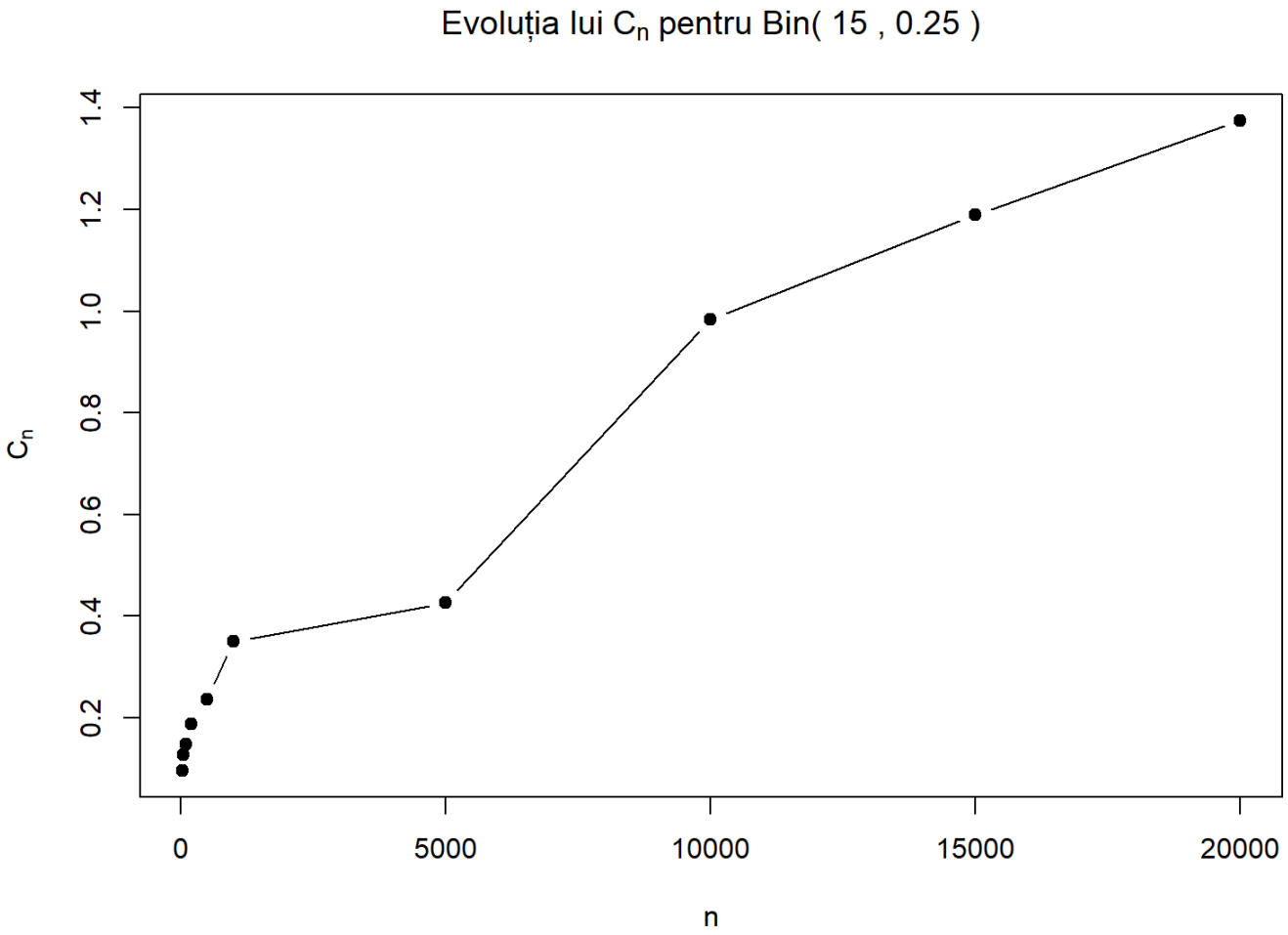
```
##        n       C_n
## 1     30 0.0958272
## 2     50 0.1270224
## 3    100 0.1480205
## 4    200 0.1872463
## 5    500 0.2369358
## 6   1000 0.3502831
## 7   5000 0.4260366
## 8  10000 0.9829751
## 9  15000 1.1895686
## 10 20000 1.3756811
```

```r
p_valori <- seq(1e-6, 1 - 1e-6, length.out = 100)
valori_Cn_p <- sapply(p_valori, function(p) calculeaza_Cn(n, N, p))
data.frame(
  p = p_valori,
  C_n = valori_Cn_p
)
```

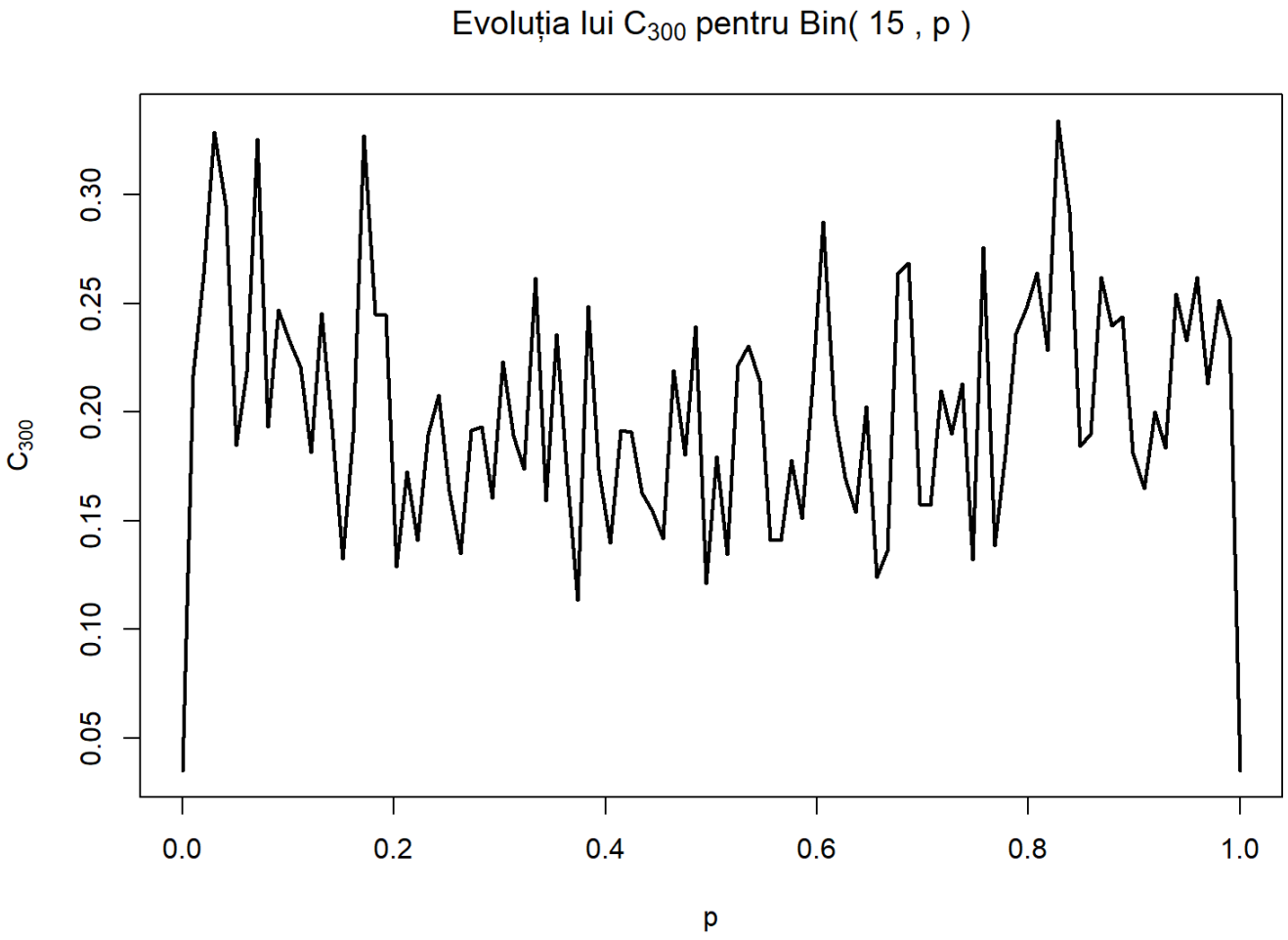
```
##              p        C_n
## 1   0.00000100 0.03492684
## 2   0.01010199 0.21694785
## 3   0.02020298 0.26375649
## 4   0.03030397 0.32893112
## 5   0.04040496 0.29544976
## 6   0.05050595 0.18480553
## 7   0.06060694 0.21854752
## 8   0.07070793 0.32570335
## 9   0.08080892 0.19314863
## 10  0.09090991 0.24700119
## 11  0.10101090 0.23344799
## 12  0.11111189 0.22059188
## 13  0.12121288 0.18162156
## 14  0.13131387 0.24545509
## 15  0.14141486 0.19246600
## 16  0.15151585 0.13270550
## 17  0.16161684 0.19087193
## 18  0.17171783 0.32703600
## 19  0.18181882 0.24495259
## 20  0.19191981 0.24481645
## 21  0.20202080 0.12894990
## 22  0.21212179 0.17241001
## 23  0.22222278 0.14105719
## 24  0.23232377 0.18953078
## 25  0.24242476 0.20779411
## 26  0.25252575 0.16370592
## 27  0.26262674 0.13484911
## 28  0.27272773 0.19143705
## 29  0.28282872 0.19341301
## 30  0.29292971 0.16043432
## 31  0.30303070 0.22302976
## 32  0.31313169 0.18911436
## 33  0.32323268 0.17394218
## 34  0.33333367 0.26146900
## 35  0.34343466 0.15916065
## 36  0.35353565 0.23566868
## 37  0.36363664 0.17106027
## 38  0.37373763 0.11351875
## 39  0.38383862 0.24869441
## 40  0.39393961 0.17353463
## 41  0.40404060 0.13986194
## 42  0.41414159 0.19158046
## 43  0.42424258 0.19087940
## 44  0.43434357 0.16289800
## 45  0.44444456 0.15437550
## 46  0.45454555 0.14187618
## 47  0.46464654 0.21920640
## 48  0.47474753 0.18035819
## 49  0.48484852 0.23947429
## 50  0.49494951 0.12116000
## 51  0.50505049 0.17964722
## 52  0.51515148 0.13444199
## 53  0.52525247 0.22162558
## 54  0.53535346 0.23068753
## 55  0.54545445 0.21418487
## 56  0.55555544 0.14106207
## 57  0.56565643 0.14111362
## 58  0.57575742 0.17794090
## 59  0.58585841 0.15103391
```

```
## 60  0.59595940 0.21268915
## 61  0.60606039 0.28742825
## 62  0.61616138 0.19865869
## 63  0.62626237 0.16989618
## 64  0.63636336 0.15411608
## 65  0.64646435 0.20249611
## 66  0.65656534 0.12401071
## 67  0.66666633 0.13632318
## 68  0.67676732 0.26407701
## 69  0.68686831 0.26847493
## 70  0.69696930 0.15763351
## 71  0.70707029 0.15708248
## 72  0.71717128 0.20975366
## 73  0.72727227 0.18989170
## 74  0.73737326 0.21297803
## 75  0.74747425 0.13206557
## 76  0.75757524 0.27579541
## 77  0.76767623 0.13875839
## 78  0.77777722 0.18077350
## 79  0.78787821 0.23614995
## 80  0.79797920 0.24809368
## 81  0.80808019 0.26396025
## 82  0.81818118 0.22843200
## 83  0.82828217 0.33431205
## 84  0.83838316 0.29187429
## 85  0.84848415 0.18441684
## 86  0.85858514 0.19012597
## 87  0.86868613 0.26219890
## 88  0.87878712 0.23960998
## 89  0.88888811 0.24370489
## 90  0.89898910 0.18098239
## 91  0.90909009 0.16476347
## 92  0.91919108 0.19997550
## 93  0.92929207 0.18343922
## 94  0.93939306 0.25435636
## 95  0.94949405 0.23271940
## 96  0.95959504 0.26218031
## 97  0.96969603 0.21300513
## 98  0.97979702 0.25150155
## 99  0.98989801 0.23437121
## 100 0.99999900 0.03494026
```

```
# d)
plot(
  n_valori, valori_Cn_n,
  type = "b",
  pch = 19,
  xlab = "n",
  ylab = expression(C[n]),
  main = bquote("Evoluția lui" ~ C[n] ~ "pentru Bin("~.(N)~","~.(p)~")")
)
```




```
plot(
  p_valori, valori_Cn_p,
  type = "l",
  lwd = 2,
  xlab = "p",
  ylab = bquote(C[.(n)]),
  main = bquote("Evoluția lui" ~ C[.(n)] ~ "pentru Bin("~.(N)~","~ p ~")")
)
```


Evoluția lui C_{300} pentru Bin(15 , p)
C_{300}
0.05
0.10
0.15
0.20
0.25
0.30
0.0
0.2
0.4
0.6
0.8
1.0
p

2.Distribuția Beta

```
# Problema 7

# Gamma de parametri alfa și beta

# Funcție ce calculează rho = E[|X-mu|^3]
f <- function(x, alfa, beta){
  mu <- alfa / (alfa + beta)

  abs(x - mu)^3 * dbeta(x, shape1 = alfa, shape2 = beta)
}

rho_beta <- function(alfa, beta) {
  integrate(function(x) f(x, alfa, beta), 0, 1)$value
}

# Funcție ce calculează marginea Berry-Esseen, dată de inegalitate
margine_beta <- function(n, alfa, beta, C = 0.4748) {
  mu <- alfa / (alfa + beta)
  sigma <- sqrt(alfa * beta / ((alfa + beta)^2 * (alfa + beta + 1)))
  rho <- rho_beta(alfa, beta)

  C * rho / (sigma^3 * sqrt(n))
}

# Funcție ce aproximează convenabil supremumul din inegalitatea Berry-Esseen
sup_beta <- function(n, alfa, beta,
                     M = 1000000,
                     grid_size = 2000,
                     q_low = 1e-4,
                     q_high = 1 - 1e-4) {
  mu <- alfa / (alfa + beta)
  sigma <- sqrt(alfa * beta / ((alfa + beta)^2 * (alfa + beta + 1)))

  X <- matrix(
    rbeta(M * n, shape1 = alfa, shape2 = beta),
    nrow = M,
    ncol = n
  )
  X_bar <- rowMeans(X)

  Z <- sqrt(n) * (X_bar - mu) / sigma

  probs <- seq(q_low, q_high, length.out = grid_size)
  x_grid <- qnorm(probs)

  cdf_empirica <- sapply(
    x_grid,
    function(x) mean(Z <= x)
  )
  cdf_normala <- pnorm(x_grid)

  max(abs(cdf_empirica - cdf_normala))
}

# Funcție ce construiește dataframe-ul cu marginile Berry-Esseen vs. volumul eșantionului
dataframe_beta <- function(n_valori, alfa, beta, C = 0.4748) {
  margini <- sapply(
    n_valori,
```

```r
    function(n) margine_beta(n, alfa, beta, C)
  )

  data.frame(
    n = n_valori,
    margine_BE = margini
  )
}

# Funcție ce reprezintă dependența x vs.
plot_cdf_diferenta_beta <- function(n, alfa, beta,
                                    M = 1000000,
                                    grid_size = 2000,
                                    q_low = 1e-4,
                                    q_high = 1 - 1e-4) {
  mu <- alfa / (alfa + beta)
  sigma <- sqrt(alfa * beta / ((alfa + beta)^2 * (alfa + beta + 1)))

  X <- matrix(
    rbeta(M * n, shape1 = alfa, shape2 = beta),
    nrow = M,
    ncol = n
  )
  X_bar <- rowMeans(X)

  Z <- sqrt(n) * (X_bar - mu) / sigma

  probs <- seq(q_low, q_high, length.out = grid_size)
  x_grid <- qnorm(probs)

  cdf_empirica <- sapply(
    x_grid,
    function(x) mean(Z <= x)
  )
  cdf_normala <- pnorm(x_grid)

  diff_abs <- abs(cdf_empirica - cdf_normala)

  plot(
    x_grid, diff_abs,
    type = "l",
    lwd = 2,
    col = "blue",
    xlab = expression(x),
    ylab = bquote(abs(F[.(n)](x) - Phi(x))),
    main = bquote("|" * F[.(n)](x) - Phi(x) * "|" ~~ "pentru Beta("~.(alfa)~","~.(beta)~")")
  )

  max_idx <- which.max(diff_abs)
  points(x_grid[max_idx], diff_abs[max_idx], col = "red", pch = 19)

  invisible(
    list(
      x = x_grid,
      diff = diff_abs,
      sup_numeric = max(diff_abs)
    )
  )
}

# Parametri
```

```
n <- 300   #volumul eșantionului
alfa <- 15    #Beta(alfa, beta)
beta <- 70    #Beta(alfa, beta)
n_valori <- c(10, 20, 50, 100, 200, 500, 1000, 5000, 10000, 50000, 100000, 500000, 1000000)

# Linii de cod pentru execuție
rho_beta(alfa, beta)
```

```
## [1] 0.0001123441
```

```
margine_beta(n, alfa, beta)
```

```
## [1] 0.04433236
```

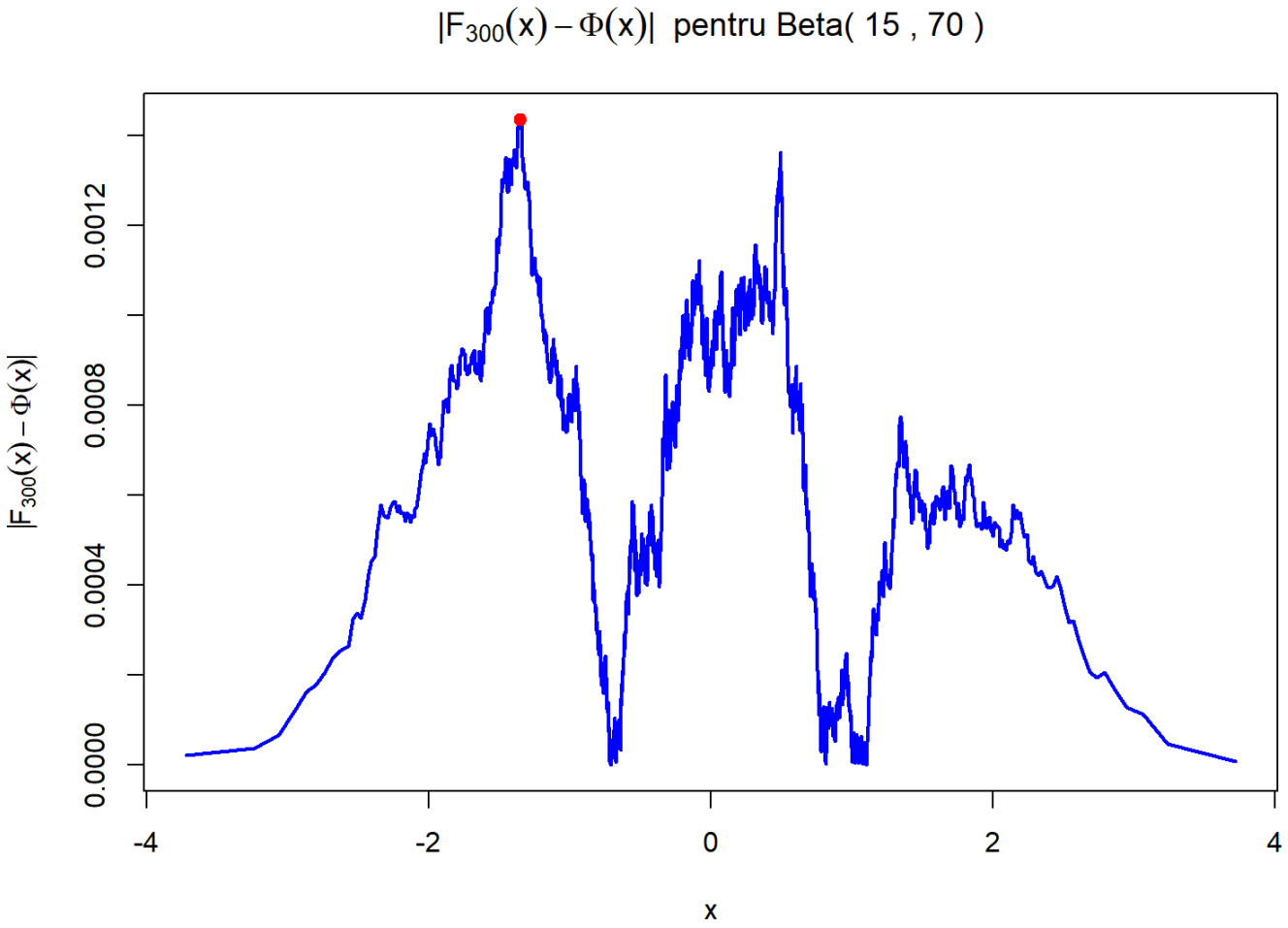
```
sup_beta(n, alfa, beta)
```

```
## [1] 0.001226894
```

```
dataframe_beta(n_valori, alfa, beta)
```

```
##        n  margine_BE
## 1  1e+01 0.242818343
## 2  2e+01 0.171698497
## 3  5e+01 0.108591664
## 4  1e+02 0.076785902
## 5  2e+02 0.054295832
## 6  5e+02 0.034339699
## 7  1e+03 0.024281834
## 8  5e+03 0.010859166
## 9  1e+04 0.007678590
## 10 5e+04 0.003433970
## 11 1e+05 0.002428183
## 12 5e+05 0.001085917
## 13 1e+06 0.000767859
```

```
plot_cdf_diferenta_beta(n, alfa, beta)
```



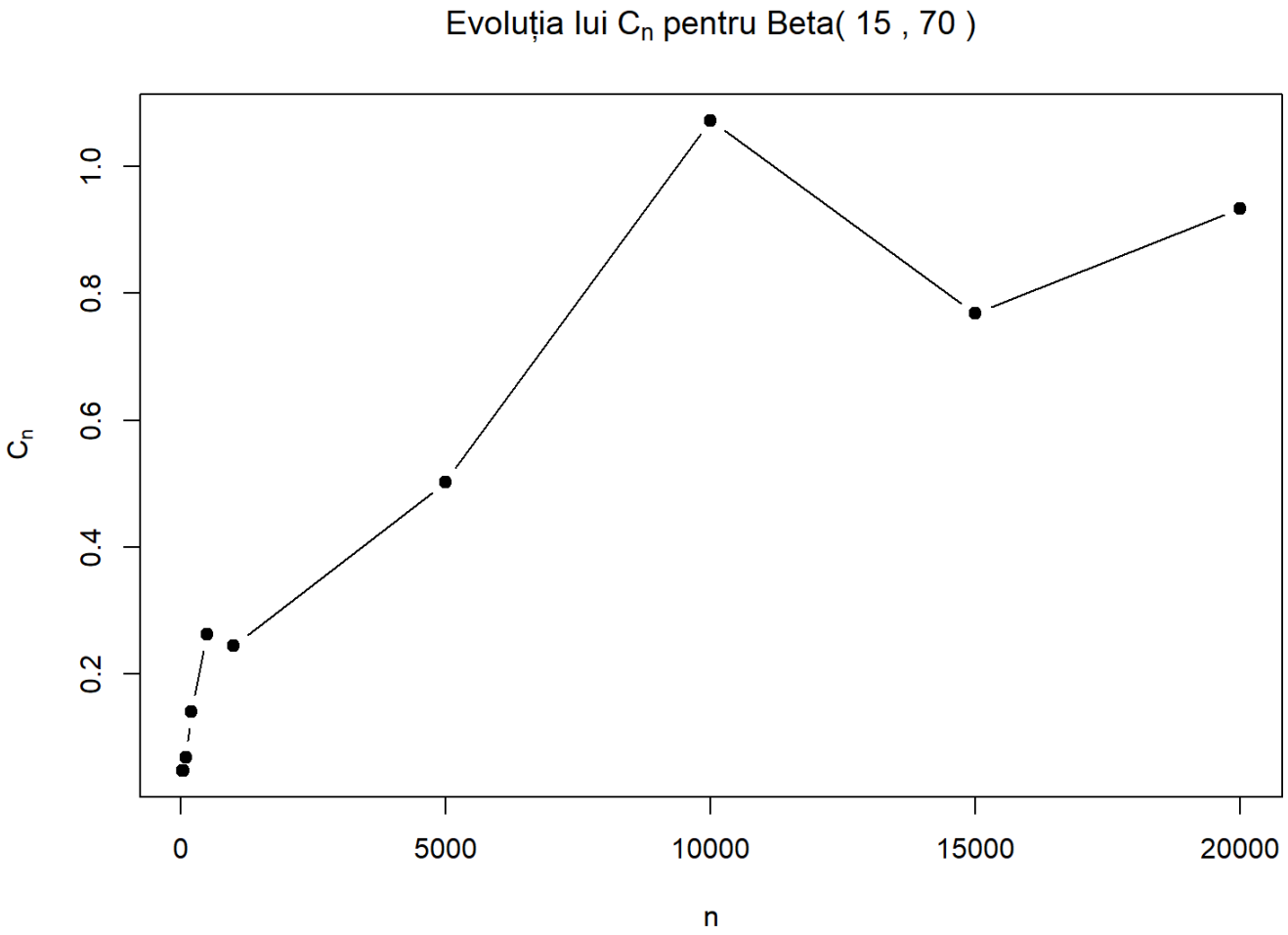

```
# Exercițiul 8
#a) definim C_n
C_n <- sup_beta(n, alfa, beta) / margine_beta(n, alfa, beta, C = 1)

#b) folosim deja metoda Monte-Carlo în funcția sup_binom
n_valori <- c(30, 50, 100, 200, 500)

calculeaza_Cn <- function(n, alfa, beta, M = 5000, grid_size = 2000, q_low = 1e-4, q_high = 1 - 1e-4,
C = 1) {
  sup_beta(n, alfa, beta, M, grid_size, q_low, q_high) / margine_beta(n, alfa, beta, C)
}
valori_Cn_n <- sapply(n_valori, function(n) calculeaza_Cn(n, alfa, beta))
data.frame(
  n = n_valori,
  C_n = valori_Cn_n
)
```

```
##     n        C_n
## 1  30 0.05691132
## 2  50 0.08193779
## 3 100 0.09866417
## 4 200 0.12872711
## 5 500 0.27406820
```

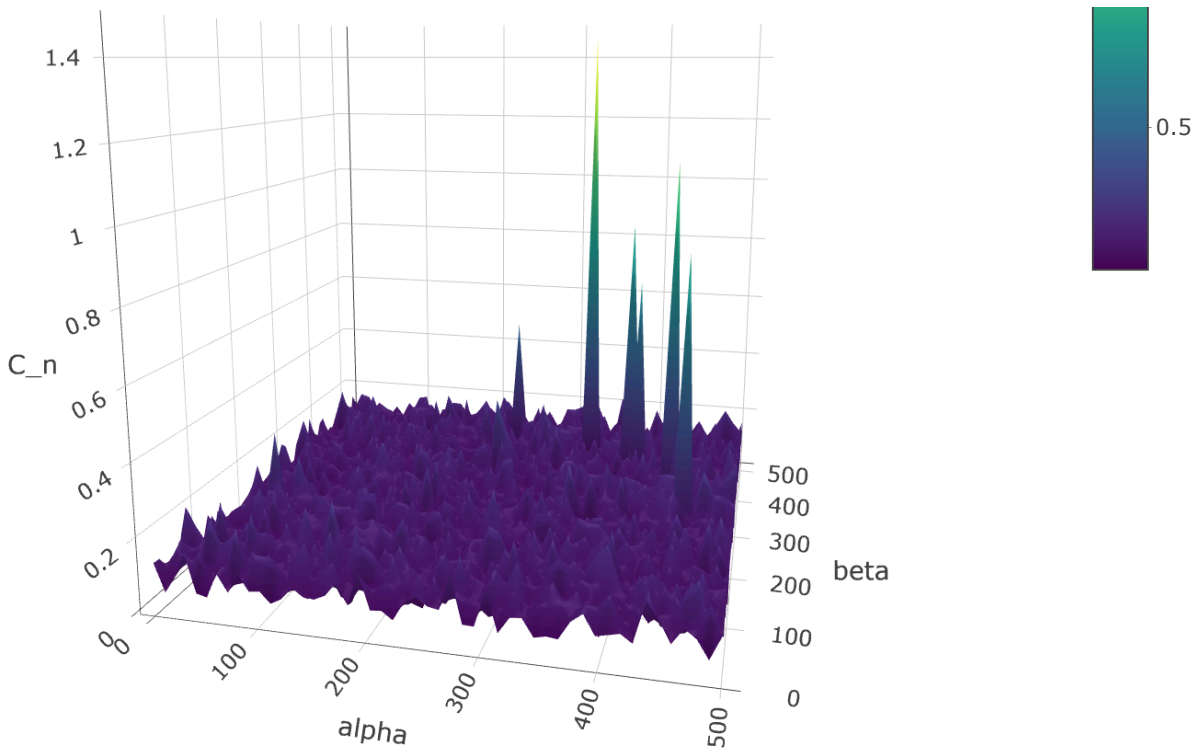
```r
#c) variem, pe rând, n (păstrând alfa și beta constante), apoi alfa și beta (păstrând n constant)
n_valori <- c(30, 50, 100, 200, 500, 1000, 5000, 10000, 15000, 20000)
valori_Cn_n <- sapply(n_valori, function(n) calculeaza_Cn(n, alfa, beta))
data.frame(
  n = n_valori,
  C_n = valori_Cn_n
)
```

```
##        n        C_n
## 1     30 0.04826930
## 2     50 0.04781733
## 3    100 0.06884205
## 4    200 0.14067221
## 5    500 0.26241903
## 6   1000 0.24495940
## 7   5000 0.50234262
## 8  10000 1.07239303
## 9  15000 0.76837809
## 10 20000 0.93259298
```

```r
alfa_valori <- seq(1, 500, length.out = 50)
beta_valori <- seq(1, 500, length.out = 50)
grid_param <- expand.grid(
  alfa = alfa_valori,
  beta = beta_valori
)
n <- 100
grid_param$C_n <- mapply(
  function(a, b) {
    calculeaza_Cn(
      n = n,
      alfa = a,
      beta = b,
      M = 2000
    )
  },
  grid_param$alfa,
  grid_param$beta
)

# d)
plot(
  n_valori, valori_Cn_n,
  type = "b",
  pch = 19,
  xlab = "n",
  ylab = expression(C[n]),
  main = bquote("Evoluția lui" ~ C[n] ~ "pentru Beta("~.(alfa)~","~.(beta)~")")
)
```




```
Cn_matrix <- matrix(
  grid_param$C_n,
  nrow = length(alfa_valori),
  ncol = length(beta_valori),
  byrow = FALSE
)
library(plotly)

plot_ly(
  x = alfa_valori,
  y = beta_valori,
  z = Cn_matrix,
  type = "surface"
) %>%
  layout(
    title = bquote("C"[.(n)] ~ "pentru Beta("~alpha~","~beta~")"),
    scene = list(
      xaxis = list(title = "alpha"),
      yaxis = list(title = "beta"),
      zaxis = list(title = "C_n")
    )
  )
```


C_n
1.4
1.2
1
0.8
0.6
0.4
0.2
0
0
100
200
300
400
500
alpha
500
400
300
200
100
0
beta
0.5

3.Distribuția Exponențială

```
# Problema 7

# Exponențială de parametru lambda

# Funcție ce calculează rho = E[|X-mu|^3]
f <- function(x, lambda){
  mu <- 1 / lambda

  abs(x - mu)^3 * dexp(x, rate = lambda)
}

rho_exp <- function(lambda) {
  integrate(function(x) f(x, lambda), 0, Inf)$value
}

# Funcție ce calculează marginea Berry-Esseen, dată de inegalitate
margine_exp <- function(n, lambda, C = 0.4748) {
  mu <- 1 / lambda
  sigma <- 1 / lambda
  rho <- rho_exp(lambda)

  C * rho / (sigma^3 * sqrt(n))
}

# Funcție ce aproximează convenabil supremumul din inegalitatea Berry-Esseen
sup_exp <- function(n, lambda,
                      M = 1000000,
                      grid_size = 2000,
                      q_low = 1e-4,
                      q_high = 1 - 1e-4) {
  mu <- 1 / lambda
  sigma <- 1 / lambda

  X <- matrix(
    rexp(M * n, rate = lambda),
    nrow = M,
    ncol = n
  )
  X_bar <- rowMeans(X)

  Z <- sqrt(n) * (X_bar - mu) / sigma

  probs <- seq(q_low, q_high, length.out = grid_size)
  x_grid <- qnorm(probs)

  cdf_empirica <- sapply(
    x_grid,
    function(x) mean(Z <= x)
  )
  cdf_normala <- pnorm(x_grid)

  max(abs(cdf_empirica - cdf_normala))
}

# Funcție ce construiește dataframe-ul cu marginile Berry-Esseen vs. volumul eșantionului
dataframe_exp <- function(n_valori, lambda, C = 0.4748) {
  margini <- sapply(
    n_valori,
```

```r
    function(n) margine_exp(n, lambda, C)
  )

  data.frame(
    n = n_valori,
    margine_BE = margini
  )
}

# Funcție ce reprezintă dependența x vs.
plot_cdf_diferenta_exp <- function(n, lambda,
                                   M = 1000000,
                                   grid_size = 2000,
                                   q_low = 1e-4,
                                   q_high = 1 - 1e-4) {
  mu <- 1 / lambda
  sigma <- 1 / lambda

  X <- matrix(
    rexp(M * n, rate = lambda),
    nrow = M,
    ncol = n
  )
  X_bar <- rowMeans(X)

  Z <- sqrt(n) * (X_bar - mu) / sigma

  probs <- seq(q_low, q_high, length.out = grid_size)
  x_grid <- qnorm(probs)

  cdf_empirica <- sapply(
    x_grid,
    function(x) mean(Z <= x)
  )
  cdf_normala <- pnorm(x_grid)

  diff_abs <- abs(cdf_empirica - cdf_normala)

  plot(
    x_grid, diff_abs,
    type = "l",
    lwd = 2,
    col = "blue",
    xlab = expression(x),
    ylab = bquote(abs(F[.(n)](x) - Phi(x))),
    main = bquote("|" * F[.(n)](x) - Phi(x) * "|" ~~ "pentru Exp("~.(lambda)~")")
  )

  max_idx <- which.max(diff_abs)
  points(x_grid[max_idx], diff_abs[max_idx], col = "red", pch = 19)

  invisible(
    list(
      x = x_grid,
      diff = diff_abs,
      sup_numeric = max(diff_abs)
    )
  )
}

# Parametri
```

```
n <- 300   #volumul eșantionului
lambda <- 23    #Exp(lambda)
n_valori <- c(10, 20, 50, 100, 200, 500, 1000, 5000, 10000, 50000, 100000, 500000, 1000000)

# Linii de cod pentru execuție
rho_exp(lambda)
```

```
## [1] 0.0001984244
```

```
margine_exp(n, lambda)
```

```
## [1] 0.06618029
```

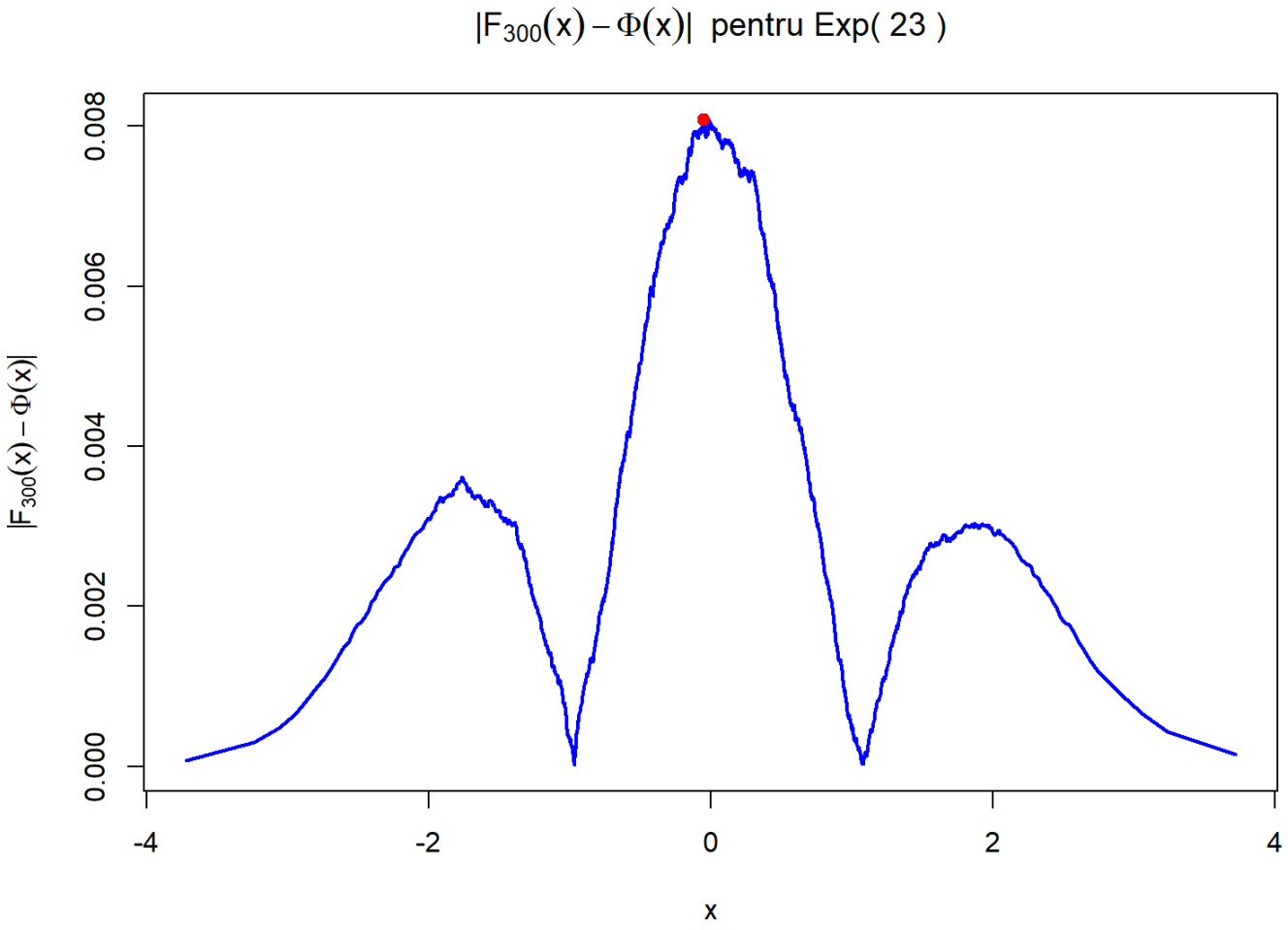
```
sup_exp(n, lambda)
```

```
## [1] 0.007503825
```

```
dataframe_exp(n_valori, lambda)
```

```
##        n  margine_BE
## 1  1e+01 0.362484365
## 2  2e+01 0.256315152
## 3  5e+01 0.162107936
## 4  1e+02 0.114627621
## 5  2e+02 0.081053968
## 6  5e+02 0.051263030
## 7  1e+03 0.036248436
## 8  5e+03 0.016210794
## 9  1e+04 0.011462762
## 10 5e+04 0.005126303
## 11 1e+05 0.003624844
## 12 5e+05 0.001621079
## 13 1e+06 0.001146276
```

```
plot_cdf_diferenta_exp(n, lambda)
```




```
# Exercițiul 8
#a) definim C_n
C_n <- sup_exp(n, lambda) / margine_exp(n, lambda, C = 1)

#b) folosim deja metoda Monte-Carlo în funcția sup_binom
n_valori <- c(30, 50, 100, 200, 500)

calculeaza_Cn <- function(n, lambda, M = 5000, grid_size = 2000, q_low = 1e-4, q_high = 1 - 1e-4, C =
1) {
  sup_exp(n, lambda, M, grid_size, q_low, q_high) / margine_exp(n, lambda, C)
}
valori_Cn_n <- sapply(n_valori, function(n) calculeaza_Cn(n, lambda))
data.frame(
  n = n_valori,
  C_n = valori_Cn_n
)
```

```
##     n        C_n
## 1  30 0.04262197
## 2  50 0.08191915
## 3 100 0.07046080
## 4 200 0.12369389
## 5 500 0.10423195
```

```r
#c) variem, pe rând, n (păstrând lambda constant), apoi lambda (păstrând n constant)
n_valori <- c(30, 50, 100, 200, 500, 1000, 5000, 10000, 15000, 20000)
valori_Cn_n <- sapply(n_valori, function(n) calculeaza_Cn(n, lambda))
data.frame(
  n = n_valori,
  C_n = valori_Cn_n
)
```

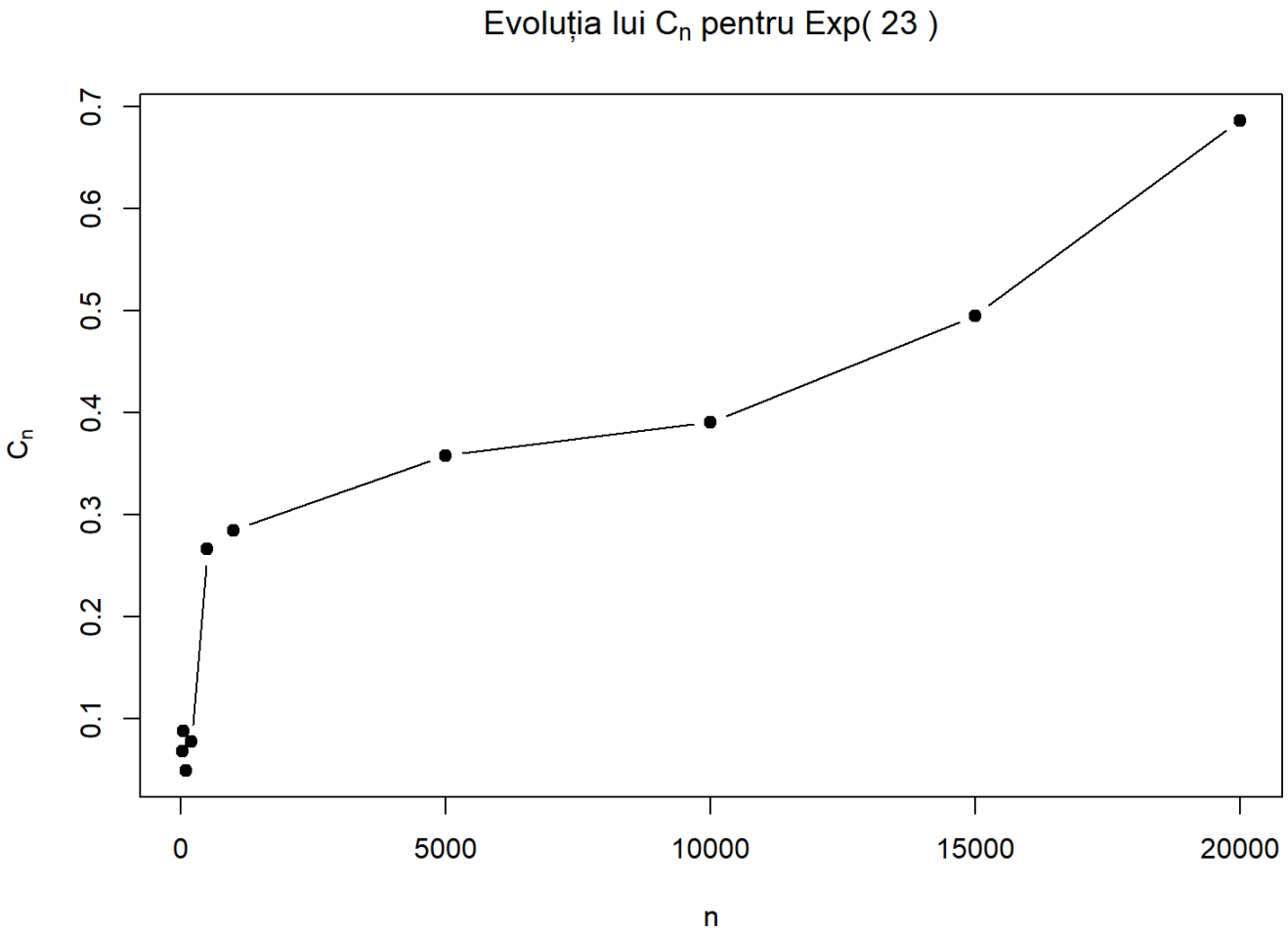
```
##        n        C_n
## 1     30 0.06744540
## 2     50 0.08799998
## 3    100 0.04869723
## 4    200 0.07747538
## 5    500 0.26592791
## 6   1000 0.28432517
## 7   5000 0.35780354
## 8  10000 0.39025538
## 9  15000 0.49475324
## 10 20000 0.68644396
```

```r
lambda_valori <- seq(1e-4, 1e+3, length.out = 100)
valori_Cn_lambda <- sapply(lambda_valori, function(lambda) calculeaza_Cn(n, lambda))
data.frame(
  lambda = lambda_valori,
  C_n = valori_Cn_lambda
)
```

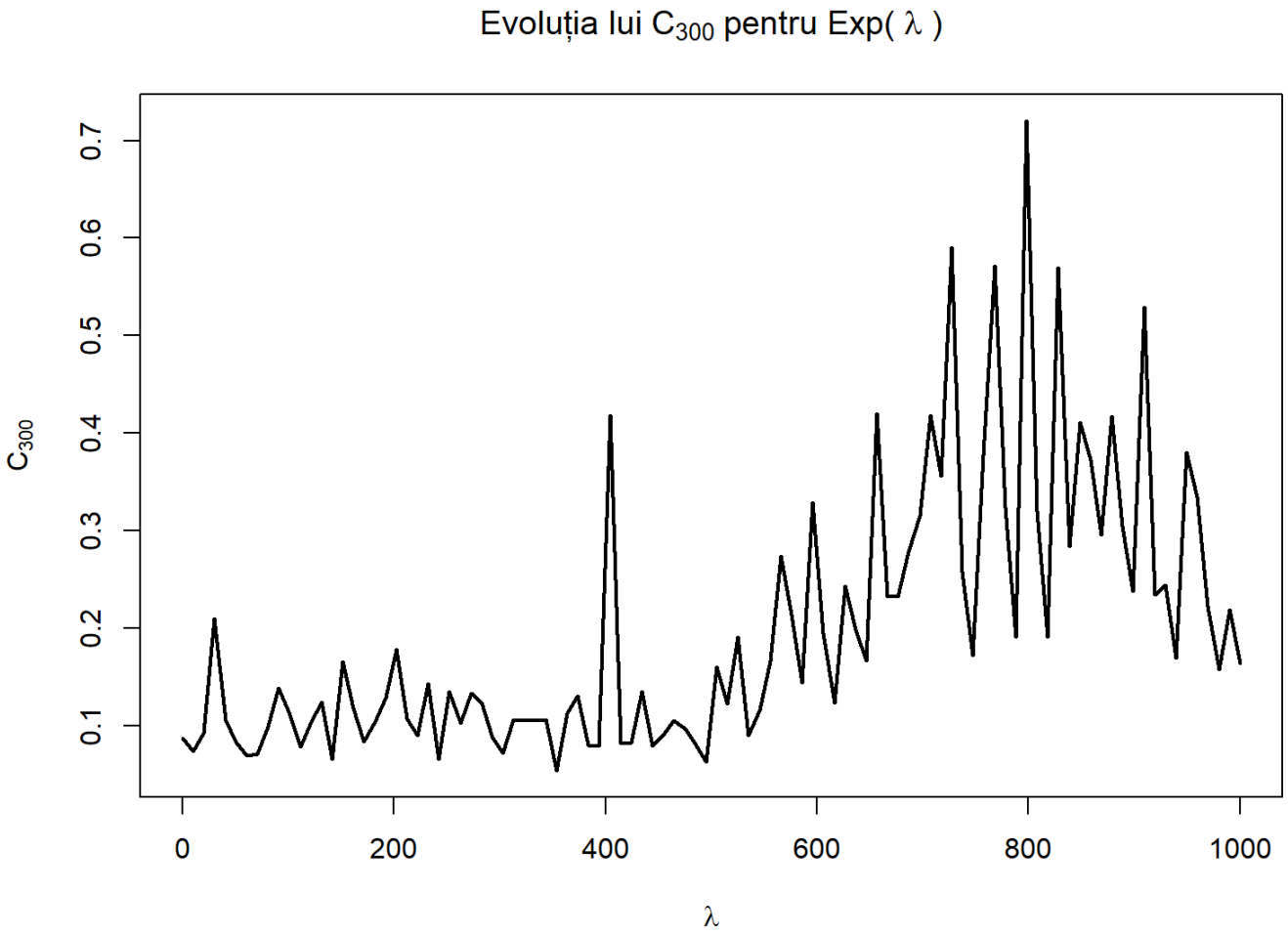
```
##         lambda        C_n
## 1      0.00010 0.08745746
## 2     10.10111 0.07349132
## 3     20.20212 0.09280542
## 4     30.30313 0.20951255
## 5     40.40414 0.10477905
## 6     50.50515 0.08241751
## 7     60.60615 0.06927132
## 8     70.70716 0.07086034
## 9     80.80817 0.09917188
## 10    90.90918 0.13883306
## 11   101.01019 0.11346407
## 12   111.11120 0.07777345
## 13   121.21221 0.10357196
## 14   131.31322 0.12424402
## 15   141.41423 0.06573256
## 16   151.51524 0.16540891
## 17   161.61625 0.11737831
## 18   171.71725 0.08368630
## 19   181.81826 0.10461420
## 20   191.91927 0.12880991
## 21   202.02028 0.17798013
## 22   212.12129 0.10688558
## 23   222.22230 0.08939307
## 24   232.32331 0.14292807
## 25   242.42432 0.06497163
## 26   252.52533 0.13455589
## 27   262.62634 0.10207332
## 28   272.72735 0.13313349
## 29   282.82835 0.12345049
## 30   292.92936 0.08786387
## 31   303.03037 0.07190173
## 32   313.13138 0.10552653
## 33   323.23239 0.10474921
## 34   333.33340 0.10486910
## 35   343.43441 0.10568462
## 36   353.53542 0.05357157
## 37   363.63643 0.11240036
## 38   373.73744 0.13006047
## 39   383.83845 0.07998409
## 40   393.93945 0.07886538
## 41   404.04046 0.41808390
## 42   414.14147 0.08167256
## 43   424.24248 0.08250195
## 44   434.34349 0.13487215
## 45   444.44450 0.07932081
## 46   454.54551 0.09005965
## 47   464.64652 0.10521453
## 48   474.74753 0.09656782
## 49   484.84854 0.08063316
## 50   494.94955 0.06238344
## 51   505.05055 0.16039537
## 52   515.15156 0.12217792
## 53   525.25257 0.19053681
## 54   535.35358 0.09015537
## 55   545.45459 0.11626315
## 56   555.55560 0.16604924
## 57   565.65661 0.27402054
## 58   575.75762 0.21323228
## 59   585.85863 0.14416873
```

```
## 60   595.95964 0.32880624
## 61   606.06065 0.19463135
## 62   616.16165 0.12342965
## 63   626.26266 0.24289041
## 64   636.36367 0.19806135
## 65   646.46468 0.16652605
## 66   656.56569 0.42005381
## 67   666.66670 0.23231562
## 68   676.76771 0.23231762
## 69   686.86872 0.27801236
## 70   696.96973 0.31503119
## 71   707.07074 0.41851204
## 72   717.17175 0.35581738
## 73   727.27275 0.59002103
## 74   737.37376 0.25791675
## 75   747.47477 0.17169020
## 76   757.57578 0.37749256
## 77   767.67679 0.57186214
## 78   777.77780 0.32349097
## 79   787.87881 0.19079427
## 80   797.97982 0.72078669
## 81   808.08083 0.31943899
## 82   818.18184 0.19115925
## 83   828.28285 0.56929244
## 84   838.38385 0.28331074
## 85   848.48486 0.41125522
## 86   858.58587 0.37193668
## 87   868.68688 0.29585883
## 88   878.78789 0.41681433
## 89   888.88890 0.30414156
## 90   898.98991 0.23736167
## 91   909.09092 0.52924979
## 92   919.19193 0.23437444
## 93   929.29294 0.24443159
## 94   939.39395 0.16887695
## 95   949.49495 0.37995603
## 96   959.59596 0.33444021
## 97   969.69697 0.22259039
## 98   979.79798 0.15761747
## 99   989.89899 0.21884649
## 100 1000.00000 0.16399714
```

```
# d)
plot(
  n_valori, valori_Cn_n,
  type = "b",
  pch = 19,
  xlab = "n",
  ylab = expression(C[n]),
  main = bquote("Evoluția lui" ~ C[n] ~ "pentru Exp("~.(lambda)~")")
)
```




```
plot(
  lambda_valori, valori_Cn_lambda,
  type = "l",
  lwd = 2,
  xlab = bquote(lambda),
  ylab = bquote(C[.(n)]),
  main = bquote("Evoluția lui" ~ C[.(n)] ~ "pentru Exp("~ lambda ~")")
)
```


Evoluția lui C_{300} pentru Exp(λ)
C_{300}
0.1
0.2
0.3
0.4
0.5
0.6
0.7
0
200
400
600
800
1000
λ

4.Distribuția Gamma

```
# Problema 7

# Gamma de parametri alfa și beta

# Funcție ce calculează rho = E[|X-mu|^3]
f <- function(x, alfa, beta){
  mu <- alfa / beta

  abs(x - mu)^3 * dgamma(x, shape = alfa, rate = beta)
}

rho_gamma <- function(alfa, beta) {
  integrate(function(x) f(x, alfa, beta), 0, Inf)$value
}

# Funcție ce calculează marginea Berry-Esseen, dată de inegalitate
margine_gamma <- function(n, alfa, beta, C = 0.4748) {
  mu <- alfa / beta
  sigma <- sqrt(alfa) / beta
  rho <- rho_gamma(alfa, beta)

  C * rho / (sigma^3 * sqrt(n))
}

# Funcție ce aproximează convenabil supremumul din inegalitatea Berry-Esseen
sup_gamma <- function(n, alfa, beta,
                      M = 1000000,
                      grid_size = 2000,
                      q_low = 1e-4,
                      q_high = 1 - 1e-4) {
  mu <- alfa / beta
  sigma <- sqrt(alfa) / beta

  X <- matrix(
    rgamma(M * n, shape = alfa, rate = beta),
    nrow = M,
    ncol = n
  )
  X_bar <- rowMeans(X)

  Z <- sqrt(n) * (X_bar - mu) / sigma

  probs <- seq(q_low, q_high, length.out = grid_size)
  x_grid <- qnorm(probs)

  cdf_empirica <- sapply(
    x_grid,
    function(x) mean(Z <= x)
  )
  cdf_normala <- pnorm(x_grid)

  max(abs(cdf_empirica - cdf_normala))
}

# Funcție ce construiește dataframe-ul cu marginile Berry-Esseen vs. volumul eșantionului
dataframe_gamma <- function(n_valori, alfa, beta, C = 0.4748) {
  margini <- sapply(
    n_valori,
```

```r
    function(n) margine_gamma(n, alfa, beta, C)
  )

  data.frame(
    n = n_valori,
    margine_BE = margini
  )
}

# Funcție ce reprezintă dependența x vs.
plot_cdf_diferenta_gamma <- function(n, alfa, beta,
                                     M = 1000000,
                                     grid_size = 2000,
                                     q_low = 1e-4,
                                     q_high = 1 - 1e-4) {
  mu <- alfa / beta
  sigma <- sqrt(alfa) / beta

  X <- matrix(
    rgamma(M * n, shape = alfa, rate = beta),
    nrow = M,
    ncol = n
  )
  X_bar <- rowMeans(X)

  Z <- sqrt(n) * (X_bar - mu) / sigma

  probs <- seq(q_low, q_high, length.out = grid_size)
  x_grid <- qnorm(probs)

  cdf_empirica <- sapply(
    x_grid,
    function(x) mean(Z <= x)
  )
  cdf_normala <- pnorm(x_grid)

  diff_abs <- abs(cdf_empirica - cdf_normala)

  plot(
    x_grid, diff_abs,
    type = "l",
    lwd = 2,
    col = "blue",
    xlab = expression(x),
    ylab = bquote(abs(F[.(n)](x) - Phi(x))),
    main = bquote("|" * F[.(n)](x) - Phi(x) * "|" ~~ "pentru Gamma("~.(alfa)~","~.(beta)~")")
  )

  max_idx <- which.max(diff_abs)
  points(x_grid[max_idx], diff_abs[max_idx], col = "red", pch = 19)

  invisible(
    list(
      x = x_grid,
      diff = diff_abs,
      sup_numeric = max(diff_abs)
    )
  )
}

# Parametri
```

```r
n <- 300   #volumul eșantionului
alfa <- 13    #Gamma(alfa, beta)
beta <- 50    #Gamma(alfa, beta)
n_valori <- c(10, 20, 50, 100, 200, 500, 1000, 5000, 10000, 50000, 100000, 500000, 1000000)

# Linii de cod pentru execuție
rho_gamma(alfa, beta)
```

```
## [1] 0.0006249291
```

```r
margine_gamma(n, alfa, beta)
```

```
## [1] 0.04568523
```

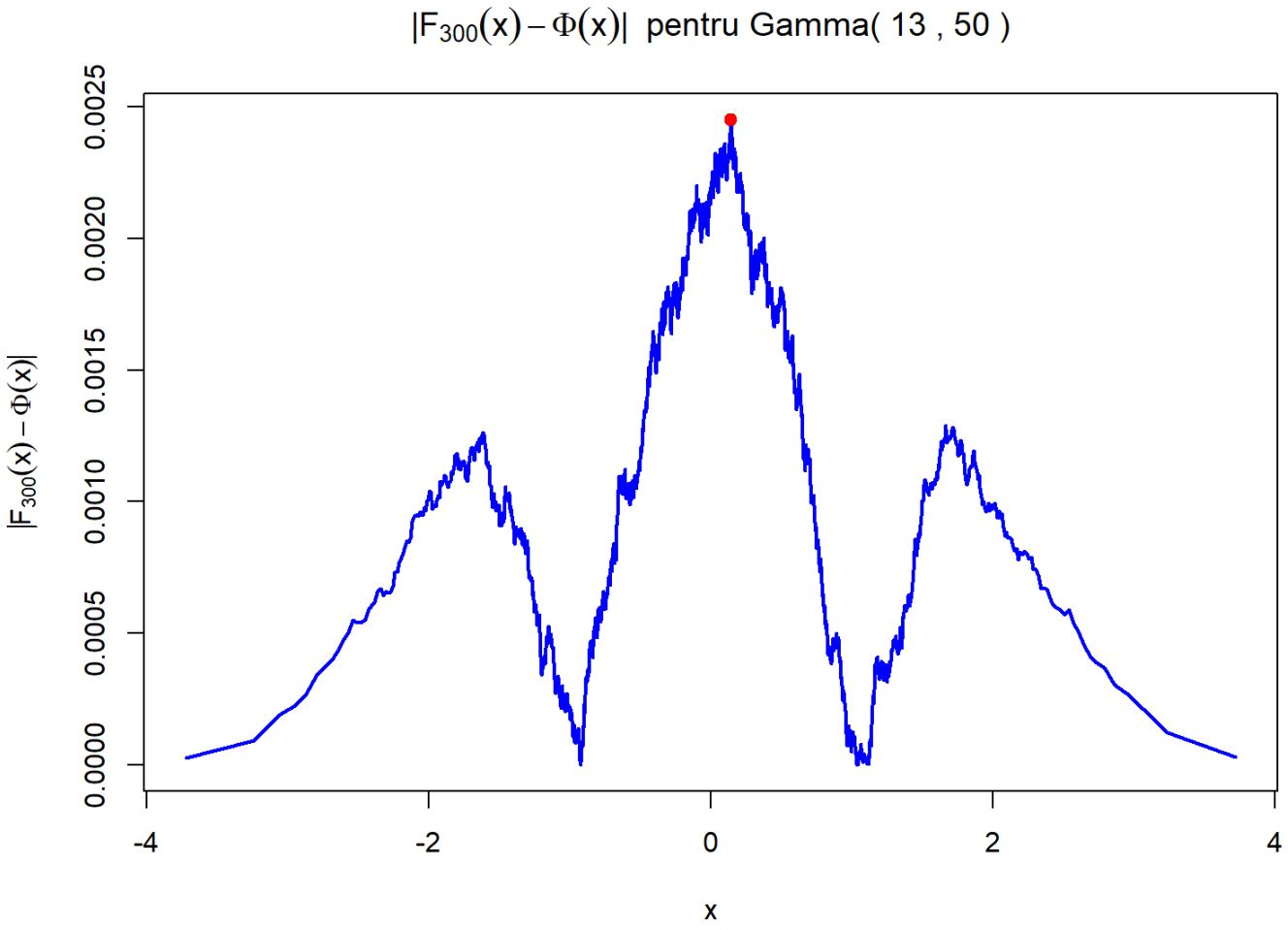
```r
sup_gamma(n, alfa, beta)
```

```
## [1] 0.002247071
```

```r
dataframe_gamma(n_valori, alfa, beta)
```

```
##        n   margine_BE
## 1  1e+01 0.2502282915
## 2  2e+01 0.1769381218
## 3  5e+01 0.1119054939
## 4  1e+02 0.0791291336
## 5  2e+02 0.0559527470
## 6  5e+02 0.0353876244
## 7  1e+03 0.0250228291
## 8  5e+03 0.0111905494
## 9  1e+04 0.0079129134
## 10 5e+04 0.0035387624
## 11 1e+05 0.0025022829
## 12 5e+05 0.0011190549
## 13 1e+06 0.0007912913
```

```r
plot_cdf_diferenta_gamma(n, alfa, beta)
```




```
# Exercițiul 8
#a) definim C_n
C_n <- sup_gamma(n, alfa, beta) / margine_gamma(n, alfa, beta, C = 1)

#b) folosim deja metoda Monte-Carlo în funcția sup_binom
n_valori <- c(30, 50, 100, 200, 500)

calculeaza_Cn <- function(n, alfa, beta, M = 5000, grid_size = 2000, q_low = 1e-4, q_high = 1 - 1e-4,
C = 1) {
  sup_gamma(n, alfa, beta, M, grid_size, q_low, q_high) / margine_gamma(n, alfa, beta, C)
}
valori_Cn_n <- sapply(n_valori, function(n) calculeaza_Cn(n, alfa, beta))
data.frame(
  n = n_valori,
  C_n = valori_Cn_n
)
```

```
##     n        C_n
## 1  30 0.06042427
## 2  50 0.09609104
## 3 100 0.04342111
## 4 200 0.10614636
## 5 500 0.08066985
```

```r
#c) variem, pe rând, n (păstrând alfa și beta constante), apoi alfa și beta (păstrând n constant)
n_valori <- c(30, 50, 100, 200, 500, 1000, 5000, 10000, 15000, 20000)
valori_Cn_n <- sapply(n_valori, function(n) calculeaza_Cn(n, alfa, beta))
data.frame(
  n = n_valori,
  C_n = valori_Cn_n
)
```

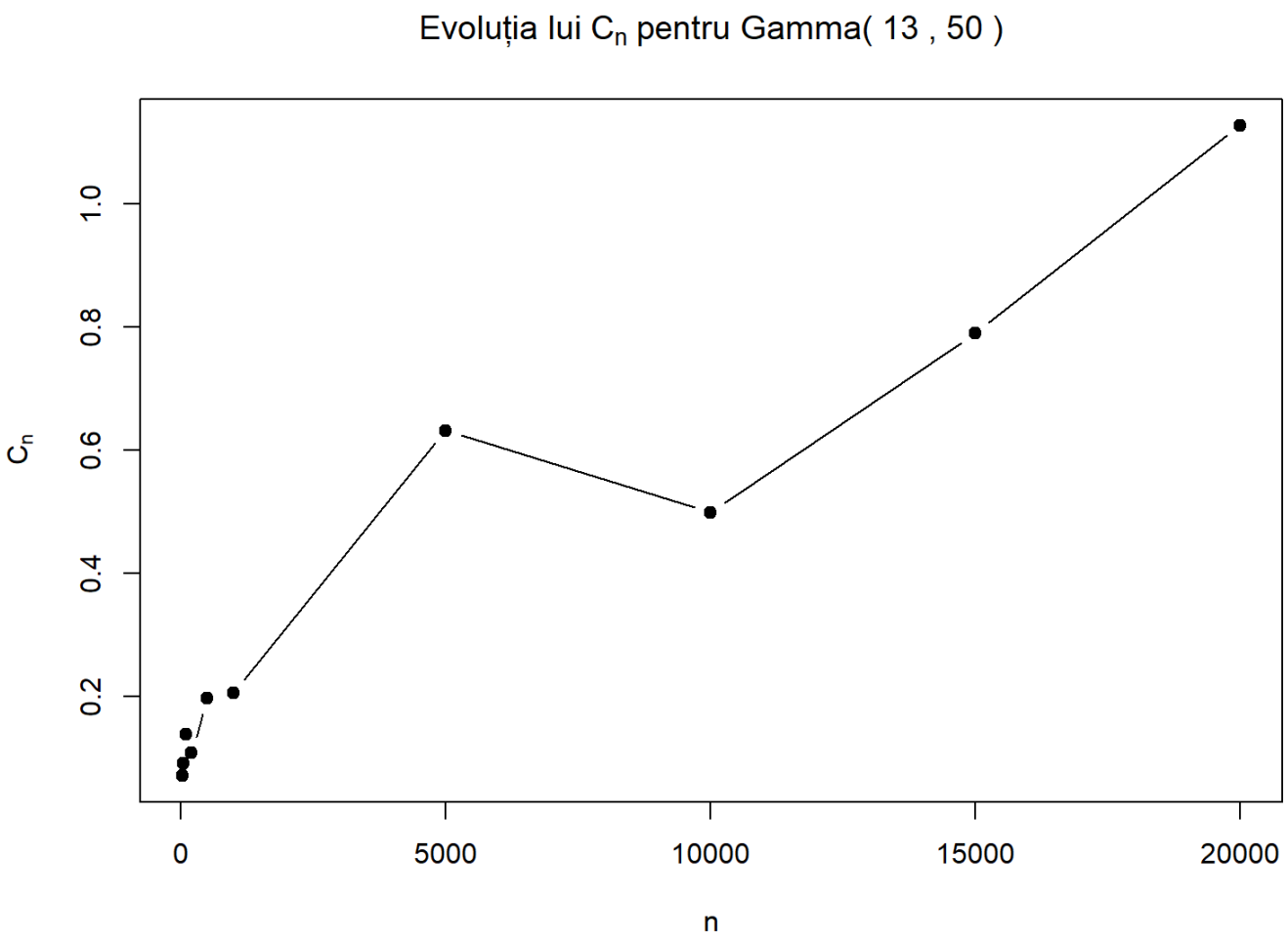
```
##        n        C_n
## 1     30 0.07087928
## 2     50 0.09118935
## 3    100 0.13813670
## 4    200 0.10809651
## 5    500 0.19751625
## 6   1000 0.20588802
## 7   5000 0.63218277
## 8  10000 0.49884288
## 9  15000 0.78974687
## 10 20000 1.12843252
```

```r
alfa_valori <- seq(1, 1000, length.out = 40)
beta_valori <- seq(1, 1000, length.out = 40)
grid_param <- expand.grid(
  alfa = alfa_valori,
  beta = beta_valori
)
n <- 100
grid_param$C_n <- mapply(
  function(a, b) {
    calculeaza_Cn(
      n = n,
      alfa = a,
      beta = b,
      M = 2000
    )
  },
  grid_param$alfa,
  grid_param$beta
)

alfa_valori <- seq(1, 500, length.out = 100)
valori_Cn_alfa <- sapply(alfa_valori, function(alfa) calculeaza_Cn(n, alfa, beta))
data.frame(
  alfa = alfa_valori,
  C_n = valori_Cn_alfa
)
```

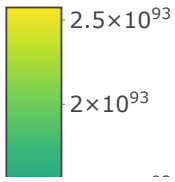
```
##           alfa         C_n
## 1     1.000000   0.08043544
## 2     6.040404   0.06051485
## 3    11.080808   0.04984640
## 4    16.121212   0.07479074
## 5    21.161616   0.09003921
## 6    26.202020   0.09375805
## 7    31.242424   0.08930008
## 8    36.282828   0.04740004
## 9    41.323232   0.07279888
## 10   46.363636   0.08507662
## 11   51.404040   0.05008260
## 12   56.444444   0.08146796
## 13   61.484848   0.07098432
## 14   66.525253   0.09790577
## 15   71.565657   0.09566518
## 16   76.606061   0.04816924
## 17   81.646465   0.08087552
## 18   86.686869   0.09527685
## 19   91.727273   0.07794684
## 20   96.767677   0.08856194
## 21  101.808081   0.06863526
## 22  106.848485   0.06882901
## 23  111.888889   0.09720547
## 24  116.929293   0.05707490
## 25  121.969697   0.05447137
## 26  127.010101   0.06299708
## 27  132.050505   0.11184275
## 28  137.090909   0.05935068
## 29  142.131313   0.05133385
## 30  147.171717   0.15785447
## 31  152.212121   0.11061042
## 32  157.252525   0.06202051
## 33  162.292929   0.09392066
## 34  167.333333   0.07926920
## 35  172.373737   0.07948277
## 36  177.414141   0.06419810
## 37  182.454545   0.05869789
## 38  187.494949   0.10349861
## 39  192.535354   0.09598539
## 40  197.575758   0.05469073
## 41  202.616162   0.06027633
## 42  207.656566   0.08425246
## 43  212.696970   0.06174686
## 44  217.737374   0.09307066
## 45  222.777778   0.12513034
## 46  227.818182   0.12920744
## 47  232.858586   0.13039033
## 48  237.898990   0.06564062
## 49  242.939394   0.09219516
## 50  247.979798   0.05336055
## 51  253.020202   0.05552815
## 52  258.060606   0.08250756
## 53  263.101010   0.07265685
## 54  268.141414   0.04407457
## 55  273.181818   0.06713292
## 56  278.222222   0.08647493
## 57  283.262626   0.04708918
## 58  288.303030   0.09862209
## 59  293.343434   0.08339346
```

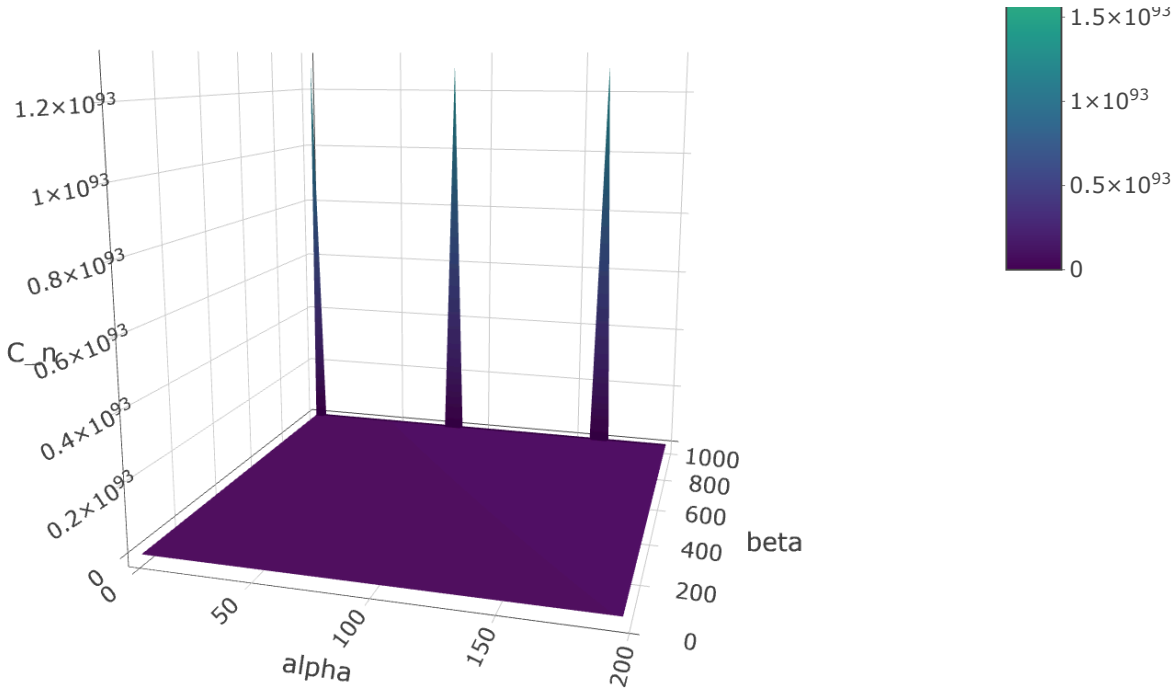
```
## 60  298.383838   0.05977988
## 61  303.424242   0.09268780
## 62  308.464646   0.05264844
## 63  313.505051   0.06067534
## 64  318.545455   0.10944900
## 65  323.585859   0.09179579
## 66  328.626263   0.07567116
## 67  333.666667   0.12319966
## 68  338.707071   0.15222700
## 69  343.747475   0.03502932
## 70  348.787879   0.10119909
## 71  353.828283   0.07857382
## 72  358.868687   0.05253916
## 73  363.909091   0.09348992
## 74  368.949495   0.06487010
## 75  373.989899   0.06939370
## 76  379.030303   0.06128583
## 77  384.070707   0.05815952
## 78  389.111111   0.05931313
## 79  394.151515   0.05854360
## 80  399.191919   0.08091317
## 81  404.232323   0.09399815
## 82  409.272727   0.11207093
## 83  414.313131   0.07065764
## 84  419.353535   0.10125883
## 85  424.393939   0.05809896
## 86  429.434343   0.07827186
## 87  434.474747 156.71677424
## 88  439.515152   0.03587561
## 89  444.555556   0.06751358
## 90  449.595960   0.07512052
## 91  454.636364 716.15239812
## 92  459.676768   0.06435092
## 93  464.717172   0.12936319
## 94  469.757576   0.08828353
## 95  474.797980   0.05434580
## 96  479.838384   0.06770360
## 97  484.878788   0.07129267
## 98  489.919192   0.05793474
## 99  494.959596   0.06925476
## 100 500.000000   0.08200270
```

```
# d)
plot(
  n_valori, valori_Cn_n,
  type = "b",
  pch = 19,
  xlab = "n",
  ylab = expression(C[n]),
  main = bquote("Evoluția lui" ~ C[n] ~ "pentru Gamma("~.(alfa)~","~.(beta)~")")
)
```

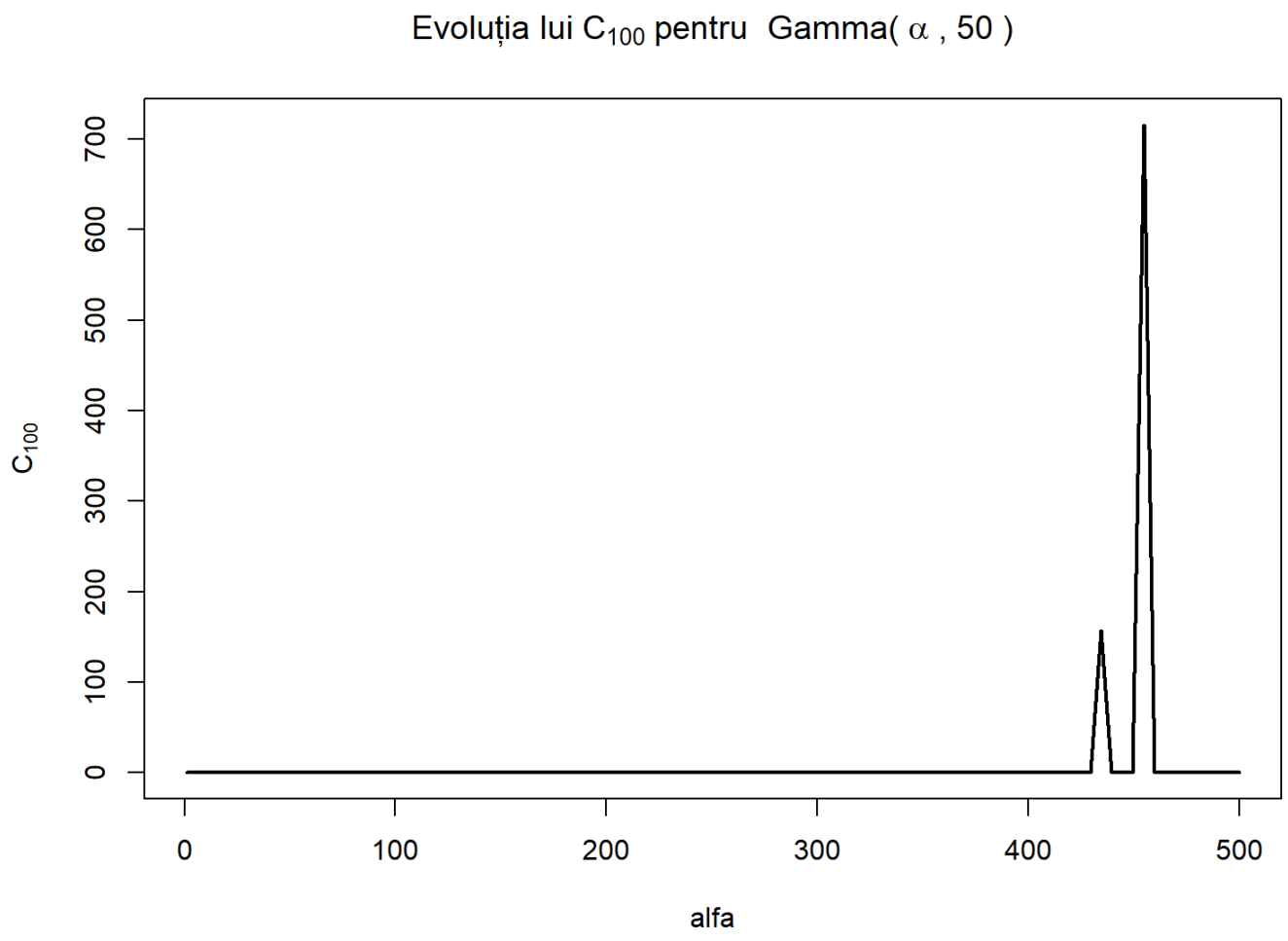



```
Cn_matrix <- matrix(
  grid_param$C_n,
  nrow = length(alfa_valori),
  ncol = length(beta_valori),
  byrow = FALSE
)
library(plotly)

plot_ly(
  x = alfa_valori,
  y = beta_valori,
  z = Cn_matrix,
  type = "surface"
) %>%
  layout(
    title = bquote("C"[.(n)] ~ "pentru Gamma("~alpha~","~beta~")"),
    scene = list(
      xaxis = list(title = "alpha"),
      yaxis = list(title = "beta"),
      zaxis = list(title = "C_n")
    )
  )
```






```
plot(
  alfa_valori, valori_Cn_alfa,
  type = "l",
  lwd = 2,
  xlab = bquote(alfa),
  ylab = bquote(C[.(n)]),
  main = bquote("Evoluția lui" ~ C[.(n)] ~ "pentru  Gamma("~alpha~","~.(beta)~")")
)
```


Evoluția lui C_{100} pentru Gamma(α , 50)
C_{100}
0
100
200
300
400
500
600
700
0
100
200
300
400
500
alfa

5.Distribuția Geometrică

```
# Problema 7

# Geometrică de parametru p - singurul inconvenient este că suportul este \mathbb{N}

# Funcție ce calculează rho = E[|X-mu|^3]
rho_geom <- function(p) {
  mu <- (1 - p) / p
  k <- 0:10^6

  sum(abs(k - mu)^3 * dgeom(k, prob = p))
}

# Funcție ce calculează marginea Berry-Esseen, dată de inegalitate
margine_geom <- function(n, p, C = 0.4748) {
  mu <- (1 - p) / p
  sigma <- sqrt(1-p) / p
  rho <- rho_geom(p)

  C * rho / (sigma^3 * sqrt(n))
}

# Funcție ce aproximează convenabil supremumul din inegalitatea Berry-Esseen
sup_geom <- function(n, p,
                     M = 1000000,
                     grid_size = 2000,
                     q_low = 1e-4,
                     q_high = 1 - 1e-4) {
  mu <- (1 - p) / p
  sigma <- sqrt(1-p) / p

  X <- matrix(
    rgeom(M * n, prob = p),
    nrow = M,
    ncol = n
  )
  X_bar <- rowMeans(X)

  Z <- sqrt(n) * (X_bar - mu) / sigma

  probs <- seq(q_low, q_high, length.out = grid_size)
  x_grid <- qnorm(probs)

  cdf_empirica <- sapply(
    x_grid,
    function(x) mean(Z <= x)
  )
  cdf_normala <- pnorm(x_grid)

  max(abs(cdf_empirica - cdf_normala))
}

# Funcție ce construiește dataframe-ul cu marginile Berry-Esseen vs. volumul eșantionului
dataframe_geom <- function(n_valori, p, C = 0.4748) {
  margini <- sapply(
    n_valori,
    function(n) margine_geom(n, p, C)
  )
```

```r
  data.frame(
    n = n_valori,
    margine_BE = margini
  )
}

# Funcție ce reprezintă dependența x vs.
plot_cdf_diferenta_geom <- function(n, p,
                                    M = 1000000,
                                    grid_size = 2000,
                                    q_low = 1e-4,
                                    q_high = 1 - 1e-4) {
  mu <- (1 - p) / p
  sigma <- sqrt(1-p) / p

  X <- matrix(
    rgeom(M * n, prob = p),
    nrow = M,
    ncol = n
  )
  X_bar <- rowMeans(X)

  Z <- sqrt(n) * (X_bar - mu) / sigma

  probs <- seq(q_low, q_high, length.out = grid_size)
  x_grid <- qnorm(probs)

  cdf_empirica <- sapply(
    x_grid,
    function(x) mean(Z <= x)
  )
  cdf_normala <- pnorm(x_grid)

  diff_abs <- abs(cdf_empirica - cdf_normala)

  plot(
    x_grid, diff_abs,
    type = "l",
    lwd = 2,
    col = "blue",
    xlab = expression(x),
    ylab = bquote(abs(F[.(n)](x) - Phi(x))),
    main = bquote("|" * F[.(n)](x) - Phi(x) * "|" ~~ "pentru Geom("~.(p)~")")
  )

  max_idx <- which.max(diff_abs)
  points(x_grid[max_idx], diff_abs[max_idx], col = "red", pch = 19)

  invisible(
    list(
      x = x_grid,
      diff = diff_abs,
      sup_numeric = max(diff_abs)
    )
  )
}

# Parametri
n <- 300   #volumul eșantionului
p <- 0.75    #Geom(p)
n_valori <- c(10, 20, 50, 100, 200, 500, 1000, 5000, 10000, 50000, 100000, 500000, 1000000)
```

```
# Linii de cod pentru execuție
rho_geom(p)
```

```
## [1] 0.7962963
```

```
margine_geom(n, p)
```

```
## [1] 0.07367134
```

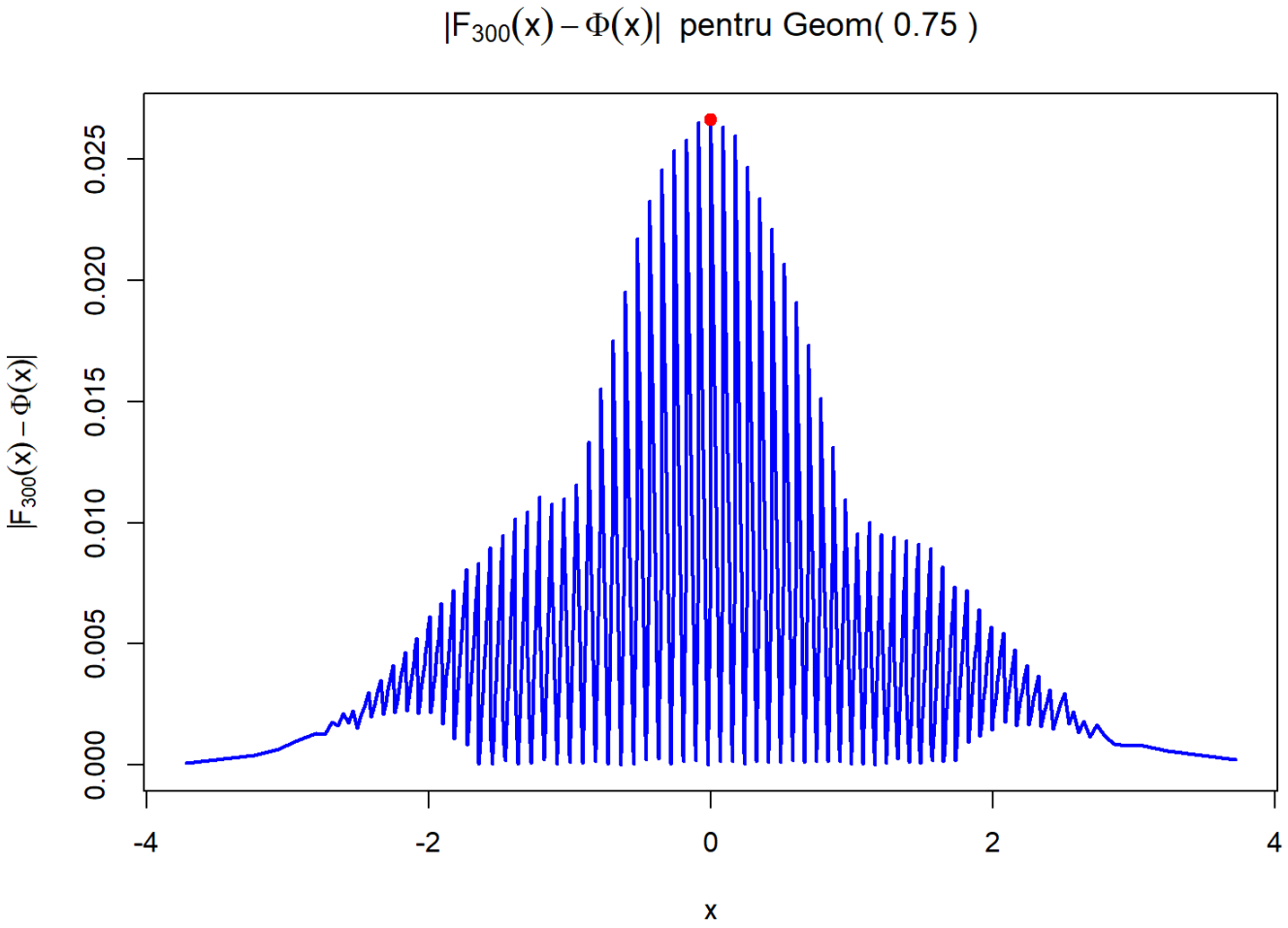
```
sup_geom(n, p)
```

```
## [1] 0.02688592
```

```
dataframe_geom(n_valori, p)
```

```
##        n  margine_BE
## 1  1e+01 0.403514535
## 2  2e+01 0.285327864
## 3  5e+01 0.180457186
## 4  1e+02 0.127602500
## 5  2e+02 0.090228593
## 6  5e+02 0.057065573
## 7  1e+03 0.040351454
## 8  5e+03 0.018045719
## 9  1e+04 0.012760250
## 10 5e+04 0.005706557
## 11 1e+05 0.004035145
## 12 5e+05 0.001804572
## 13 1e+06 0.001276025
```

```
plot_cdf_diferenta_geom(n, p)
```




```
# Exercițiul 8
#a) definim C_n
C_n <- sup_geom(n, p) / margine_geom(n, p, C = 1)

#b) folosim deja metoda Monte-Carlo în funcția sup_binom
n_valori <- c(30, 50, 100, 200, 500)

calculeaza_Cn <- function(n, p, M = 5000, grid_size = 2000, q_low = 1e-4, q_high = 1 - 1e-4, C = 1) {
  sup_geom(n, p, M, grid_size, q_low, q_high) / margine_geom(n, p, C)
}
valori_Cn_n <- sapply(n_valori, function(n) calculeaza_Cn(n, p))
data.frame(
  n = n_valori,
  C_n = valori_Cn_n
)
```

```
##     n       C_n
## 1  30 0.1559455
## 2  50 0.1520545
## 3 100 0.2007662
## 4 200 0.1765019
## 5 500 0.2196076
```

```r
#c) variem, pe rând, n (păstrând p constant), apoi p (păstrând n constant)
n_valori <- c(30, 50, 100, 200, 500, 1000, 5000, 10000, 15000, 20000)
valori_Cn_n <- sapply(n_valori, function(n) calculeaza_Cn(n, p))
data.frame(
  n = n_valori,
  C_n = valori_Cn_n
)
```

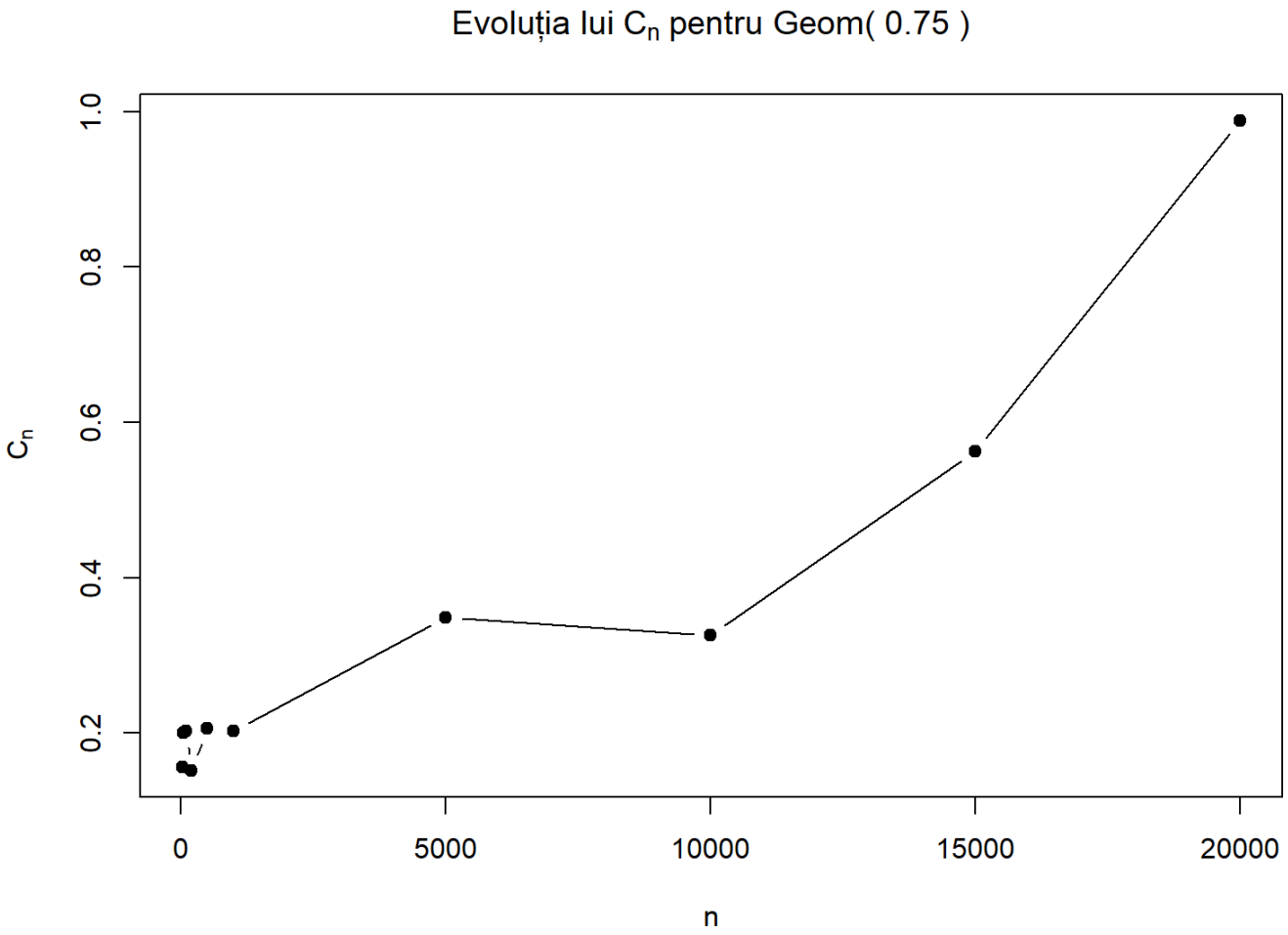
```
##        n       C_n
## 1     30 0.1567608
## 2     50 0.1998757
## 3    100 0.2021876
## 4    200 0.1516598
## 5    500 0.2064717
## 6   1000 0.2022981
## 7   5000 0.3490406
## 8  10000 0.3262785
## 9  15000 0.5624585
## 10 20000 0.9890258
```

```r
p_valori <- seq(1e-6, 1 - 1e-6, length.out = 100)
valori_Cn_p <- sapply(p_valori, function(p) calculeaza_Cn(n, p))
data.frame(
  p = p_valori,
  C_n = valori_Cn_p
)
```

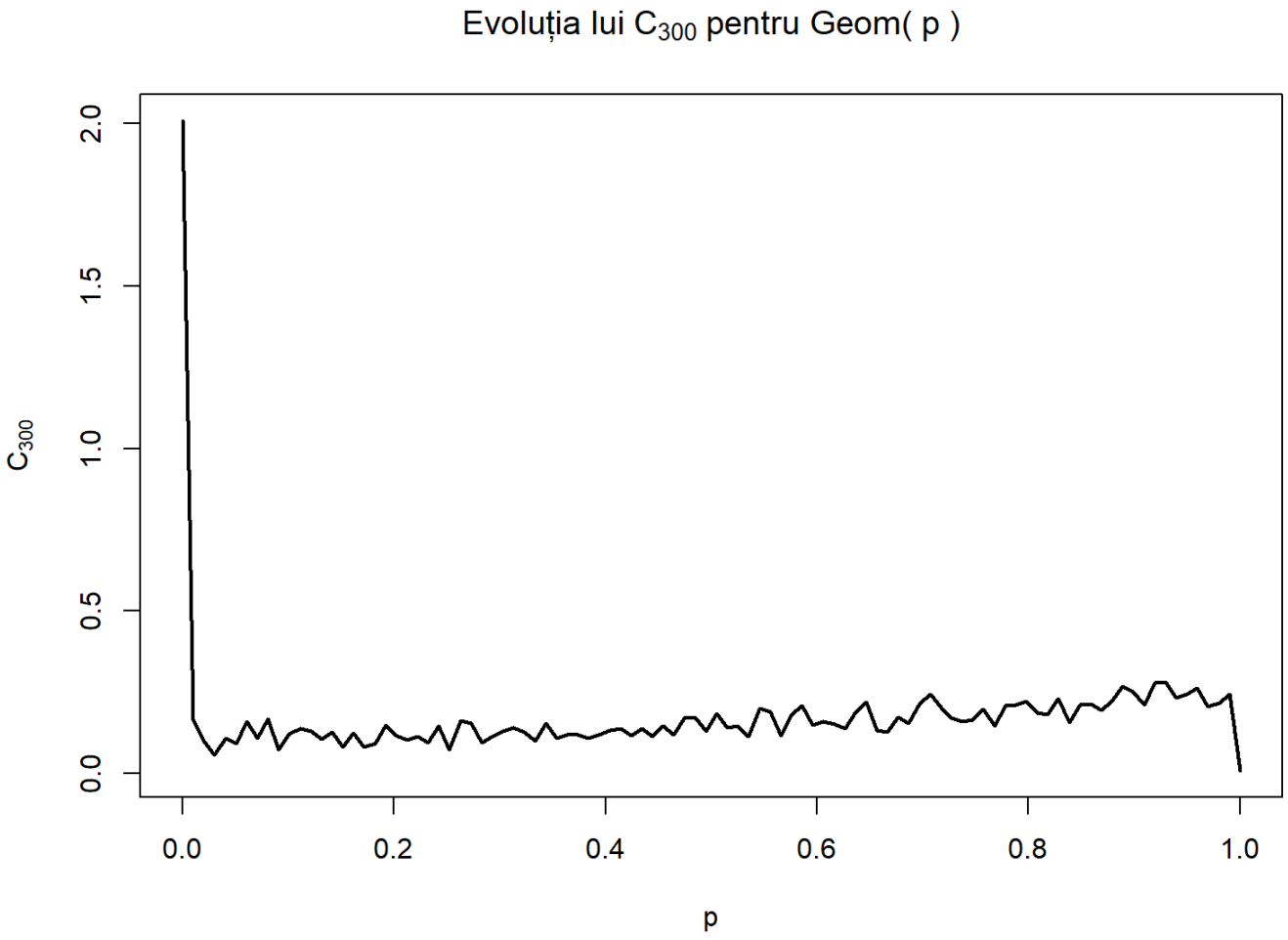
```
##              p         C_n
## 1   0.00000100 2.009352916
## 2   0.01010199 0.166726955
## 3   0.02020298 0.099841442
## 4   0.03030397 0.056775448
## 5   0.04040496 0.110052181
## 6   0.05050595 0.091560725
## 7   0.06060694 0.160022670
## 8   0.07070793 0.110180941
## 9   0.08080892 0.167433365
## 10  0.09090991 0.072582542
## 11  0.10101090 0.121426203
## 12  0.11111189 0.139724372
## 13  0.12121288 0.131002282
## 14  0.13131387 0.106699651
## 15  0.14141486 0.128301217
## 16  0.15151585 0.082296996
## 17  0.16161684 0.126019122
## 18  0.17171783 0.082614314
## 19  0.18181882 0.092099740
## 20  0.19191981 0.148464779
## 21  0.20202080 0.116242554
## 22  0.21212179 0.104102985
## 23  0.22222278 0.115300042
## 24  0.23232377 0.096450889
## 25  0.24242476 0.146188099
## 26  0.25252575 0.073191434
## 27  0.26262674 0.162395749
## 28  0.27272773 0.155697588
## 29  0.28282872 0.096274367
## 30  0.29292971 0.113865488
## 31  0.30303070 0.129364605
## 32  0.31313169 0.141569727
## 33  0.32323268 0.127903885
## 34  0.33333367 0.101408324
## 35  0.34343466 0.156393570
## 36  0.35353565 0.108836107
## 37  0.36363664 0.119593281
## 38  0.37373763 0.121241008
## 39  0.38383862 0.108482664
## 40  0.39393961 0.120923037
## 41  0.40404060 0.134609245
## 42  0.41414159 0.139781672
## 43  0.42424258 0.117706370
## 44  0.43434357 0.138659665
## 45  0.44444456 0.113413684
## 46  0.45454555 0.146938493
## 47  0.46464654 0.120327180
## 48  0.47474753 0.173025172
## 49  0.48484852 0.170901368
## 50  0.49494951 0.131768556
## 51  0.50505049 0.183543742
## 52  0.51515148 0.140831800
## 53  0.52525247 0.147590617
## 54  0.53535346 0.115755726
## 55  0.54545445 0.201157560
## 56  0.55555544 0.190911940
## 57  0.56565643 0.116985733
## 58  0.57575742 0.180992767
## 59  0.58585841 0.207948739
```

```
## 60  0.59595940 0.148607900
## 61  0.60606039 0.159855332
## 62  0.61616138 0.153187623
## 63  0.62626237 0.138310557
## 64  0.63636336 0.189655896
## 65  0.64646435 0.219312091
## 66  0.65656534 0.132257316
## 67  0.66666633 0.127183320
## 68  0.67676732 0.174030750
## 69  0.68686831 0.154872729
## 70  0.69696930 0.218402974
## 71  0.70707029 0.244153619
## 72  0.71717128 0.203704352
## 73  0.72727227 0.172629938
## 74  0.73737326 0.159644705
## 75  0.74747425 0.165228969
## 76  0.75757524 0.198031800
## 77  0.76767623 0.147708832
## 78  0.77777722 0.209739881
## 79  0.78787821 0.213007151
## 80  0.79797920 0.223318671
## 81  0.80808019 0.186541872
## 82  0.81818118 0.183332124
## 83  0.82828217 0.230471673
## 84  0.83838316 0.156486412
## 85  0.84848415 0.212872592
## 86  0.85858514 0.214476444
## 87  0.86868613 0.195325775
## 88  0.87878712 0.223729935
## 89  0.88888811 0.269344159
## 90  0.89898910 0.253677883
## 91  0.90909009 0.212258472
## 92  0.91919108 0.278252889
## 93  0.92929207 0.281227323
## 94  0.93939306 0.233109609
## 95  0.94949405 0.243981481
## 96  0.95959504 0.262805217
## 97  0.96969603 0.205672065
## 98  0.97979702 0.217746071
## 99  0.98989801 0.245291258
## 100 0.99999900 0.008777194
```

```
# d)
plot(
  n_valori, valori_Cn_n,
  type = "b",
  pch = 19,
  xlab = "n",
  ylab = expression(C[n]),
  main = bquote("Evoluția lui" ~ C[n] ~ "pentru Geom("~.(p)~")")
)
```




```
plot(
  p_valori, valori_Cn_p,
  type = "l",
  lwd = 2,
  xlab = "p",
  ylab = bquote(C[.(n)]),
  main = bquote("Evoluția lui" ~ C[.(n)] ~ "pentru Geom("~ p ~")")
)
```


Evoluția lui C_{300} pentru Geom(p)
C_{300}
0.0
0.5
1.0
1.5
2.0
0.0
0.2
0.4
0.6
0.8
1.0
p

6.Distribuția Poisson

```
# Problema 7

# Poisson de parametru lambda - singurul inconvenient este că suportul este \mathbb{N}

# Funcție ce calculează rho = E[|X-mu|^3]
rho_poisson <- function(lambda) {
  mu <- lambda
  k <- 0:10^6

  sum(abs(k - mu)^3 * dpois(k, lambda = lambda))
}

# Funcție ce calculează marginea Berry-Esseen, dată de inegalitate
margine_poisson <- function(n, lambda, C = 0.4748) {
  mu <- lambda
  sigma <- sqrt(lambda)
  rho <- rho_poisson(lambda)

  C * rho / (sigma^3 * sqrt(n))
}

# Funcție ce aproximează convenabil supremumul din inegalitatea Berry-Esseen
sup_poisson <- function(n, lambda,
                        M = 1000000,
                        grid_size = 2000,
                        q_low = 1e-4,
                        q_high = 1 - 1e-4) {
  mu <- lambda
  sigma <- sqrt(lambda)

  X <- matrix(
    rpois(M * n, lambda = lambda),
    nrow = M,
    ncol = n
  )
  X_bar <- rowMeans(X)

  Z <- sqrt(n) * (X_bar - mu) / sigma

  probs <- seq(q_low, q_high, length.out = grid_size)
  x_grid <- qnorm(probs)

  cdf_empirica <- sapply(
    x_grid,
    function(x) mean(Z <= x)
  )
  cdf_normala <- pnorm(x_grid)

  max(abs(cdf_empirica - cdf_normala))
}

# Funcție ce construiește dataframe-ul cu marginile Berry-Esseen vs. volumul eșantionului
dataframe_poisson <- function(n_valori, lambda, C = 0.4748) {
  margini <- sapply(
    n_valori,
    function(n) margine_poisson(n, lambda, C)
  )
```

```r
  data.frame(
    n = n_valori,
    margine_BE = margini
  )
}

# Funcție ce reprezintă dependența x vs.
plot_cdf_diferenta_poisson <- function(n, lambda,
                                       M = 1000000,
                                       grid_size = 2000,
                                       q_low = 1e-4,
                                       q_high = 1 - 1e-4) {
  mu <- lambda
  sigma <- sqrt(lambda)

  X <- matrix(
    rpois(M * n, lambda = lambda),
    nrow = M,
    ncol = n
  )
  X_bar <- rowMeans(X)

  Z <- sqrt(n) * (X_bar - mu) / sigma

  probs <- seq(q_low, q_high, length.out = grid_size)
  x_grid <- qnorm(probs)

  cdf_empirica <- sapply(
    x_grid,
    function(x) mean(Z <= x)
  )
  cdf_normala <- pnorm(x_grid)

  diff_abs <- abs(cdf_empirica - cdf_normala)

  plot(
    x_grid, diff_abs,
    type = "l",
    lwd = 2,
    col = "blue",
    xlab = expression(x),
    ylab = bquote(abs(F[.(n)](x) - Phi(x))),
    main = bquote("|" * F[.(n)](x) - Phi(x) * "|" ~~ "pentru Pois("~.(lambda)~")")
  )

  max_idx <- which.max(diff_abs)
  points(x_grid[max_idx], diff_abs[max_idx], col = "red", pch = 19)

  invisible(
    list(
      x = x_grid,
      diff = diff_abs,
      sup_numeric = max(diff_abs)
    )
  )
}

# Parametri
n <- 300   #volumul eșantionului
lambda <- 13    #Poisson(lambda)
n_valori <- c(10, 20, 50, 100, 200, 500, 1000, 5000, 10000, 50000, 100000, 500000, 1000000)
```

```
# Linii de cod pentru execuție
rho_poisson(lambda)
```

```
## [1] 75.27859
```

```
margine_poisson(n, lambda)
```

```
## [1] 0.04402573
```

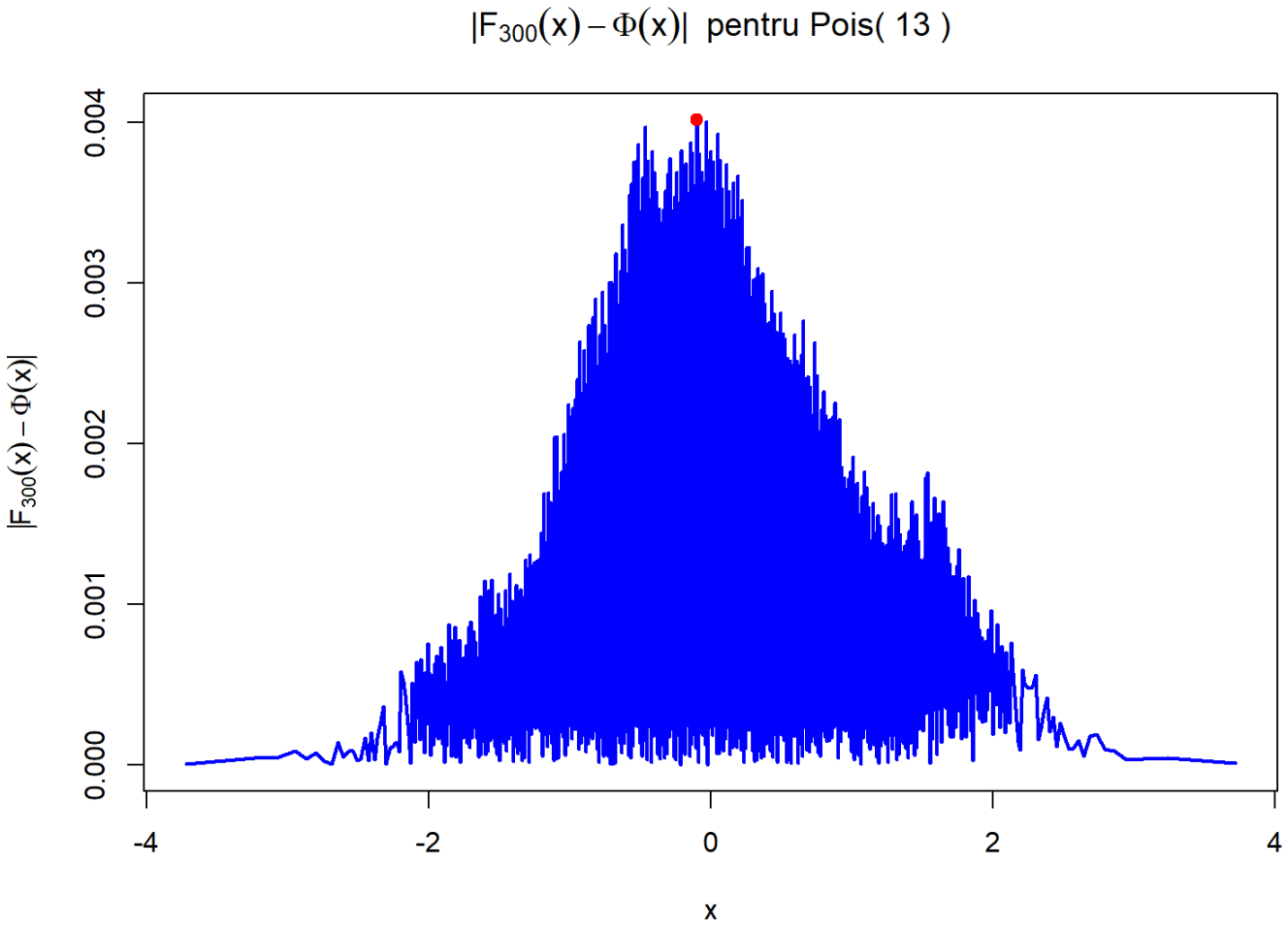
```
sup_poisson(n, lambda)
```

```
## [1] 0.003520481
```

```
dataframe_poisson(n_valori, lambda)
```

```
##        n  margine_BE
## 1  1e+01 0.241138838
## 2  2e+01 0.170510907
## 3  5e+01 0.107840567
## 4  1e+02 0.076254796
## 5  2e+02 0.053920283
## 6  5e+02 0.034102181
## 7  1e+03 0.024113884
## 8  5e+03 0.010784057
## 9  1e+04 0.007625480
## 10 5e+04 0.003410218
## 11 1e+05 0.002411388
## 12 5e+05 0.001078406
## 13 1e+06 0.000762548
```

```
plot_cdf_diferenta_poisson(n, lambda)
```




```
# Exercițiul 8
#a) definim C_n
C_n <- sup_poisson(n, lambda) / margine_poisson(n, lambda, C = 1)

#b) folosim deja metoda Monte-Carlo în funcția sup_binom
n_valori <- c(30, 50, 100, 200, 500)

calculeaza_Cn <- function(n, lambda, M = 5000, grid_size = 2000, q_low = 1e-4, q_high = 1 - 1e-4, C =
1) {
  sup_poisson(n, lambda, M, grid_size, q_low, q_high) / margine_poisson(n, lambda, C)
}
valori_Cn_n <- sapply(n_valori, function(n) calculeaza_Cn(n, lambda))
data.frame(
  n = n_valori,
  C_n = valori_Cn_n
)
```

```
##     n        C_n
## 1  30 0.05052173
## 2  50 0.07551643
## 3 100 0.13046513
## 4 200 0.14149075
## 5 500 0.19841299
```

```r
#c) variem, pe rând, n (păstrând lambda constant), apoi lambda (păstrând n constant)
n_valori <- c(30, 50, 100, 200, 500, 1000, 5000, 10000, 15000, 20000)
valori_Cn_n <- sapply(n_valori, function(n) calculeaza_Cn(n, lambda))
data.frame(
  n = n_valori,
  C_n = valori_Cn_n
)
```

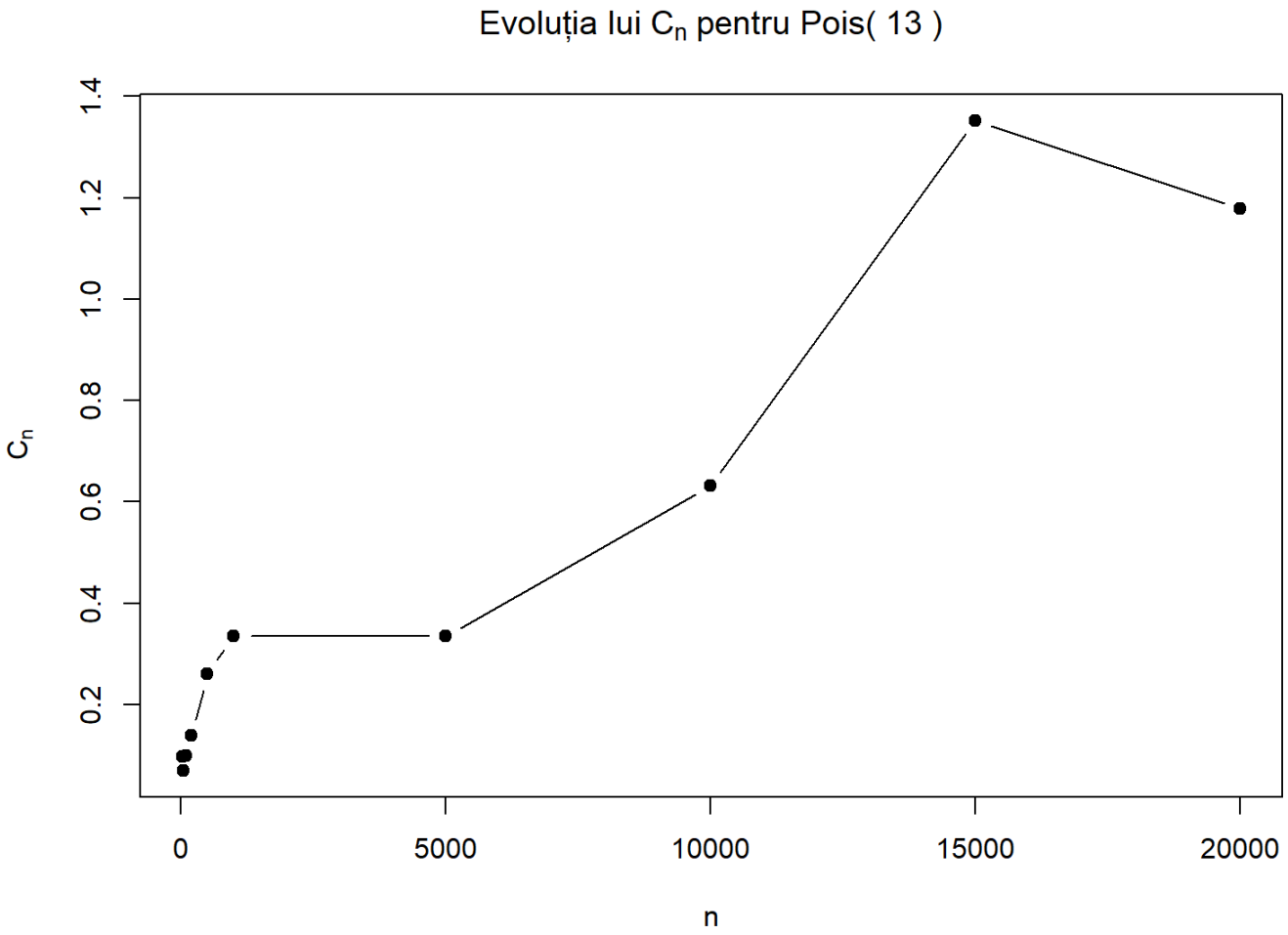
```
##        n        C_n
## 1     30 0.09676622
## 2     50 0.06948643
## 3    100 0.09941489
## 4    200 0.13881163
## 5    500 0.26044669
## 6   1000 0.33437472
## 7   5000 0.33457064
## 8  10000 0.63104376
## 9  15000 1.35271573
## 10 20000 1.17846480
```

```r
lambda_valori <- seq(1e-6, 1e+3, length.out = 100)
valori_Cn_lambda <- sapply(lambda_valori, function(lambda) calculeaza_Cn(n, lambda))
data.frame(
  lambda = lambda_valori,
  C_n = valori_Cn_lambda
)
```

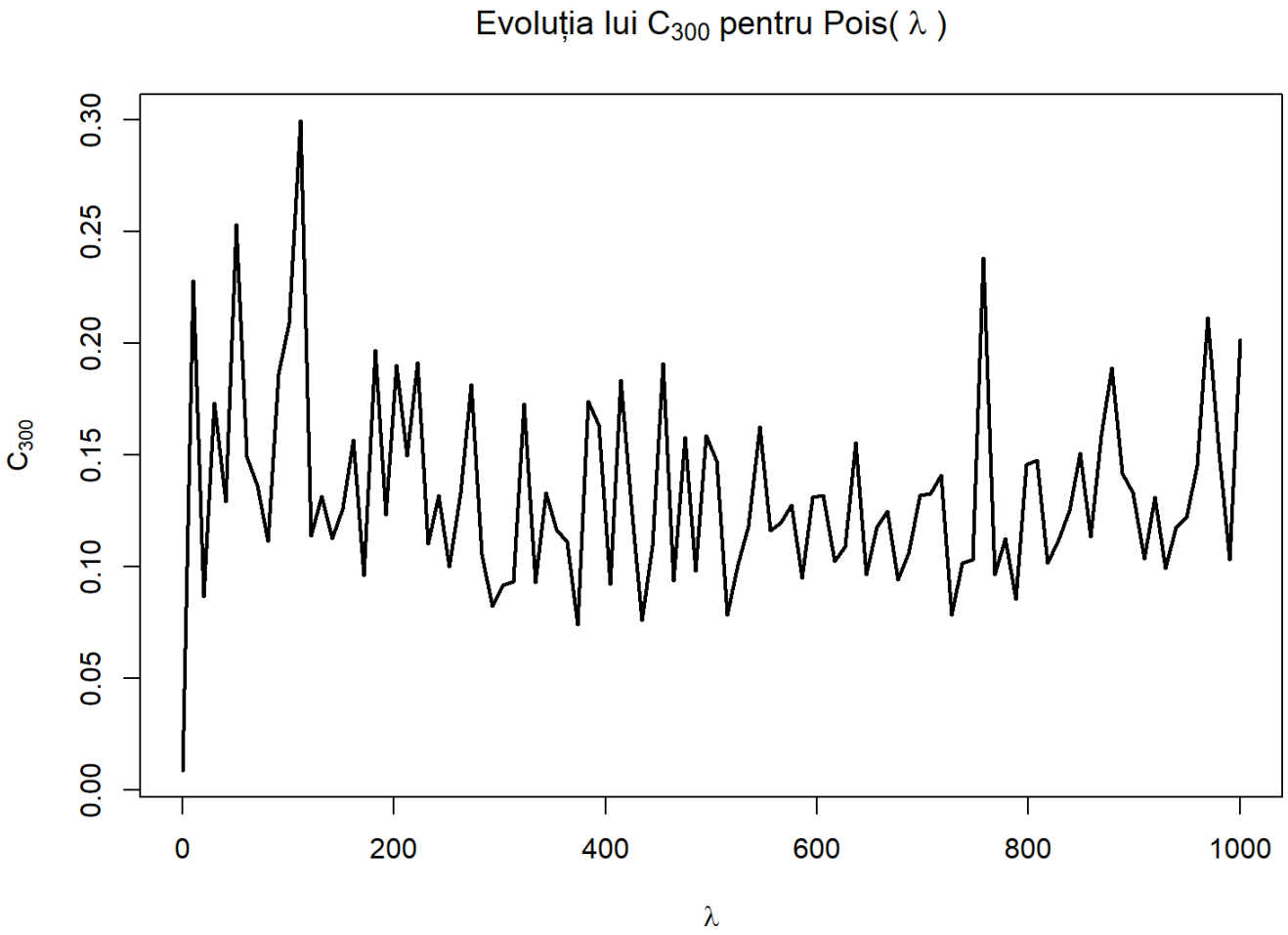
```
##          lambda         C_n
## 1      0.000001 0.008773738
## 2     10.101011 0.228010525
## 3     20.202021 0.086584953
## 4     30.303031 0.173305752
## 5     40.404041 0.129317678
## 6     50.505051 0.253412589
## 7     60.606062 0.149170518
## 8     70.707072 0.136476562
## 9     80.808082 0.111669266
## 10    90.909092 0.186235439
## 11   101.010102 0.209262050
## 12   111.111112 0.299892696
## 13   121.212122 0.113712004
## 14   131.313132 0.131563984
## 15   141.414142 0.112656335
## 16   151.515152 0.126009454
## 17   161.616162 0.156770341
## 18   171.717173 0.096362879
## 19   181.818183 0.197127542
## 20   191.919193 0.123531760
## 21   202.020203 0.190243656
## 22   212.121213 0.149846578
## 23   222.222223 0.191362396
## 24   232.323233 0.110507922
## 25   242.424243 0.132138507
## 26   252.525253 0.100226246
## 27   262.626263 0.133314446
## 28   272.727273 0.181461866
## 29   282.828284 0.105800123
## 30   292.929294 0.082560986
## 31   303.030304 0.091873357
## 32   313.131314 0.093267070
## 33   323.232324 0.173079421
## 34   333.333334 0.093091613
## 35   343.434344 0.133175657
## 36   353.535354 0.116835876
## 37   363.636364 0.111032508
## 38   373.737374 0.074264784
## 39   383.838384 0.174186286
## 40   393.939395 0.163628674
## 41   404.040405 0.092286819
## 42   414.141415 0.183613158
## 43   424.242425 0.127871494
## 44   434.343435 0.076168219
## 45   444.444445 0.109620241
## 46   454.545455 0.191027748
## 47   464.646465 0.093900558
## 48   474.747475 0.157913730
## 49   484.848485 0.098299664
## 50   494.949495 0.158781296
## 51   505.050506 0.147456209
## 52   515.151516 0.078299703
## 53   525.252526 0.101093572
## 54   535.353536 0.118066723
## 55   545.454546 0.162551429
## 56   555.555556 0.116295395
## 57   565.656566 0.119789762
## 58   575.757576 0.127791240
## 59   585.858586 0.095177672
```

```
## 60   595.959596 0.131039473
## 61   606.060606 0.131885613
## 62   616.161617 0.102680116
## 63   626.262627 0.109042045
## 64   636.363637 0.155730705
## 65   646.464647 0.096403731
## 66   656.565657 0.117807350
## 67   666.666667 0.124822989
## 68   676.767677 0.094282611
## 69   686.868687 0.106168092
## 70   696.969697 0.131857576
## 71   707.070707 0.132873040
## 72   717.171717 0.141079878
## 73   727.272728 0.078408723
## 74   737.373738 0.101603093
## 75   747.474748 0.103322107
## 76   757.575758 0.238300133
## 77   767.676768 0.096421979
## 78   777.777778 0.112898477
## 79   787.878788 0.085531372
## 80   797.979798 0.145868969
## 81   808.080808 0.147611921
## 82   818.181818 0.101631372
## 83   828.282828 0.111536491
## 84   838.383839 0.125168223
## 85   848.484849 0.150938570
## 86   858.585859 0.113482712
## 87   868.686869 0.158779770
## 88   878.787879 0.189033671
## 89   888.888889 0.141729121
## 90   898.989899 0.133282489
## 91   909.090909 0.103756320
## 92   919.191919 0.131286975
## 93   929.292929 0.099195968
## 94   939.393939 0.117905234
## 95   949.494950 0.122093467
## 96   959.595960 0.145578356
## 97   969.696970 0.211627579
## 98   979.797980 0.150873233
## 99   989.898990 0.103129473
## 100 1000.000000 0.201649127
```

```
# d)
plot(
  n_valori, valori_Cn_n,
  type = "b",
  pch = 19,
  xlab = "n",
  ylab = expression(C[n]),
  main = bquote("Evoluția lui" ~ C[n] ~ "pentru Pois("~.(lambda)~")")
)
```




```
plot(
  lambda_valori, valori_Cn_lambda,
  type = "l",
  lwd = 2,
  xlab = bquote(lambda),
  ylab = bquote(C[.(n)]),
  main = bquote("Evoluția lui" ~ C[.(n)] ~ "pentru Pois("~ lambda ~")")
)
```


Evoluția lui C_{300} pentru Pois(λ)
C_{300}
0.00
0.05
0.10
0.15
0.20
0.25
0.30
0
200
400
600
800
1000
λ

7.Uniforma Discretă

```
# Problema 7

# Uniformă discretă pe mulțimea {a, a+1, ..., b-1, b}

# Funcție ce calculează rho = E[|X-mu|^3]
rho_dunif <- function(a, b) {
  mu <- (a + b) / 2
  k <- a:b
  p <- 1 / (b - a + 1)

  sum(abs(k - mu)^3 * p)
}

# Funcție ce calculează marginea Berry-Esseen, dată de inegalitate
margine_dunif <- function(n, a, b, C = 0.4748) {
  mu <- (a + b) / 2
  sigma <- sqrt(((b - a + 1)^2 - 1) / 12)
  rho <- rho_dunif(a, b)

  C * rho / (sigma^3 * sqrt(n))
}

# Funcție ce aproximează convenabil supremumul din inegalitatea Berry-Esseen
sup_dunif <- function(n, a, b,
                      M = 1000000,
                      grid_size = 2000,
                      q_low = 1e-4,
                      q_high = 1 - 1e-4) {
  mu <- (a + b) / 2
  sigma <- sqrt(((b - a + 1)^2 - 1) / 12)

  X <- matrix(
    sample(a:b, size = M * n, replace = TRUE),
    nrow = M,
    ncol = n
  )
  X_bar <- rowMeans(X)

  Z <- sqrt(n) * (X_bar - mu) / sigma

  probs <- seq(q_low, q_high, length.out = grid_size)
  x_grid <- qnorm(probs)

  cdf_empirica <- sapply(
    x_grid,
    function(x) mean(Z <= x)
  )
  cdf_normala <- pnorm(x_grid)

  max(abs(cdf_empirica - cdf_normala))
}

# Funcție ce construiește dataframe-ul cu marginile Berry-Esseen vs. volumul eșantionului
dataframe_dunif <- function(n_valori, a, b, C = 0.4748) {
  margini <- sapply(
    n_valori,
    function(n) margine_dunif(n, a, b, C)
  )
```

```r
  data.frame(
    n = n_valori,
    margine_BE = margini
  )
}

# Funcție ce reprezintă dependența x vs.
plot_cdf_diferenta_dunif <- function(n, a, b,
                                     M = 1000000,
                                     grid_size = 2000,
                                     q_low = 1e-4,
                                     q_high = 1 - 1e-4) {
  mu <- (a + b) / 2
  sigma <- sqrt(((b - a + 1)^2 - 1) / 12)

  X <- matrix(
    sample(a:b, size = M * n, replace = TRUE),
    nrow = M,
    ncol = n
  )
  X_bar <- rowMeans(X)

  Z <- sqrt(n) * (X_bar - mu) / sigma

  probs <- seq(q_low, q_high, length.out = grid_size)
  x_grid <- qnorm(probs)

  cdf_empirica <- sapply(
    x_grid,
    function(x) mean(Z <= x)
  )
  cdf_normala <- pnorm(x_grid)

  diff_abs <- abs(cdf_empirica - cdf_normala)

  plot(
    x_grid, diff_abs,
    type = "l",
    lwd = 2,
    col = "blue",
    xlab = expression(x),
    ylab = bquote(abs(F[.(n)](x) - Phi(x))),
    main = bquote("|" * F[.(n)](x) - Phi(x) * "|" ~~ "pentru DUnif("~.(a)~","~.(b)~")")
  )

  max_idx <- which.max(diff_abs)
  points(x_grid[max_idx], diff_abs[max_idx], col = "red", pch = 19)

  invisible(
    list(
      x = x_grid,
      diff = diff_abs,
      sup_numeric = max(diff_abs)
    )
  )
}

# Parametri
n <- 300   #volumul eșantionului
a <- 50    #Unif({a, a+1, ..., b-1, b})
```

```
b <- 123   #Unif({a, a+1, ..., b-1, b})
n_valori <- c(10, 20, 50, 100, 200, 500, 1000, 5000, 10000, 50000, 100000, 500000, 1000000)

# Linii de cod pentru execuție
rho_dunif(a, b)
```

```
## [1] 12658.62
```

```
margine_dunif(n, a, b)
```

```
## [1] 0.03560675
```

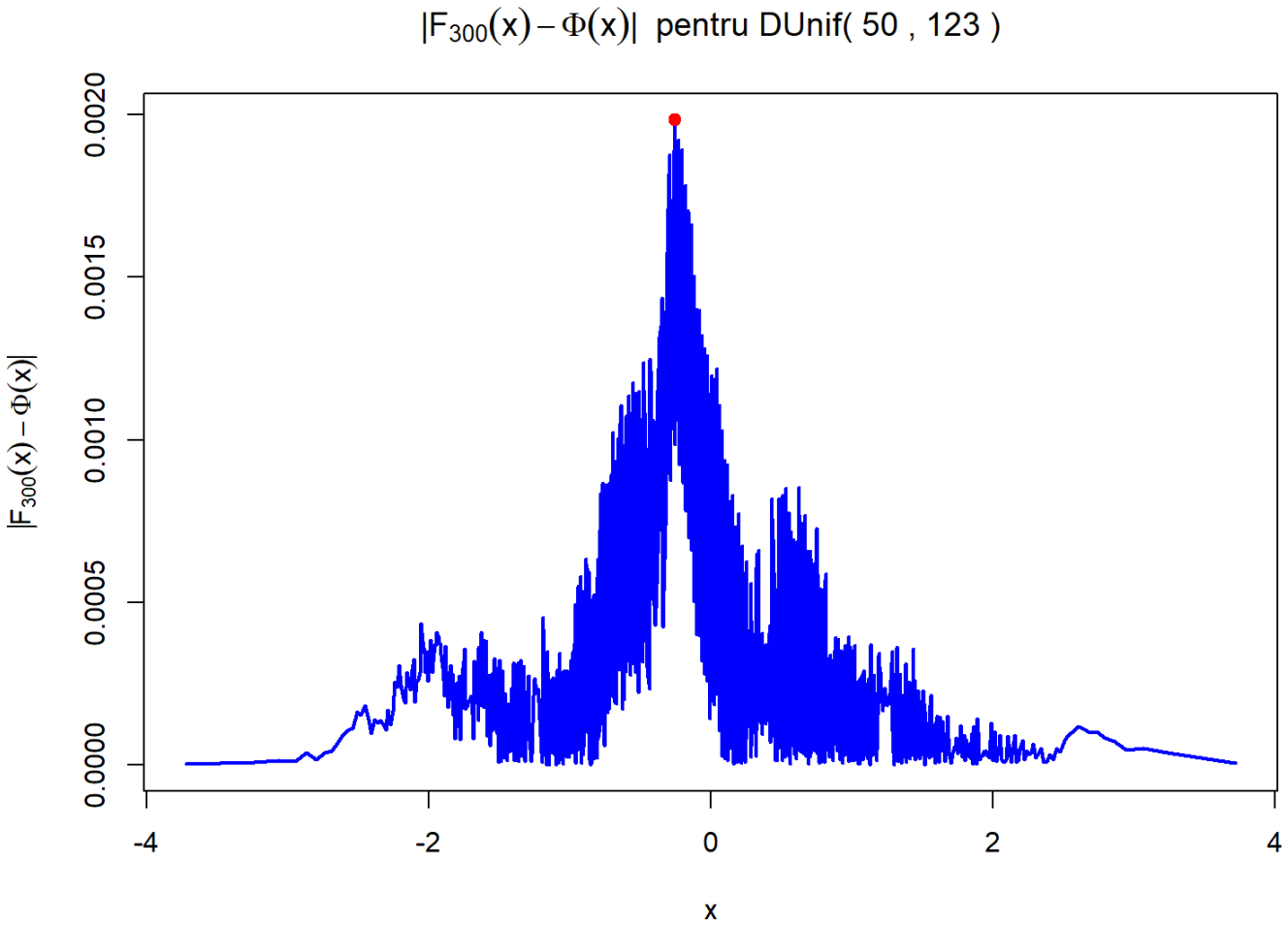
```
sup_dunif(n, a, b)
```

```
## [1] 0.0007966843
```

```
dataframe_dunif(n_valori, a, b)
```

```
##        n   margine_BE
## 1  1e+01 0.1950261864
## 2  2e+01 0.1379043389
## 3  5e+01 0.0872183620
## 4  1e+02 0.0616726952
## 5  2e+02 0.0436091810
## 6  5e+02 0.0275808678
## 7  1e+03 0.0195026186
## 8  5e+03 0.0087218362
## 9  1e+04 0.0061672695
## 10 5e+04 0.0027580868
## 11 1e+05 0.0019502619
## 12 5e+05 0.0008721836
## 13 1e+06 0.0006167270
```

```
plot_cdf_diferenta_dunif(n, a, b)
```




```
# Exercițiul 8
#a) definim C_n
C_n <- sup_dunif(n, a, b) / margine_dunif(n, a, b, C = 1)

#b) folosim deja metoda Monte-Carlo în funcția sup_binom
n_valori <- c(30, 50, 100, 200, 500)

calculeaza_Cn <- function(n, a, b, M = 5000, grid_size = 2000, q_low = 1e-4, q_high = 1 - 1e-4, C = 1)
{
  sup_dunif(n, a, b, M, grid_size, q_low, q_high) / margine_dunif(n, a, b, C)
}
valori_Cn_n <- sapply(n_valori, function(n) calculeaza_Cn(n, a, b))
data.frame(
  n = n_valori,
  C_n = valori_Cn_n
)
```

```
##     n        C_n
## 1  30 0.05761429
## 2  50 0.07400447
## 3 100 0.09366503
## 4 200 0.19848412
## 5 500 0.20092439
```

```r
#c) variem, pe rând, n (păstrând b-a constant), apoi b-a (păstrând n constant)
n_valori <- c(30, 50, 100, 200, 500, 1000, 5000, 10000, 15000, 20000)
valori_Cn_n <- sapply(n_valori, function(n) calculeaza_Cn(n, a, b))
data.frame(
  n = n_valori,
  C_n = valori_Cn_n
)
```

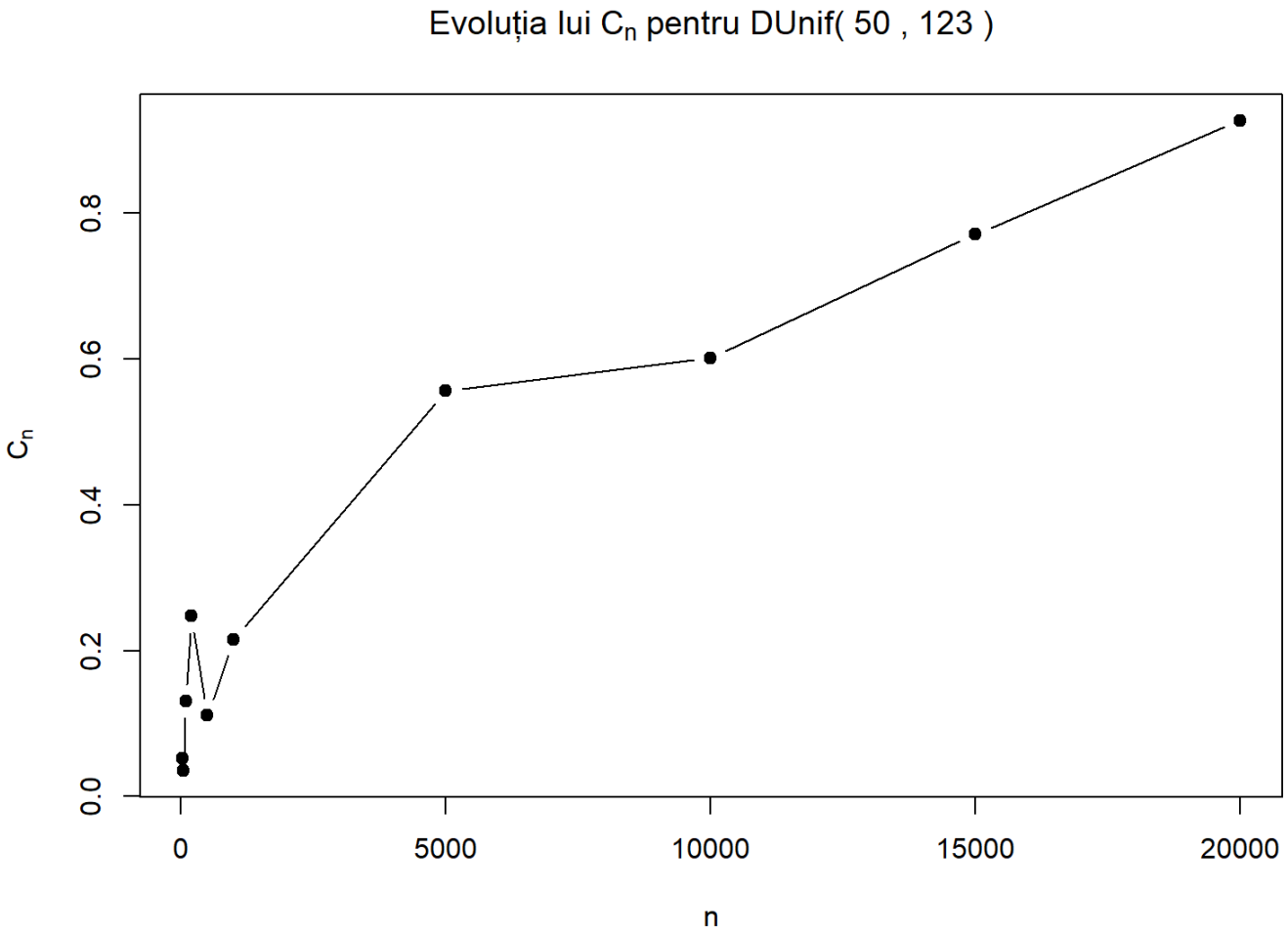
```
##        n        C_n
## 1     30 0.05261747
## 2     50 0.03535752
## 3    100 0.13093771
## 4    200 0.24717066
## 5    500 0.11115063
## 6   1000 0.21576108
## 7   5000 0.55660552
## 8  10000 0.60124243
## 9  15000 0.77080147
## 10 20000 0.92742986
```

```r
centru <- (a + b) / 2
l_valori <- seq(1, 1e+3, length.out = 100)
valori_Cn_l <- sapply(l_valori, function(l) {
  a <- centru - floor(l / 2)
  b <- centru + ceiling (l / 2)
  calculeaza_Cn(n, a, b)
  })
data.frame(
  delta_centru = l_valori,
  C_n = valori_Cn_l
)
```

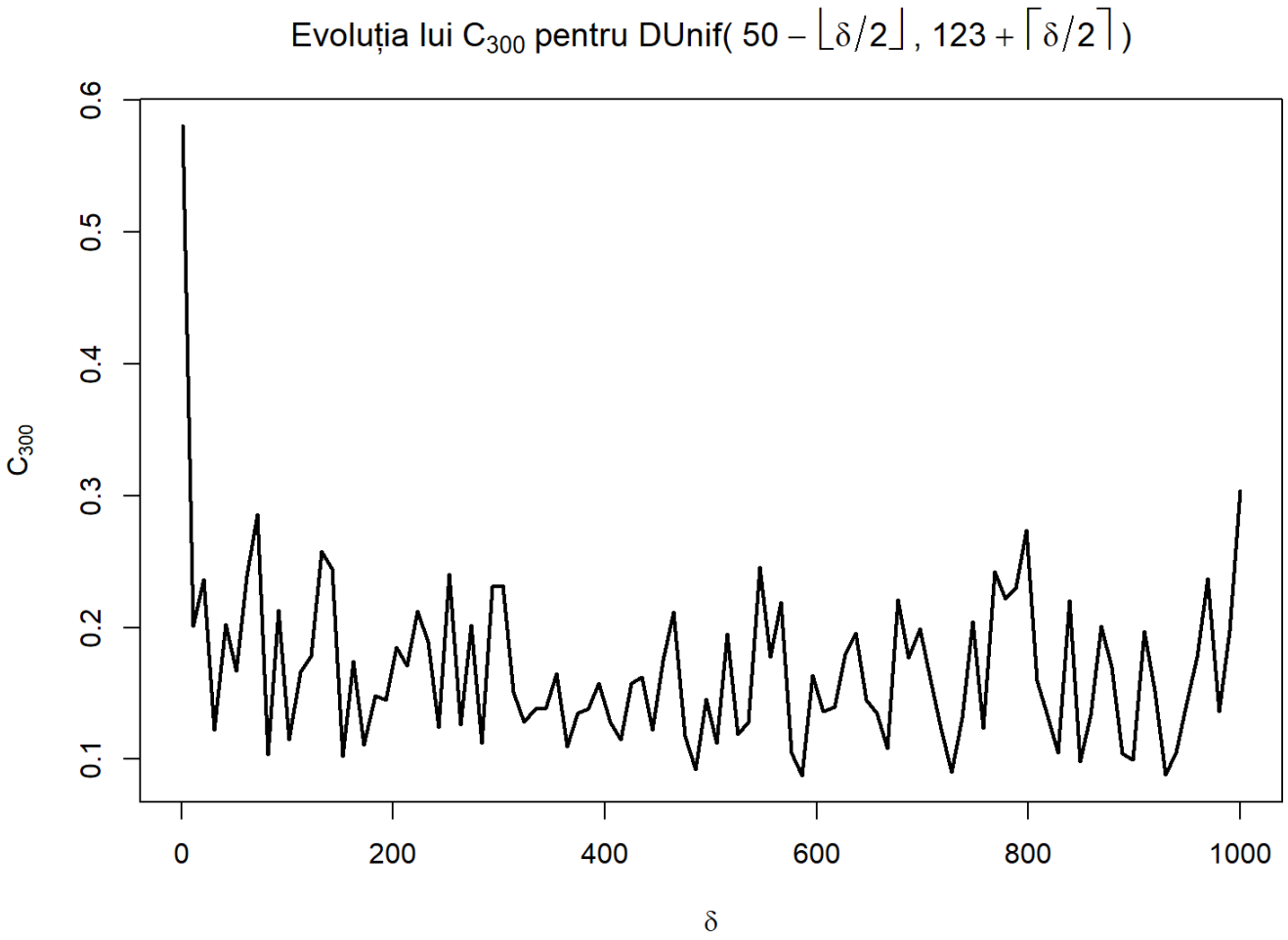
```
##     delta_centru        C_n
## 1        1.00000 0.58086260
## 2       11.09091 0.20071258
## 3       21.18182 0.23660041
## 4       31.27273 0.12211612
## 5       41.36364 0.20259277
## 6       51.45455 0.16682694
## 7       61.54545 0.23901260
## 8       71.63636 0.28605261
## 9       81.72727 0.10389100
## 10      91.81818 0.21279296
## 11     101.90909 0.11472888
## 12     112.00000 0.16657569
## 13     122.09091 0.17803947
## 14     132.18182 0.25797817
## 15     142.27273 0.24435540
## 16     152.36364 0.10232999
## 17     162.45455 0.17416432
## 18     172.54545 0.11101934
## 19     182.63636 0.14859716
## 20     192.72727 0.14505645
## 21     202.81818 0.18478595
## 22     212.90909 0.17115344
## 23     223.00000 0.21208482
## 24     233.09091 0.18910628
## 25     243.18182 0.12409175
## 26     253.27273 0.24044275
## 27     263.36364 0.12655285
## 28     273.45455 0.20152010
## 29     283.54545 0.11258699
## 30     293.63636 0.23079340
## 31     303.72727 0.23185385
## 32     313.81818 0.15129175
## 33     323.90909 0.12853355
## 34     334.00000 0.13863994
## 35     344.09091 0.13897874
## 36     354.18182 0.16476503
## 37     364.27273 0.10952117
## 38     374.36364 0.13487658
## 39     384.45455 0.13859443
## 40     394.54545 0.15797749
## 41     404.63636 0.12804508
## 42     414.72727 0.11475438
## 43     424.81818 0.15794674
## 44     434.90909 0.16237161
## 45     445.00000 0.12261828
## 46     455.09091 0.17535410
## 47     465.18182 0.21166699
## 48     475.27273 0.11757838
## 49     485.36364 0.09263518
## 50     495.45455 0.14593126
## 51     505.54545 0.11208893
## 52     515.63636 0.19477976
## 53     525.72727 0.11908042
## 54     535.81818 0.12803958
## 55     545.90909 0.24570526
## 56     556.00000 0.17796727
## 57     566.09091 0.21918793
## 58     576.18182 0.10513606
## 59     586.27273 0.08760460
```

```
## 60     596.36364 0.16362271
## 61     606.45455 0.13662250
## 62     616.54545 0.13997884
## 63     626.63636 0.17962537
## 64     636.72727 0.19546664
## 65     646.81818 0.14517476
## 66     656.90909 0.13473752
## 67     667.00000 0.10808683
## 68     677.09091 0.22136626
## 69     687.18182 0.17695600
## 70     697.27273 0.19927651
## 71     707.36364 0.16011555
## 72     717.45455 0.12292758
## 73     727.54545 0.09065541
## 74     737.63636 0.13163727
## 75     747.72727 0.20407489
## 76     757.81818 0.12341181
## 77     767.90909 0.24214328
## 78     778.00000 0.22196516
## 79     788.09091 0.22991714
## 80     798.18182 0.27382713
## 81     808.27273 0.15991675
## 82     818.36364 0.13305129
## 83     828.45455 0.10487985
## 84     838.54545 0.22015357
## 85     848.63636 0.09829655
## 86     858.72727 0.13392505
## 87     868.81818 0.20092793
## 88     878.90909 0.16987638
## 89     889.00000 0.10457836
## 90     899.09091 0.09979062
## 91     909.18182 0.19705798
## 92     919.27273 0.15140579
## 93     929.36364 0.08804741
## 94     939.45455 0.10495187
## 95     949.54545 0.14243596
## 96     959.63636 0.17782301
## 97     969.72727 0.23730878
## 98     979.81818 0.13662905
## 99     989.90909 0.19792907
## 100   1000.00000 0.30398481
```

```
# d)
plot(
  n_valori, valori_Cn_n,
  type = "b",
  pch = 19,
  xlab = "n",
  ylab = expression(C[n]),
  main = bquote("Evoluția lui" ~ C[n] ~ "pentru DUnif("~.(a)~","~.(b)~")")
)
```




```
plot(
  l_valori, valori_Cn_l,
  type = "l",
  lwd = 2,
  xlab = bquote(delta),
  ylab = bquote(C[.(n)]),
  main = bquote("Evoluția lui" ~ C[.(n)] ~ "pentru DUnif(" ~
                 .(a) ~ - ~ group(lfloor, delta/2, rfloor) ~ "," ~
                 .(b) ~ + ~ group(lceil, delta/2, rceil) ~ ")")
)
```


Evoluția lui C_{300} pentru DUnif($50 - \lfloor \delta/2 \rfloor$, $123 + \lceil \delta/2 \rceil$)
C_{300}
0.1
0.2
0.3
0.4
0.5
0.6
0
200
400
600
800
1000
δ

8. Uniforma Continuă

```
# Problema 7

# Uniformă continuă pe intervalul [a, b]

# Funcție ce calculează rho = E[|X-mu|^3]
f <- function(x, a, b){
  mu <- (a + b) / 2

  abs(x - mu)^3 * dunif(x, min = a, max = b)
}

rho_cunif <- function(a, b) {
  integrate(function(x) f(x, a, b), a, b)$value
}

# Funcție ce calculează marginea Berry-Esseen, dată de inegalitate
margine_cunif <- function(n, a, b, C = 0.4748) {
  mu <- (a + b) / 2
  sigma <- (b - a) / sqrt(12)
  rho <- rho_cunif(a, b)

  C * rho / (sigma^3 * sqrt(n))
}

# Funcție ce aproximează convenabil supremumul din inegalitatea Berry-Esseen
sup_cunif <- function(n, a, b,
                      M = 1000000,
                      grid_size = 2000,
                      q_low = 1e-4,
                      q_high = 1 - 1e-4) {
  mu <- (a + b) / 2
  sigma <- (b - a) / sqrt(12)

  X <- matrix(
    runif(M * n, min = a, max = b),
    nrow = M,
    ncol = n
  )
  X_bar <- rowMeans(X)

  Z <- sqrt(n) * (X_bar - mu) / sigma

  probs <- seq(q_low, q_high, length.out = grid_size)
  x_grid <- qnorm(probs)

  cdf_empirica <- sapply(
    x_grid,
    function(x) mean(Z <= x)
  )
  cdf_normala <- pnorm(x_grid)

  max(abs(cdf_empirica - cdf_normala))
}

# Funcție ce construiește dataframe-ul cu marginile Berry-Esseen vs. volumul eșantionului
dataframe_cunif <- function(n_valori, a, b, C = 0.4748) {
  margini <- sapply(
    n_valori,
```

```r
    function(n) margine_cunif(n, a, b, C)
  )

  data.frame(
    n = n_valori,
    margine_BE = margini
  )
}

# Funcție ce reprezintă dependența x vs.
plot_cdf_diferenta_cunif <- function(n, a, b,
                                     M = 1000000,
                                     grid_size = 2000,
                                     q_low = 1e-4,
                                     q_high = 1 - 1e-4) {
  mu <- (a + b) / 2
  sigma <- (b - a) / sqrt(12)

  X <- matrix(
    runif(M * n, min = a, max = b),
    nrow = M,
    ncol = n
  )
  X_bar <- rowMeans(X)

  Z <- sqrt(n) * (X_bar - mu) / sigma

  probs <- seq(q_low, q_high, length.out = grid_size)
  x_grid <- qnorm(probs)

  cdf_empirica <- sapply(
    x_grid,
    function(x) mean(Z <= x)
  )
  cdf_normala <- pnorm(x_grid)

  diff_abs <- abs(cdf_empirica - cdf_normala)

  plot(
    x_grid, diff_abs,
    type = "l",
    lwd = 2,
    col = "blue",
    xlab = expression(x),
    ylab = bquote(abs(F[.(n)](x) - Phi(x))),
    main = bquote("|" * F[.(n)](x) - Phi(x) * "|" ~~ "pentru Unif("~.(a)~","~.(b)~")")
  )

  max_idx <- which.max(diff_abs)
  points(x_grid[max_idx], diff_abs[max_idx], col = "red", pch = 19)

  invisible(
    list(
      x = x_grid,
      diff = diff_abs,
      sup_numeric = max(diff_abs)
    )
  )
}

# Parametri
```

```
n <- 300   #volumul eșantionului
a <- 25    #Unif([a, b])
b <- 150   #Unif([a, b])
n_valori <- c(10, 20, 50, 100, 200, 500, 1000, 5000, 10000, 50000, 100000, 500000, 1000000)

# Linii de cod pentru execuție
rho_cunif(a, b)
```

```
## [1] 61035.16
```

```
margine_cunif(n, a, b)
```

```
## [1] 0.03561
```

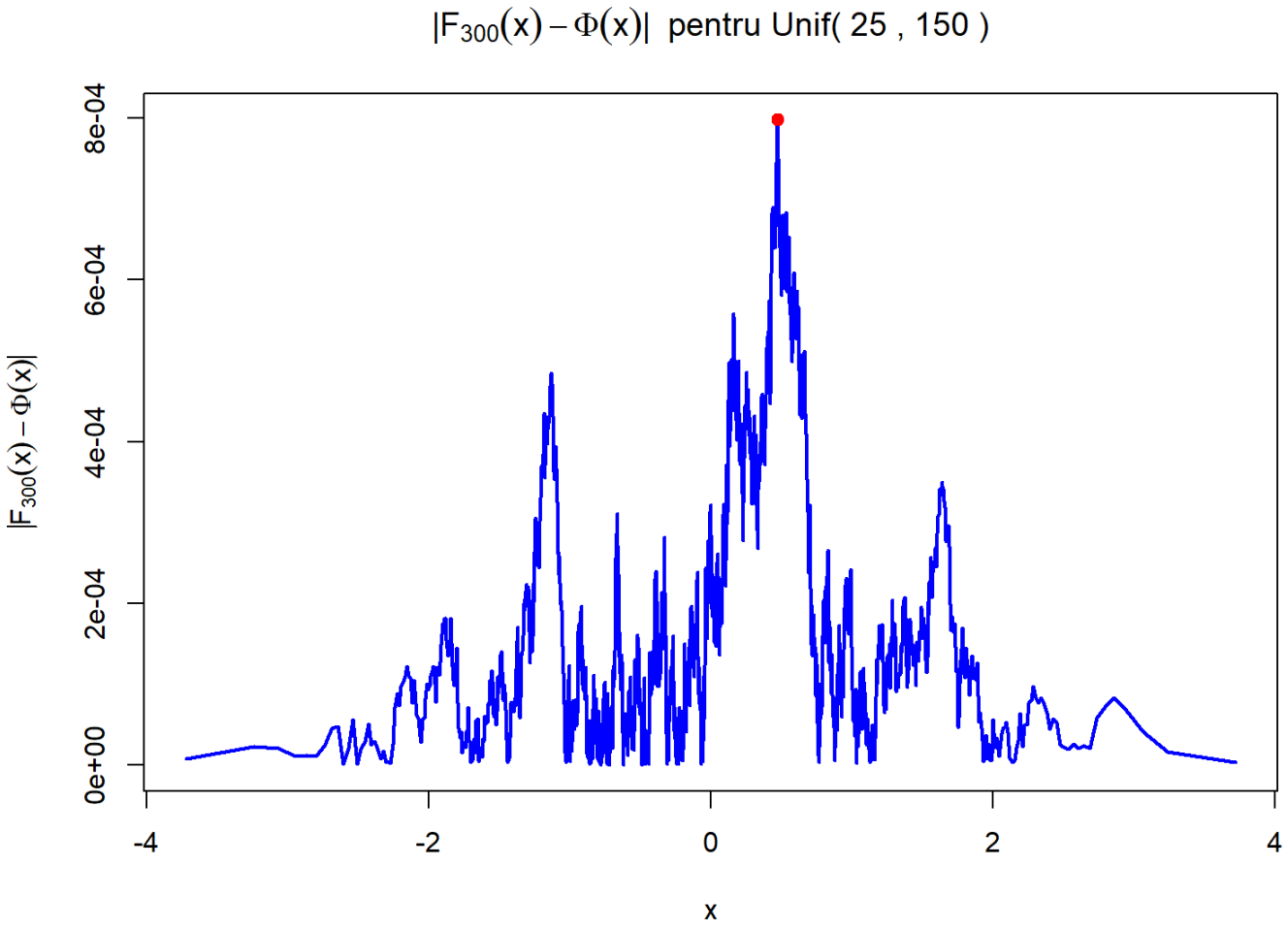
```
sup_cunif(n, a, b)
```

```
## [1] 0.0007322706
```

```
dataframe_cunif(n_valori, a, b)
```

```
##        n   margine_BE
## 1  1e+01 0.1950440027
## 2  2e+01 0.1379169370
## 3  5e+01 0.0872263297
## 4  1e+02 0.0616783293
## 5  2e+02 0.0436131649
## 6  5e+02 0.0275833874
## 7  1e+03 0.0195044003
## 8  5e+03 0.0087226330
## 9  1e+04 0.0061678329
## 10 5e+04 0.0027583387
## 11 1e+05 0.0019504400
## 12 5e+05 0.0008722633
## 13 1e+06 0.0006167833
```

```
plot_cdf_diferenta_cunif(n, a, b)
```




```
# Exercițiul 8
#a) definim C_n
C_n <- sup_cunif(n, a, b) / margine_cunif(n, a, b, C = 1)

#b) folosim deja metoda Monte-Carlo în funcția sup_binom
n_valori <- c(30, 50, 100, 200, 500)

calculeaza_Cn <- function(n, a, b, M = 5000, grid_size = 2000, q_low = 1e-4, q_high = 1 - 1e-4, C = 1)
{
  sup_cunif(n, a, b, M, grid_size, q_low, q_high) / margine_cunif(n, a, b, C)
}
valori_Cn_n <- sapply(n_valori, function(n) calculeaza_Cn(n, a, b))
data.frame(
  n = n_valori,
  C_n = valori_Cn_n
)
```

```
##     n        C_n
## 1  30 0.03659215
## 2  50 0.06926619
## 3 100 0.11140232
## 4 200 0.09767842
## 5 500 0.25482426
```

```r
#c) variem, pe rând, n (păstrând b-a constant), apoi b-a (păstrând n constant)
n_valori <- c(30, 50, 100, 200, 500, 1000, 5000, 10000, 15000, 20000)
valori_Cn_n <- sapply(n_valori, function(n) calculeaza_Cn(n, a, b))
data.frame(
  n = n_valori,
  C_n = valori_Cn_n
)
```

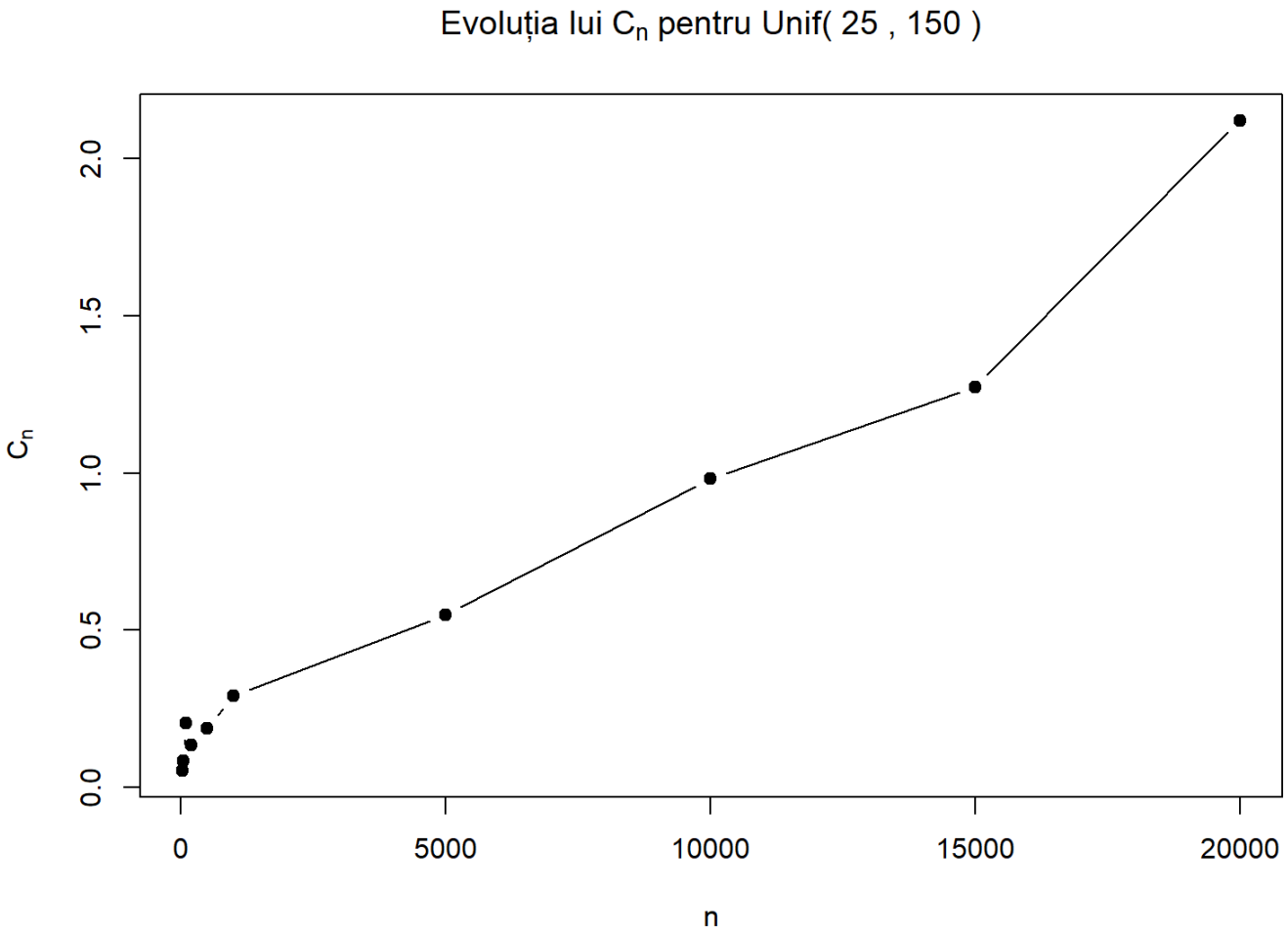
```
##        n        C_n
## 1     30 0.05318933
## 2     50 0.08381037
## 3    100 0.20453160
## 4    200 0.13405793
## 5    500 0.18876289
## 6   1000 0.29168152
## 7   5000 0.54818957
## 8  10000 0.98299154
## 9  15000 1.27345250
## 10 20000 2.12288022
```

```r
centru <- (a + b) / 2
l_valori <- seq(1, 1e+3, length.out = 100)
valori_Cn_l <- sapply(l_valori, function(l) {
  a <- centru - l / 2
  b <- centru + l / 2
  calculeaza_Cn(n, a, b)
})
data.frame(
  delta_centru = l_valori,
  C_n = valori_Cn_l
)
```

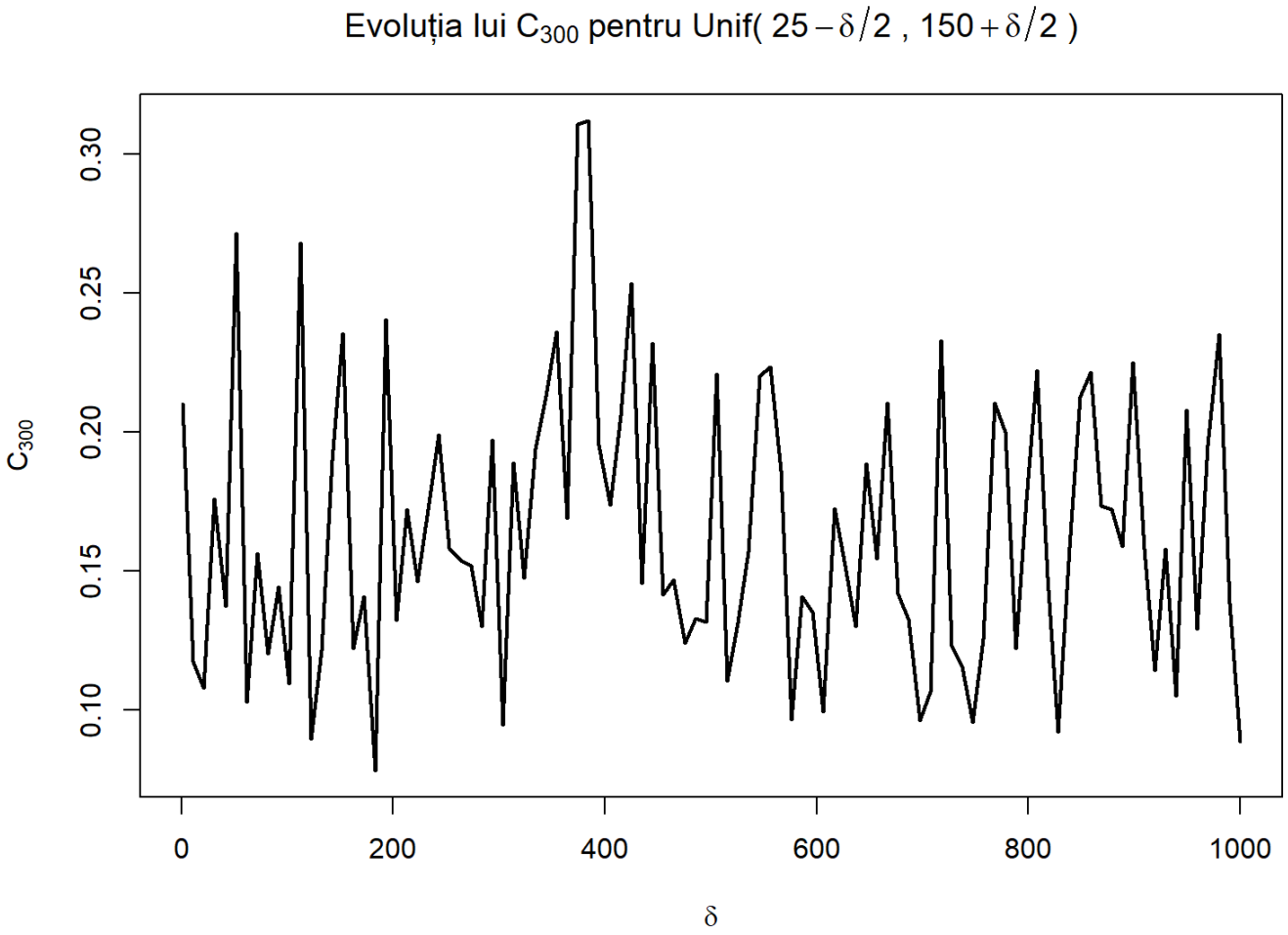
```
##     delta_centru        C_n
## 1        1.00000 0.21023112
## 2       11.09091 0.11744606
## 3       21.18182 0.10776655
## 4       31.27273 0.17584726
## 5       41.36364 0.13714657
## 6       51.45455 0.27147240
## 7       61.54545 0.10286543
## 8       71.63636 0.15641087
## 9       81.72727 0.12005536
## 10      91.81818 0.14420477
## 11     101.90909 0.10932466
## 12     112.00000 0.26814140
## 13     122.09091 0.08936402
## 14     132.18182 0.12141204
## 15     142.27273 0.18972886
## 16     152.36364 0.23539703
## 17     162.45455 0.12215908
## 18     172.54545 0.14078306
## 19     182.63636 0.07803702
## 20     192.72727 0.24041554
## 21     202.81818 0.13221744
## 22     212.90909 0.17210939
## 23     223.00000 0.14616842
## 24     233.09091 0.17211072
## 25     243.18182 0.19918159
## 26     253.27273 0.15816108
## 27     263.36364 0.15378289
## 28     273.45455 0.15171252
## 29     283.54545 0.13014307
## 30     293.63636 0.19724062
## 31     303.72727 0.09445990
## 32     313.81818 0.18903252
## 33     323.90909 0.14728231
## 34     334.00000 0.19394030
## 35     344.09091 0.21298849
## 36     354.18182 0.23613073
## 37     364.27273 0.16896382
## 38     374.36364 0.31078139
## 39     384.45455 0.31227547
## 40     394.54545 0.19539303
## 41     404.63636 0.17360080
## 42     414.72727 0.20599700
## 43     424.81818 0.25345273
## 44     434.90909 0.14554411
## 45     445.00000 0.23211939
## 46     455.09091 0.14137669
## 47     465.18182 0.14682741
## 48     475.27273 0.12384926
## 49     485.36364 0.13286577
## 50     495.45455 0.13163315
## 51     505.54545 0.22098049
## 52     515.63636 0.11022645
## 53     525.72727 0.13142238
## 54     535.81818 0.15700984
## 55     545.90909 0.22023078
## 56     556.00000 0.22330432
## 57     566.09091 0.18621111
## 58     576.18182 0.09650092
## 59     586.27273 0.14076572
```

```
## 60     596.36364 0.13497349
## 61     606.45455 0.09930232
## 62     616.54545 0.17236151
## 63     626.63636 0.15182324
## 64     636.72727 0.12991896
## 65     646.81818 0.18864966
## 66     656.90909 0.15424712
## 67     667.00000 0.21030582
## 68     677.09091 0.14173420
## 69     687.18182 0.13254827
## 70     697.27273 0.09596065
## 71     707.36364 0.10669802
## 72     717.45455 0.23283175
## 73     727.54545 0.12301017
## 74     737.63636 0.11536768
## 75     747.72727 0.09541771
## 76     757.81818 0.12564215
## 77     767.90909 0.21047390
## 78     778.00000 0.19993263
## 79     788.09091 0.12223645
## 80     798.18182 0.17567250
## 81     808.27273 0.22219576
## 82     818.36364 0.14773720
## 83     828.45455 0.09212006
## 84     838.54545 0.15854527
## 85     848.63636 0.21265499
## 86     858.72727 0.22150075
## 87     868.81818 0.17352876
## 88     878.90909 0.17218676
## 89     889.00000 0.15894814
## 90     899.09091 0.22491579
## 91     909.18182 0.15791562
## 92     919.27273 0.11432316
## 93     929.36364 0.15800500
## 94     939.45455 0.10508521
## 95     949.54545 0.20795131
## 96     959.63636 0.12921061
## 97     969.72727 0.19513557
## 98     979.81818 0.23526630
## 99     989.90909 0.13811239
## 100   1000.00000 0.08867034
```

```
# d)
plot(
  n_valori, valori_Cn_n,
  type = "b",
  pch = 19,
  xlab = "n",
  ylab = expression(C[n]),
  main = bquote("Evoluția lui" ~ C[n] ~ "pentru Unif("~.(a)~","~.(b)~")")
)
```




```
plot(
  l_valori, valori_Cn_l,
  type = "l",
  lwd = 2,
  xlab = bquote(delta),
  ylab = bquote(C[.(n)]),
  main = bquote("Evoluția lui" ~ C[.(n)] ~ "pentru Unif("~.(a) - delta/2 ~","~.(b) + delta / 2~")")
)
```



Exercițiul 9

a. Rolul familiei exponențiale în simplificarea estimării parametrilor. Stiind apartenenta unei distributii la familia exponentiala, avem forma generala pentru fiecare densitate/functie de masa, asta ajutandu-ne sa calculam functia de log-verosimilitate mai usor si sa eficientizam calculele in estimarea parametrului prin metoda verosimilitatii maxime.

b. Influența momentului de ordin 3 asupra vitezei de convergență în TLC. Momentul de ordinul 3 influenteaza viteza de convergenta prin inegalitatea Berry-Esseen care ne arata ca pentru rho mic, supremumul este mai mic, deci convergenta e mai rapida. De asemenea raportul rho/sigma^3 cuantifica asimetria distributiei, pentru rho mare avem distributii asimetrice si convergenta lenta din cauza cozilor grele.

c. Semnificația practică este aceea că inegalitatea Berry-Esseen este conservatoare. În practică, eroarea de aproximare normală este adesea mult mai mică decât prezice limita teoretică, ceea ce ne permite să aplicăm Teorema Limită Centrală (TLC) cu încredere chiar și pentru eșantioane moderate (n≈30−50), fără a atinge limita pesimistă a inegalității.

d. La distribuțiile discrete (Binomială, Poisson), funcția de repartiție Fn este “în trepte”. Chiar dacă distribuția este simetrică, aproximarea printr-o funcție continuă (Φ) suferă de o eroare inerentă legată de discretizare (efect de rețea), spre deosebire de distributiile continue pentru care aproximarile au fost mult mai fine.Așa cum am văzut la punctul (b), distribuțiile puternic asimetrice (Exponențiala) au cea mai slabă acuratețe pentru n mic. Diferența |Fn −Φ| rămâne semnificativă în “cozi” mult timp după ce zona centrală a devenit normală.

e. In estimare, așa cum s-a discutat la (a), structura analitică permite calculul exact și eficient al parametrilor, fără necesitatea unor metode numerice complexe. In sensul aproximarii asimptotice, membrii familiei exponențiale au momente bine definite și finite (ρ<∞). Graficele evoluției lui Cn din exercițiul 8 arată curbe cu variatie marginita și

predictibile în funcție de parametrii α,β,p. Nu există comportamente haotice. Această stabilitate sugerează că trecerea de la distribuția specifică la cea Normală (via TLC) este un proces natural și controlat de parametrii de formă ai familiei.